Implementasi Sistem Keamanan Server Linux Menggunakan Firewall dan Fail2Ban Berbasis *Log Analysis* untuk Mitigasi *Brute Force* dan *Dictionary attack* pada Layanan SSH

TUGAS AKHIR



OLEH:

Azua Ramadhan

2222026

FAKULTAS TEKNOLOGI INFORMASI

INSTITUT TEKNOLOGI BATAM

BATAM

2026

Implementasi Sistem Keamanan Server Linux Menggunakan Firewall dan Fail2Ban Berbasis *Log Analysis* untuk Mitigasi *Brute Force* dan *Dictionary attack* pada Layanan SSH

TUGAS AKHIR

DIAJUKAN SEBAGAI SALAH SATU SYARAT UNTUK MENDAPATKAN GELAR SARJANA (STRATA-1) PADA PROGRAM STUDI TEKNIK KOMPUTER



OLEH:

Azua Ramadhan

2222026

FAKULTAS TEKNOLOGI INFORMASI

INSTITUT TEKNOLOGI BATAM

BATAM

2026

HALAMAN PENGESAHAN

Tugas akhir ini berjudul Implementasi Sistem Keamanan Server Linux Menggunakan Firewall dan Fail2Ban Berbasis *Log Analysis* untuk Mitigasi *Brute Force* dan *Dictionary attack* pada Layanan SSH ditulis dan diserahkan oleh **Azua Ramadhan – 2222026** sebagai salah satu syarat untuk meraih gelar **Sarjana Teknik Komputer,** telah diperiksa dan oleh karena itu direkomendasikan untuk disahkan dan diterima.

Tanggal: 21 September 2026	**Zainul Munir, S.T., Me.TC** NUPTK. 5146758659130193 Pembimbing I
Tanggal: 21 September 2026	**M. Abrar Masril, M.Kom** NUPTK. 3159770671130343 Pembimbing II

PENGUJI

Disahkan oleh Penguji pada ujian Tugas Akhir
21/09/26

Tanggal: 21 September 2026	**Mhd. Adi Setiawan Aritonang, M.Kom** NUPTK. 1235770671130283 Ketua Penguji
Tanggal: 21 September 2026	**Luki Hernando, M.Kom** NUPTK. 7536770671130292 Anggota Penguji

Diterima dan disahkan sebagai salah satu syarat untuk meraih gelar **Sarjana Teknik Komputer**

Tanggal: 21 September 2026	**M. Abrar Masril, M.Kom** NUPTK. 3159770671130343 Ketua Program Studi Teknik Komputer
Tanggal: 21 September 2026	**Assoc. Prof. Dr. Ir. Ririt Dwiputri Permatasari, S.T., M.SI** NUPTK. 2337759660230223 Dekan Fakultas Teknologi Informasi

KATA PENGANTAR

Puji syukur atas kehadiran Allah Ta'ala atas rahmat dan bantuannya, sehingga penulis dapat menyelesaikan penulisan tugas akhir dengan judul "Implementasi Sistem Keamanan Server Linux Menggunakan Firewall dan Fail2Ban Berbasis *Log Analysis* untuk Mitigasi *Brute Force* dan *Dictionary attack* pada Layanan SSH" yang merupakan salah satu persyaratan untuk menyelesaikan program studi Strata satu (S1) pada Program Studi Teknik Komputer di Institut Teknologi Batam. Penulis menyadari bahwa masih banyak kekurangan dalam penyusunan tugas akhir. Sehingga, penulis mengharapkan kritik dan saran yang membangun untuk membantu menyempurnakan tugas akhir berikutnya. Penulis menyadari tugas akhir ini tidak mungkin terselesaikan dengan baik tanpa bantuan, dorongan, dan dukungan dari banyak pihak. Oleh karena itu dengan segala hormat dan kerendahan hati, pada kesempatan ini penulis mengucapkan terimakasih kepada Allah Ta'ala yang telah memberikan rahmat dan kasih sayangnya. Orang tua tersayang, Mama dan Papa serta keluarga besar yang mendukung dan mendoakan penulis selama ini, terutama dari penulis melaksanakan perkuliahan sampai penulisan tugas akhir ini. Terima kasih atas dukungan, nasihat, serta kasih sayang diberikan sehingga penulis bisa menyelesaikan tugas akhir ini dengan baik. Penyusunan tugas akhir ini tidak dapat berjalan tanpa bantuan dari berbagai pihak. Oleh karena itu, penulis ingin mengucapkan terima kasih kepada:

1. Bapak Prof. Dr.-Ing. Ir. H. Hairul Abral. selaku Rektor Institut Teknologi Batam.
2. Ibu Assoc. Prof. Dr. Ir. Ririt Dwiputri Permatasari, S.T., M.SI. selaku Dekan Fakultas Teknologi Informasi.
3. Ibu Assoc. Prof. Vitri Aprilla Handayani, M.Si. selaku Kepala Sekretariat Institusi Institut Teknologi Batam.
4. Bapak Luki Hernando, S.Kom., M.Kom. selaku Sekretaris Dekan Fakultas Teknologi Informasi.
5. Bapak M. Abrar Masril, M.Kom. selaku Kepala Program Studi Teknik Komputer.
6. Bapak Zainul Munir, S.T., Me.TC. selaku Pembimbing I tugas akhir yang telah memberikan arahan, ilmu, dan bimbingan kepada penulis dalam menyelesaikan tugas akhir.
7. Bapak M. Abrar Masril, M.Kom. selaku Pembimbing II tugas akhir yang telah memberi banyak bantuan dalam penulisan tugas akhir dan saran kepada penulis untuk menghasilkan tulisan yang lebih baik.

8. Bapak Mhd. Adi Setiawan Aritonang, M.Kom. selaku dosen Penguji 1 yang telah memberikan kritik, dan masukan yang membangun sehingga penulis dapat menyempurnakan tugas akhir ini.
9. Bapak Luki Hernando, S.Kom., M.Kom. selaku dosen Penguji 2 yang telah memberikan kritik, saran, dan masukan yang membangun sehingga penulis dapat menyempurnakan tugas akhir ini.
10. Seluruh dosen dan teman-teman seperjuangan Program Studi Teknik Komputer angkatan 2022.
11. Teman-teman saya Nazwan, Hilmy, Surgiwe, Vera, Evi, dan Roland yang selalu memberikan dukungan luar biasa kepada saya dari awal saya mengikuti perkuliahan hingga sampai tahap akhir perkuliahan.

Penulis juga ingin menyampaikan permohonan maaf jika terdapat kekurangan dalam penulisan tugas akhir ini. Kritik, saran, dan masukan dari pembaca akan sangat penulis hargai untuk perbaikan di masa yang akan datang. Semoga tugas akhir ini dapat memberikan manfaat dan kontribusi ilmiah yang bermanfaat bagi perkembangan ilmu pengetahuan di bidang Teknik Komputer dan masyarakat pada umumnya. Akhir kata, penulis ingin dapat membantu pembaca. Penulis mengucapkan rasa syukur dan terima kasih kepada Allah Ta'ala dan semua orang yang telah menawarkan dan membantu, semoga Allah Ta'ala membalas kebaikan dan rahmatnya kepada penulis dan pembaca, Aamiin.

Batam, 19 September 2026

Azua Ramadhan
2222026

ABSTRAK

Perkembangan teknologi informasi mendorong meningkatnya penggunaan server Linux sebagai infrastruktur layanan jaringan, namun layanan Secure Shell (SSH) yang digunakan untuk administrasi jarak jauh rentan terhadap serangan brute force dan dictionary attack. Penelitian ini bertujuan mengimplementasikan sistem keamanan server Linux yang mengintegrasikan Firewall, Fail2Ban, Auditd, website monitoring berbasis Flask, dan Telegram Bot dalam satu kesatuan sistem. Metode yang digunakan adalah eksperimen dengan pendekatan kuantitatif pada lingkungan virtual machine, di mana serangan disimulasikan menggunakan Hydra dari Kali Linux terhadap server Ubuntu Server. Fail2Ban menganalisis berkas /var/log/auth.log menggunakan regex matching untuk mendeteksi pola kegagalan autentikasi, kemudian melakukan pemblokiran otomatis melalui firewall. Hasil pengujian menunjukkan bahwa dari 10 percobaan serangan (5 brute force attack dan 5 dictionary attack), seluruhnya berhasil diblokir dengan tingkat keberhasilan 100%. Rata-rata response time pemblokiran adalah 13,7 detik, dengan dictionary attack lebih cepat (12,8 detik) dibandingkan brute force attack (14,6 detik). Notification rate mencapai 100% dengan rata-rata delay notifikasi 2,2 detik, sedangkan accuracy sistem sebesar 85,71% yang dipengaruhi oleh 1 false positive dan 3 false negative. Auditd berhasil mencatat aktivitas akses terhadap berkas sensitif, sedangkan website monitoring mampu menampilkan status server, aktivitas login SSH, daftar IP yang diblokir, dan event Auditd secara real-time. Sistem ini terbukti efektif dalam memitigasi serangan brute force dan dictionary attack, meskipun masih memiliki keterbatasan pada skenario evasion attack seperti low-and-slow brute force.

Kata Kunci: Auditd, Brute Force, Dictionary Attack, Fail2Ban, Firewall, Keamanan Server, Log Analysis, SSH, Telegram Bot, Website Monitoring.

ABSTRACT

The rapid development of information technology has driven the increasing use of Linux servers as network service infrastructure, yet the Secure Shell (SSH) service used for remote administration remains vulnerable to brute force and dictionary attacks. This research aims to implement a Linux server security system that integrates Firewall, Fail2Ban, Auditd, a Flask-based monitoring website, and Telegram Bot into a unified system. The method used is an experiment with a quantitative approach in a virtual machine environment, where attacks are simulated using Hydra from Kali Linux against an Ubuntu Server. Fail2Ban analyzes the /var/log/auth.log file using regex matching to detect authentication failure patterns, then performs automatic blocking through the firewall. The test results show that out of 10 attack attempts (5 brute force attacks and 5 dictionary attacks), all were successfully blocked with a 100% success rate. The average blocking response time is 13.7 seconds, with dictionary attacks being faster (12.8 seconds) compared to brute force attacks (14.6 seconds). The notification rate reached 100% with an average notification delay of 2.2 seconds, while the system accuracy was 85.71%, influenced by 1 false positive and 3 false negatives. Auditd successfully recorded access activities to sensitive files, while the monitoring website was able to display server status, SSH login activities, blocked IP lists, and Auditd events in real-time. This system has proven effective in mitigating brute force and dictionary attacks, although it still has limitations in evasion attack scenarios such as low-and-slow brute force.

Keywords: *Auditd, Brute Force, Dictionary Attack, Fail2Ban, Firewall, Log Analysis, Server Security, SSH, Telegram Bot, Website Monitoring.*

DAFTAR ISI

KATA PENGANTAR ii
ABSTRAK iii
ABSTRACT iv
DAFTAR ISI v
DAFTAR TABEL ix
DAFTAR GAMBAR x
BAB I PENDAHULUAN 1
1.1 Latar Belakang Masalah 1
1.2 Rumusan Permasalahan 3
1.3 Batasan Masalah 3
1.4 Tujuan Penelitian 4
1.5 Manfaat Penelitian 5
1.6 Sistematika Penulisan 6
BAB II KAJIAN PUSTAKA 7
2.1 Keamanan Jaringan Komputer 7
2.2 Server Linux 7
2.3 Firewall 8
2.4 Fail2Ban 9
2.5 *Secure Shell* (SSH) 10
2.6 Brute force attack 11
2.7 *Log Analysis Method* 12
2.8 Intrusion Detection System (IDS) 13
2.9 Intrusion prevention system (IPS) 14
2.10 System hardening 15
2.11 *Real-time* Monitoring & Notifikasi Sistem 15
2.12 *Response time* 16
2.13 *Centralized log management* (Manajemen Log Terpusat) 16
2.14 Hydra 17
2.15 *Dictionary Attack* 17
2.16 Website Monitoring Server 19

2.17 Flask 19
2.18 Telegram Bot 20
2.19 Penelitian Terdahulu 20
2.20 State of The Art 23
BAB III METODE PENELITIAN 26
3.1 Metode Penelitian 26
3.2 Kerangka Penelitian 27
3.3 Rancangan Penelitian 30
3.2.1 Penelitian Literatur 31
3.2.2 Perancangan Sistem 31
3.2.3 Implementasi Firewall dan Fail2Ban 32
3.2.4 Implementasi Serangan Brute Force dan Dictionary Attack 32
3.2.5 Implementasi *Website Monitoring* dan Telegram Bot 33
3.2.6 Analisis *Log* Sistem 33
3.2.7 Pengujian dan Analisa Sistem 34
3.4 Bahan dan Peralatan Yang Digunakan 35
BAB IV ANALISIS DAN PERANCANGAN 37
4.1 Analisa Sistem 37
4.1.1 Blok Diagram 37
4.1.2 Analisa Sistem Berjalan dan Permasalahan 39
4.1.3 Analisa Mekanisme Serangan (*Brute Force & Dictionary Attack*) 40
4.1.4 Analisa Log Layanan SSH 41
4.1.5 Analisa Kebutuhan Perangkat Keras dan Perangkat Lunak 41
4.2 Perancangan Topologi Jaringan Simulasi 43
4.3 Perancangan Arsitektur dan Alur Sistem Keamanan 44
4.3.1 Mekanisme Log Analysis (Regex Matching) 45
4.3.2 Perancangan Aturan Firewall dan Parameter Fail2Ban 46
4.3.3 Integrasi Notifikasi Telegram Bot 47
4.4 Perancangan Sistem 47
4.4.1 *Use Case* Diagram Sistem Keamanan 48
4.4.2 *Activity Diagram* Analisis Log dan Pemblokiran 50
4.4.3 Flowchart Proses Mitigasi Serangan 53
4.4.4 Perancangan Antarmuka Website 56

4.5 Parameter Metrik Evaluasi Performa Server 57
BAB V HASIL & PEMBAHASAN 59
5.1 Lingkungan Pengujian 59
5.2 Hasil Implementasi Sistem 59
5.2.1 Implementasi Firewall 60
5.2.2 Implementasi Fail2Ban 60
5.2.3 Implementasi Telegram Bot 62
5.3 Hasil Pengujian Sistem 62
5.3.1 Pengujian Serangan Brute Force 62
5.3.2 Pengujian Standard Dictionary Attack 65
5.3.3 Perbandingan Hasil Serangan Brute Force & Dictionary Attack 67
5.3.4 Pengujian Login Sah (Pengukuran False Positive) 68
5.3.5 Pengujian *Low-and-Slow* Brute Force (Pengukuran *False Negative*) . 69
5.4 Hasil Pengujian Serangan Lainnya 71
5.4.1 Pengujian Reconnaissance & Service Enumeration 71
5.4.2 Pengujian SQL Injection 73
5.4.3 Pengujian Man-in-the-Middle (MITM) / ARP Spoofing 75
5.4.4 Pengujian Denial of Service (DoS) 77
5.4.5 Pengujian Distributed Brute Force (Multi-IP via Netns) 80
5.4.6 Pengujian Flask Web Dashboard Login Brute Force 82
5.5 Pengujian Unban IP 84
5.6 Simulasi Perolehan Kredensial Valid sebagai Celah Awal Eksploitasi 84
5.7 Simulasi Akses Tidak Sah Menggunakan Kredensial yang Diperoleh 85
5.8 Pengujian Akses Data Sensitif 86
5.9 Pengujian Auditd terhadap Akses File 86
5.10 Integrasi Auditd dengan Website 87
5.11 Hasil Pengujian Notifikasi Telegram 88
5.12 Hasil Monitoring Website 89
5.13 Analisis Hasil Pengujian Auditd 91
5.14 Analisis Perhitungan Kinerja Sistem Berdasarkan Data Pengujian 91
5.15 Analisis Dataset CSE-CIC-IDS2018 untuk Deteksi Serangan 95
5.16 Pembahasan 98
BAB VI KESIMPULAN & SARAN 100

6.1 Kesimpulan 100
6.2 Saran 100
DAFTAR PUSTAKA 102

DAFTAR TABEL

Tabel 2.1 Dataset Serangan Brute Force 12
Tabel 2.2 Dataset Dictionary Attack 18
Tabel 2.3 Perbandingan Penelitian Terdahulu dan Penelitian Saat Ini 21
Tabel 2.4 Lanjutan Perbandingan Penelitian Terdahulu dan Penelitian Saat Ini 22
Tabel 3.1 Perangkat Keras (*Hardware*) 35
Tabel 3.2 Perangkat Lunak (*Software*) 36
Tabel 4.1 Parameter Konfigurasi Jail Fail2Ban 46
Tabel 4.2 Matriks Pengujian dan Metrik Evaluasi Performa 58
Tabel 5.1 Lingkungan Pengujian 59
Tabel 5.2 Hasil Pengujian Brute Force Attack 64
Tabel 5.3 Hasil Pengujian Dictionary Attack 66
Tabel 5.4 Perbandingan Brute Force dan Dictionary Attack 67
Tabel 5.5 Hasil Pengujian Login Sah 68
Tabel 5.6 Lanjutan Tabel Hasil Pengujian Login Sah 69
Tabel 5.7 Hasil Pengujian *Low-and-Slow* Brute Force 71
Tabel 5.8 Hasil Pengujian Reconnaissance & Service Enumeration 73
Tabel 5.9 Hasil Pengujian SQL Injection 75
Tabel 5.10 Hasil Pengujian Man-in-the-Middle (MITM) / ARP Spoofing 77
Tabel 5.11 Hasil Pengujian Denial of Service (DoS) 78
Tabel 5.12 Hasil Pengujian Distributed Brute Force 82
Tabel 5.13 Hasil Pengujian Flask Web Dashboard Login Brute Force 84
Tabel 5.14 Analisa *Response Time* 92
Tabel 5.15 Analisa *Success Rate* Pemblokiran 93
Tabel 5.16 Analisa *Notification Rate* 94
Tabel 5.17 Analisa *Accuracy* 95
Tabel 5.18 Dimensi dan Struktur Data 96
Tabel 5.19 Hasil Distribusi label serangan 96
Tabel 5.20 Hasil Klasifikasi Data 98

DAFTAR GAMBAR

Gambar 2.1 Linux 7
Gambar 2.2 Firewall 8
Gambar 2.3 Fail2Ban 9
Gambar 2.4 Secure Shell (SSH) 10
Gambar 2.5 Intrusion Detection System 13
Gambar 2.6 Hydra 17
Gambar 2.7 Flask 19
Gambar 2.8 Telegram Bot 20
Gambar 2.9 State of The Art 23
Gambar 3.1 Kerangka Penelitian 27
Gambar 4.1 Blok Diagram 37
Gambar 4.2 Authentication Log 41
Gambar 4.3 Topologi Jaringan Simulasi 43
Gambar 4.4 Integrasi Notifikasi Telegram Bot 47
Gambar 4.5 Use Case Diagram 48
Gambar 4.6 Activity Diagram Analisis Log dan Pemblokiran 50
Gambar 4.7 FlowChart Proses Mitigasi Serangan 53
Gambar 4.8 Perancangan Website 56
Gambar 5.1 Implementasi Firewall 60
Gambar 5.2 Status Layanan Fail2Ban 61
Gambar 5.3 Implementasi Fail2Ban 61
Gambar 5.4 Implementasi Telegram Bot 62
Gambar 5.5 Perintah Serangan Brute Force 63
Gambar 5.6 Proses *Auth.log* Brute Force 63
Gambar 5.7 Banned IP List sshd Bruteforce 64
Gambar 5.8 Perintah Serangan Dictionary Attack 65
Gambar 5.9 Proses *Auth.log* Dictionary 65
Gambar 5.10 Banned IP List sshd Dictionary 66
Gambar 5.11 Perintah Serangan *Low-and-slow* Brute Force 70
Gambar 5.12 Proses *Auth.log* Serangan *Low-and-Slow* Brute Force 70
Gambar 5.13 Hasil Reconnaissance Server 72
Gambar 5.14 Hasil Deteksi Scanning pada Website Monitoring 72
Gambar 5.15 Proses Penyerangan sqlmap 73

Gambar 5.16 Deteksi Serangan dari Server 74
Gambar 5.17 Deteksi Serangan dari Website 74
Gambar 5.18 Proses Serangan MITM 75
Gambar 5.19 Hasil website deteksi serangan MITM 76
Gambar 5.20 Perubahan MAC pada server 76
Gambar 5.21 Proses Serangan DoS 77
Gambar 5.22 Hasil Website Serangan DoS 78
Gambar 5.23 Proses Serangan Distributed Brute Force Penyerang 1 80
Gambar 5.24 Proses Serangan Distributed Brute Force Penyerang 2 81
Gambar 5.25 Proses Auth.log Serangan Distributed Brute Force 81
Gambar 5.26 Proses Serangan Flask Web Dashboard Login Brute Force 83
Gambar 5.27 Proses Auth.log Serangan Flask Web Dashboard Login Brute Force 83
Gambar 5.28 Unban IP 84
Gambar 5.29 Serangan Kredential 85
Gambar 5.30 Login sebagai user 86
Gambar 5.31 Uji Akses Data Sensitif 86
Gambar 5.32 Pemblokiran Akses Data Sensitif 87
Gambar 5.33 Hasil Auditd terhadap Akses File ISO 87
Gambar 5.34 Tampilan Monitoring Auditd 88
Gambar 5.35 Notifikasi Telegram 88
Gambar 5.36 Dashboard 89
Gambar 5.37 Monitoring Log 89
Gambar 5.38 Firewall 90
Gambar 5.39 Fail2ban 90
Gambar 5.40 ISO Audit 91
Gambar 5.41 Visualisasi Distribusi Trafik 97

BAB I
PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan teknologi informasi yang semakin pesat mendorong meningkatnya penggunaan server berbasis Linux sebagai infrastruktur utama dalam penyediaan berbagai layanan jaringan. Linux dipilih karena memiliki tingkat stabilitas, fleksibilitas, serta kemampuan pengelolaan sumber daya yang tinggi sehingga banyak diterapkan pada lingkungan perusahaan, institusi pendidikan, maupun penyedia layanan berbasis cloud. Namun, semakin luasnya pemanfaatan server Linux juga diikuti dengan meningkatnya ancaman keamanan siber yang menargetkan layanan-layanan penting yang berjalan di dalamnya. Oleh karena itu, penerapan sistem keamanan yang mampu mendeteksi, mencegah, dan memantau aktivitas mencurigakan secara berkelanjutan menjadi kebutuhan yang tidak dapat diabaikan (Usman et al., 2024).

Salah satu layanan yang paling sering menjadi sasaran serangan adalah Secure Shell (SSH) yang digunakan sebagai media administrasi server secara jarak jauh. Walaupun SSH telah menerapkan mekanisme enkripsi yang kuat, layanan ini tetap rentan terhadap serangan brute force dan dictionary attack, yaitu metode penyerangan yang dilakukan dengan mencoba berbagai kombinasi nama pengguna dan kata sandi secara otomatis hingga memperoleh kredensial yang benar. Serangan tersebut umumnya memanfaatkan port SSH yang terbuka untuk umum, khususnya port default 22, sehingga menjadi target utama berbagai bot otomatis yang terus melakukan pemindaian terhadap server yang terhubung ke internet (Utomo et al., 2024).

Pada praktiknya, masih banyak administrator sistem yang hanya mengandalkan konfigurasi keamanan dasar tanpa menerapkan mekanisme pertahanan aktif terhadap percobaan login berulang. Kondisi tersebut menyebabkan server harus memproses ribuan permintaan autentikasi ilegal yang berlangsung secara terus-menerus. Selain meningkatkan peluang keberhasilan penyerang memperoleh akses tidak sah, aktivitas tersebut juga berdampak pada meningkatnya penggunaan sumber daya server seperti CPU, memori, dan bandwidth jaringan. Dalam kondisi tertentu, tingginya intensitas percobaan login bahkan dapat mengakibatkan penurunan performa layanan hingga menyebabkan server tidak dapat diakses secara optimal oleh pengguna yang sah (Ridho et al., 2025).

Salah satu pendekatan yang banyak digunakan untuk mengurangi risiko tersebut adalah penerapan Fail2Ban yang bekerja berdasarkan metode Log Analysis. Fail2Ban melakukan pemantauan terhadap file log sistem, khususnya /var/log/auth.log, kemudian menganalisis pola kegagalan autentikasi menggunakan parameter seperti maxretry, findtime, dan bantime. Apabila jumlah percobaan login gagal melebihi ambang batas yang telah ditentukan, sistem akan secara otomatis menambahkan aturan pada firewall untuk memblokir alamat IP penyerang dalam jangka waktu tertentu. Integrasi antara Fail2Ban dan firewall memungkinkan proses mitigasi berlangsung secara otomatis sehingga administrator tidak perlu melakukan pemblokiran secara manual setiap kali terjadi serangan (Nursetyo et al., 2026).

Meskipun mekanisme pemblokiran otomatis telah banyak diterapkan, sebagian besar penelitian terdahulu masih berfokus pada aspek deteksi dan pencegahan serangan tanpa menyediakan fasilitas monitoring yang mampu menampilkan kondisi keamanan server secara komprehensif. Informasi mengenai aktivitas login, alamat IP yang diblokir, status firewall, maupun kondisi server umumnya hanya dapat diperoleh melalui terminal Linux dengan membaca file log secara manual. Cara tersebut memerlukan kemampuan administrasi sistem yang cukup baik dan menjadi kurang efisien ketika jumlah log yang dihasilkan semakin besar. Akibatnya, proses analisis insiden keamanan, audit sistem, serta evaluasi efektivitas mekanisme pertahanan menjadi lebih sulit dilakukan (Farhannullah & Hardjianto, 2022).

Selain keterbatasan dalam visualisasi data keamanan, proses penyampaian informasi kepada administrator juga masih menjadi tantangan. Pada banyak implementasi, administrator baru mengetahui adanya serangan setelah melakukan pemeriksaan log secara manual sehingga respons terhadap insiden sering kali terlambat. Padahal, kecepatan memperoleh informasi merupakan salah satu faktor penting dalam menjaga keamanan layanan yang berjalan secara terus-menerus. Oleh karena itu, diperlukan mekanisme notifikasi yang mampu mengirimkan informasi secara otomatis setiap kali sistem mendeteksi aktivitas mencurigakan maupun melakukan pemblokiran terhadap alamat IP penyerang (Abdullah et al., 2024).

Berdasarkan kondisi tersebut, penelitian ini mengembangkan sistem keamanan server Linux yang mengintegrasikan Firewall, Fail2Ban, website monitoring berbasis Flask, dan Telegram Bot dalam satu kesatuan sistem. Website monitoring dirancang untuk menampilkan informasi keamanan server secara real-time, meliputi status server, penggunaan sumber daya, aktivitas login SSH, isi file auth.log, status firewall, daftar alamat

IP yang diblokir oleh Fail2Ban, serta statistik percobaan serangan yang terjadi. Di sisi lain, Telegram Bot digunakan sebagai media notifikasi otomatis yang mengirimkan informasi secara langsung kepada administrator ketika sistem mendeteksi maupun memblokir serangan brute force dan dictionary attack. Integrasi kedua komponen tersebut diharapkan mampu meningkatkan efektivitas monitoring, mempercepat respons administrator terhadap ancaman keamanan, serta mempermudah proses evaluasi kondisi keamanan server secara berkelanjutan.

Berdasarkan uraian tersebut, penulis mengangkat penelitian dengan judul "**Implementasi Sistem Keamanan Server Linux Menggunakan Firewall dan Fail2Ban Berbasis Log Analysis untuk Mitigasi Brute Force dan Dictionary Attack pada Layanan SSH**".

1.2 Rumusan Permasalahan

Berdasarkan latar belakang yang telah diuraikan diatas, dapat dirumuskan permasalahan sebagai berikut:

1. Bagaimana mengimplementasikan mekanisme keamanan berbasis Firewall, Fail2Ban, dan Log Analysis untuk mendeteksi, merespons, serta memantau berbagai skenario serangan pada server Linux dan website monitoring yang meliputi serangan *reconnaissance, dictionary attack, brute force attack, SQL Injection, credential stuffing, Man-in-the-Middle (MITM)/ARP spoofing, Denial of Service (DoS), low-and-slow brute force, distributed brute force, dan brute force* pada login website monitoring?
2. Bagaimana efektivitas sistem dalam merespons serangan *Brute Force Attack* dan *Dictionary Attack* berdasarkan waktu respons pemblokiran, keberhasilan pemblokiran alamat IP, serta keberhasilan pengiriman notifikasi kepada administrator, dan bagaimana kemampuan Auditd dalam mencatat aktivitas akses terhadap berkas sensitif sebagai mekanisme pertahanan tambahan?
3. Bagaimana merancang dan mengimplementasikan website monitoring berbasis Flask yang mampu menampilkan informasi keamanan server secara real-time, meliputi status server, aktivitas login SSH, log autentikasi (*auth.log*), status Firewall, serta daftar alamat IP yang diblokir oleh Fail2Ban?

1.3 Batasan Masalah

Agar penelitian lebih terarah dan tidak menyimpang dari tujuan, maka batasan masalah dalam penelitian ini adalah:

1. Objek penelitian berfokus pada implementasi keamanan layanan Secure Shell (SSH) pada sistem operasi Ubuntu Server yang berjalan di lingkungan Virtual Machine.
2. Mekanisme pertahanan sistem menggunakan kombinasi Fail2Ban dan Firewall (iptables/UFW) melalui metode Log Analysis pada berkas /var/log/auth.log, serta Auditd sebagai mekanisme pertahanan lapis kedua untuk mencatat aktivitas akses terhadap berkas sensitif.
3. Jenis serangan yang diujikan dalam simulasi ini adalah brute force attack dan dictionary attack menggunakan tools Hydra, dengan jumlah pengujian masing-masing 5 kali percobaan. Pengujian tambahan juga dilakukan terhadap serangan reconnaissance (Nmap scanning), SQL Injection, credential stuffing, Man-in-the-Middle (MITM)/ARP Spoofing, Denial of Service (DoS), low-and-slow brute force, distributed brute force, dan brute force pada login website monitoring, namun tidak mencakup serangan terdistribusi skala besar dari banyak sumber IP secara simultan.
4. Skenario celah keamanan disimulasikan menggunakan akun uji dengan kredensial lemah yang sengaja disiapkan, guna mendemonstrasikan kemampuan mekanisme pertahanan lapis kedua (Auditd) dalam mendeteksi aktivitas pasca-eksploitasi, bukan dengan mengeksploitasi kerentanan sistem yang sesungguhnya.
5. Pengujian konsumsi sumber daya server (CPU, RAM, Disk) serta pengujian keamanan terhadap website monitoring itu sendiri tidak menjadi fokus utama penelitian ini dan disajikan sebagai bagian dari saran pengembangan selanjutnya.
6. Parameter evaluasi sistem dibatasi pada empat metrik utama, yaitu Response Time (kecepatan respons pemblokiran alamat IP), Success Rate (keberhasilan pemblokiran), Notification Rate (keberhasilan pengiriman notifikasi Telegram), dan Accuracy (akurasi deteksi berdasarkan komponen TP, TN, FP, FN), serta kemampuan website monitoring dalam menampilkan informasi keamanan secara real-time.

1.4 Tujuan Penelitian

Berdasarkan rumusan masalah tersebut maka tujuan dari penelitian ini adalah:

1. Mengimplementasikan sistem keamanan jaringan pada server Linux dengan mengintegrasikan Firewall, Fail2Ban, dan Auditd berbasis analisis log yang berjalan secara otomatis terhadap serangan brute force dan dictionary attack, serta

mampu mencatat aktivitas akses terhadap berkas sensitif sebagai mekanisme pertahanan lapis kedua.

2. Menguji dan mengevaluasi efektivitas sistem secara kuantitatif dalam merespons serangan brute force dan dictionary attack, ditinjau dari Response Time (kecepatan respons pemblokiran alamat IP), Success Rate (keberhasilan pemblokiran alamat IP), Notification Rate (keberhasilan pengiriman notifikasi Telegram), dan Accuracy (akurasi deteksi berdasarkan komponen TP, TN, FP, dan FN), serta mengevaluasi kemampuan Auditd dalam mendeteksi aktivitas mencurigakan terhadap berkas sensitif apabila lapisan pertahanan pertama berhasil dilewati.
3. Merancang dan mengimplementasikan website monitoring berbasis Flask yang mampu menampilkan informasi keamanan server secara real-time, meliputi status server, aktivitas login SSH, log autentikasi (auth.log), status Firewall, status Fail2Ban, daftar alamat IP yang terdeteksi dan diblokir, serta aktivitas akses berkas sensitif yang dicatat oleh Auditd.

1.5 Manfaat Penelitian

Adapun manfaat yang diharapkan dari penelitian ini adalah:

1. Untuk peneliti, penelitian ini diharapkan dapat menambah pengetahuan dan pengalaman dalam merancang serta mengimplementasikan sistem keamanan server Linux dengan mengintegrasikan Firewall, Fail2Ban, website monitoring berbasis Flask, dan Telegram Bot. Selain itu, penelitian ini memberikan pemahaman mengenai penerapan metode Log Analysis dalam mendeteksi aktivitas mencurigakan, melakukan mitigasi serangan brute force dan dictionary attack, serta menyajikan informasi keamanan server secara real-time melalui antarmuka website dan notifikasi Telegram.
2. Untuk perkembangan teknologi keamanan, penelitian ini diharapkan dapat memberikan kontribusi dalam pengembangan sistem keamanan server yang lebih efektif, otomatis, dan mudah dipantau. Integrasi Firewall, Fail2Ban, website monitoring, serta Telegram Bot memungkinkan administrator melakukan pemantauan kondisi server, aktivitas autentikasi SSH, status Firewall, status Fail2Ban, dan informasi alamat IP yang diblokir secara real-time. Hasil penelitian ini juga diharapkan dapat menjadi referensi bagi pengembangan sistem keamanan server berbasis Log Analysis dalam mendukung deteksi dini dan respons yang lebih cepat terhadap ancaman keamanan jaringan.

1.6 Sistematika Penulisan

Sistematika penulisan dalam penelitian ini disusun secara terstruktur untuk mempermudah pemahaman terhadap keseluruhan isi penelitian. Adapun sistematika penulisan laporan penelitian ini adalah sebagai berikut:

BAB I PENDAHULUAN

Bab ini berisi latar belakang penelitian, rumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, serta sistematika penulisan.

BAB II KAJIAN PUSTAKA

Bab ini memuat landasan teori, penelitian terdahulu, dan *state of the art* yang menjadi dasar dalam penyusunan dan pengembangan penelitian.

BAB III METODE PENELITIAN

Bab ini menjelaskan metode penelitian, tahapan pelaksanaan, serta perangkat keras dan perangkat lunak yang digunakan dalam perancangan, implementasi, dan pengujian sistem.

BAB IV ANALISIS DAN PERANCANGAN

Bab ini membahas analisis sistem serta perancangan sistem keamanan server yang meliputi blok diagram, analisis input, proses, dan output, diagram UML, perancangan basis data, perancangan antarmuka website monitoring, serta arsitektur integrasi Firewall, Fail2Ban, Website Monitoring, dan Telegram Bot.

BAB V HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil implementasi sistem, pengujian terhadap serangan brute force dan dictionary attack, serta pembahasan mengenai kinerja Firewall, Fail2Ban, website monitoring, dan Telegram Bot dalam mendeteksi, memonitor, dan memitigasi serangan.

BAB VI KESIMPULAN DAN SARAN

Bab ini berisi kesimpulan yang diperoleh dari hasil penelitian serta saran yang dapat digunakan sebagai pengembangan pada penelitian selanjutnya.

BAB II
KAJIAN PUSTAKA

2.1 Keamanan Jaringan Komputer

Keamanan jaringan komputer merupakan serangkaian upaya sistematis yang bertujuan melindungi infrastruktur jaringan dari ancaman yang dapat mengganggu kerahasiaan, integritas, dan ketersediaan data. Ancaman tersebut meliputi akses tidak sah, penyalahgunaan informasi, hingga serangan siber yang berpotensi merusak sistem. Dalam penerapannya, keamanan jaringan tidak hanya mencakup perlindungan perangkat keras, tetapi juga pengelolaan perangkat lunak, konfigurasi sistem, serta kebijakan kontrol akses pengguna. Salah satu pendekatan penting adalah *defense in depth*, yaitu strategi perlindungan berlapis untuk meminimalkan risiko kegagalan pada satu titik. Dengan pendekatan ini, setiap komponen keamanan saling mendukung dalam memberikan perlindungan yang lebih menyeluruh.

Pendekatan tersebut diperlukan agar seluruh elemen jaringan terlindungi secara optimal. Seiring perkembangan teknologi, sistem keamanan modern mengadopsi berbagai mekanisme seperti pemantauan lalu lintas jaringan, *Intrusion Detection System* (IDS), serta analisis aktivitas sistem untuk mendeteksi dan merespons ancaman secara cepat. Selain itu, prinsip least privilege dan zero trust architecture juga diterapkan untuk membatasi akses dan memastikan verifikasi berkelanjutan. Integrasi berbagai mekanisme ini memungkinkan peningkatan efektivitas deteksi dan respons terhadap serangan, sekaligus menjaga stabilitas dan keandalan sistem jaringan secara berkelanjutan (Alsubhi et al., 2023).

2.2 Server Linux

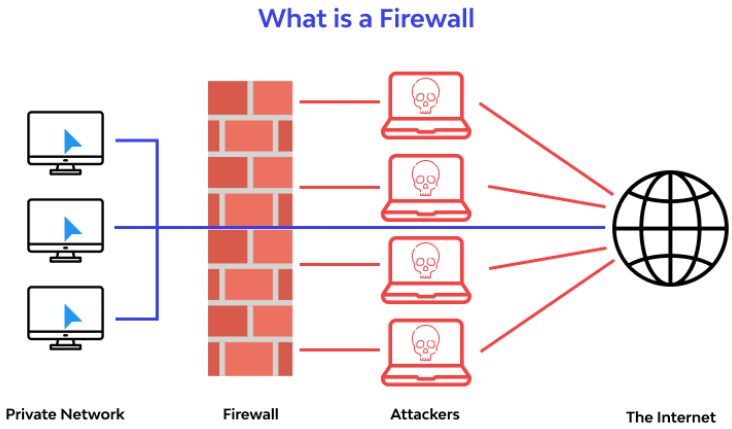


Gambar 2.1 Linux

Linux merupakan sistem operasi berbasis open-source yang широко dimanfaatkan sebagai platform server karena dikenal memiliki tingkat stabilitas yang tinggi, fleksibilitas dalam konfigurasi, serta dukungan fitur keamanan yang andal. Karakteristik ini menjadikan Linux sebagai pilihan utama dalam pengelolaan infrastruktur server di berbagai lingkungan, baik skala kecil maupun besar. Melalui sistem ini, *administrator* memiliki kemampuan untuk mengelola beragam layanan jaringan, seperti web server, database server, hingga layanan akses jarak jauh, dengan tingkat kendali yang mendalam terhadap seluruh aspek sistem.

Selain itu, Linux juga menyediakan dukungan terhadap berbagai metode penguatan keamanan atau system hardening, yang memungkinkan pengaturan akses secara ketat, pengelolaan layanan yang berjalan, serta pemantauan aktivitas sistem melalui pencatatan log secara sistematis. Dengan memanfaatkan fitur-fitur tersebut, *administrator* dapat meningkatkan tingkat keamanan server sekaligus mengurangi potensi risiko terhadap ancaman yang mungkin terjadi (Kumar & Singh, 2024).

2.3 Firewall



Gambar 2.2 Firewall

Firewall merupakan salah satu elemen utama dalam sistem keamanan jaringan yang memiliki fungsi untuk mengatur serta mengendalikan arus lalu lintas data, baik yang masuk maupun keluar dari suatu jaringan, dengan mengacu pada kebijakan keamanan yang telah ditetapkan. Peran ini menjadikan firewall sebagai lapisan perlindungan awal dalam menghadapi berbagai ancaman eksternal, di mana sistem akan melakukan proses penyaringan terhadap setiap paket data untuk menentukan apakah data tersebut layak diizinkan atau harus ditolak. Dengan mekanisme tersebut, firewall mampu membatasi akses yang tidak sah serta melindungi jaringan dari potensi gangguan. Dalam penerapannya, firewall dapat berupa perangkat lunak maupun perangkat keras yang berfungsi sebagai

pembatas antara jaringan internal yang relatif aman dan jaringan eksternal yang memiliki tingkat kepercayaan rendah, seperti internet.

Dalam praktik implementasinya, firewall bekerja dengan melakukan analisis terhadap setiap paket data yang melintasi jaringan berdasarkan sejumlah parameter tertentu, seperti alamat IP sumber dan tujuan, nomor port, serta jenis protokol yang digunakan dalam komunikasi. Pada lingkungan sistem operasi Linux, penerapan firewall umumnya memanfaatkan tools seperti iptables atau Uncomplicated Firewall (UFW), yang memberikan keleluasaan bagi *administrator* untuk menetapkan aturan pemfilteran secara rinci dan spesifik sesuai kebutuhan sistem. Melalui konfigurasi yang tepat, firewall dapat memastikan bahwa hanya pengguna atau layanan tertentu yang memperoleh akses, sekaligus mencegah berbagai bentuk serangan jaringan, termasuk akses ilegal, port scanning, serta eksploitasi terhadap layanan yang terbuka. Dengan demikian, keberadaan firewall menjadi komponen yang sangat penting dalam menjaga keamanan, stabilitas, dan keandalan operasional jaringan secara menyeluruh (Zhang et al., 2024).

2.4 Fail2Ban



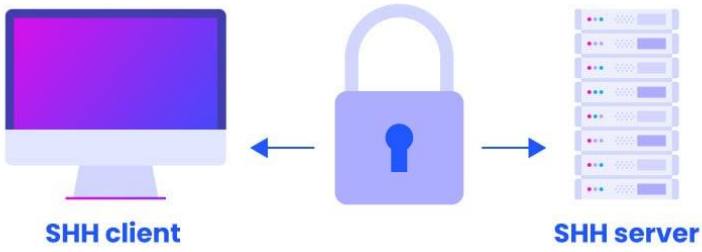
Gambar 2.3 Fail2Ban

Fail2Ban merupakan perangkat lunak keamanan yang dikembangkan untuk memberikan perlindungan terhadap server dari berbagai bentuk serangan siber, terutama serangan brute force yang menyasar layanan autentikasi seperti Secure Shell (SSH). Mekanisme kerjanya bertumpu pada proses pembacaan dan analisis file log sistem guna mengidentifikasi aktivitas login yang gagal secara berulang dalam rentang waktu tertentu. Informasi yang diproses dari log tersebut meliputi alamat IP asal, waktu terjadinya percobaan login, serta jumlah kegagalan autentikasi yang tercatat. Dengan memanfaatkan data tersebut, sistem mampu mengenali pola serangan secara lebih akurat dan sistematis,

sehingga potensi ancaman dapat dideteksi sejak tahap awal sebelum menimbulkan dampak yang lebih serius terhadap keamanan server.

Dalam operasionalnya, Fail2Ban menerapkan seperangkat aturan yang berfungsi untuk menentukan ambang batas maksimum percobaan login yang diperbolehkan oleh sistem. Apabila jumlah kegagalan autentikasi melampaui batas yang telah ditetapkan, maka sistem secara otomatis akan melakukan pemblokiran terhadap alamat IP yang terindikasi sebagai sumber serangan. Proses pemblokiran ini dilakukan melalui integrasi dengan firewall, seperti iptables atau Uncomplicated Firewall (UFW), sehingga tindakan pencegahan dapat langsung diterapkan pada tingkat jaringan. Seluruh proses tersebut berlangsung tanpa memerlukan intervensi langsung dari *administrator*, yang pada akhirnya meningkatkan efisiensi serta kecepatan respons dalam menangani ancaman secara *real-time*. Dengan kemampuan konfigurasi yang fleksibel, Fail2Ban menjadi salah satu solusi yang efektif dalam memperkuat keamanan server Linux, khususnya terhadap serangan yang menargetkan mekanisme autentikasi (Ridho et al., 2025).

2.5 *Secure Shell* (SSH)



Gambar 2.4 Secure Shell (SSH)

Secure Shell (SSH) merupakan salah satu protokol jaringan yang dimanfaatkan untuk menyediakan akses jarak jauh secara aman ke dalam sistem komputer, baik melalui jaringan internet maupun jaringan lokal. Protokol ini dirancang dengan mekanisme enkripsi yang kuat sehingga memungkinkan proses komunikasi berlangsung secara terlindungi dari potensi penyadapan oleh pihak yang tidak berwenang. Melalui SSH, *administrator* dapat melakukan berbagai aktivitas penting, seperti konfigurasi sistem, pengelolaan layanan server, serta transfer data dengan tingkat keamanan yang tinggi.

Meskipun demikian, keberadaan layanan SSH yang umumnya terbuka pada port publik demi mendukung kebutuhan administrasi jarak jauh menjadikannya sebagai salah satu target utama dalam serangan siber. Salah satu bentuk serangan yang paling sering terjadi adalah brute force, di mana penyerang mencoba berbagai kombinasi kredensial secara berulang untuk memperoleh akses tidak sah ke dalam sistem (Farhannullah & Hardjianto, 2022).

2.6 Brute force attack

Brute force attack merupakan salah satu metode serangan siber yang dilakukan dengan cara mencoba berbagai kombinasi *username* dan *password* secara berulang hingga ditemukan kredensial yang sesuai dengan sistem target. Serangan ini umumnya memanfaatkan tools otomatis seperti Hydra untuk mempercepat proses autentikasi secara masif pada layanan jaringan, terutama *Secure Shell* (SSH). Pada server Linux yang masih menggunakan konfigurasi default tanpa mekanisme pembatasan login, *brute force attack* dapat meningkatkan jumlah percobaan autentikasi gagal secara signifikan dan membuka peluang terjadinya akses ilegal ke dalam sistem. Selain berpotensi mengambil alih akun *administrator*, serangan ini juga dapat menyebabkan peningkatan penggunaan sumber daya server akibat tingginya aktivitas autentikasi yang diproses secara terus-menerus.

Penelitian mengenai keamanan layanan SSH terhadap *brute force attack* menunjukkan bahwa layanan SSH tanpa perlindungan tambahan memiliki tingkat kerentanan yang cukup tinggi terhadap percobaan login ilegal secara berulang. Dalam penelitian tersebut dilakukan simulasi serangan menggunakan *tools* Hydra untuk menguji efektivitas mekanisme keamanan berbasis Fail2Ban pada server Linux. Hasil penelitian menunjukkan bahwa Fail2Ban mampu melakukan monitoring terhadap *file* log autentikasi SSH dan secara otomatis memblokir alamat IP yang melebihi batas percobaan login gagal yang telah ditentukan. Implementasi tersebut terbukti mampu menghentikan proses *brute force* secara langsung dan meningkatkan keamanan layanan SSH pada server Linux (Ridho et al., 2025).

Penelitian lain mengenai implementasi keamanan jaringan menggunakan Fail2Ban menjelaskan bahwa metode *log analysis* sangat efektif digunakan untuk mendeteksi aktivitas *brute force attack* pada layanan SSH. Sistem keamanan bekerja dengan membaca file *auth.log* untuk mengidentifikasi pola login gagal secara berulang dalam interval waktu tertentu. Ketika jumlah kegagalan autentikasi melampaui parameter *maxretry,* sistem secara otomatis menjalankan firewall untuk melakukan pemblokiran alamat IP penyerang. Penelitian tersebut menunjukkan bahwa kombinasi firewall dan Fail2Ban mampu

meningkatkan efektivitas mitigasi serangan *brute force* serta membantu *administrator* dalam melakukan monitoring aktivitas keamanan server secara *real-time* (Utomo et al., 2024).

Tabel 2.1 Dataset Serangan Brute Force

Dataset Kaggle	**Total Data**	**Jenis Serangan**	**Data Brute Force**	**Sumber**
NF-UQ-NIDS	11.994.893 trafik	SSH *Bruteforce,* FTP *Bruteforce*, DoS, DDoS	Mengandung jutaan trafik *brute force*	Kaggle – NF-UQ-NIDS Dataset
InSDN *Dataset*	343.939 trafik	*Brute Force, Probe,* DoS	1.405 *record brute force*	Kaggle – InSDN Dataset
CSE-CIC-IDS2018	1.048.575 trafik	SSH *Bruteforce*& FTP *Bruteforce*	Simulasi serangan *brute force*	Kaggle – CSE-CIC-IDS2018

2.7 *Log Analysis Method*

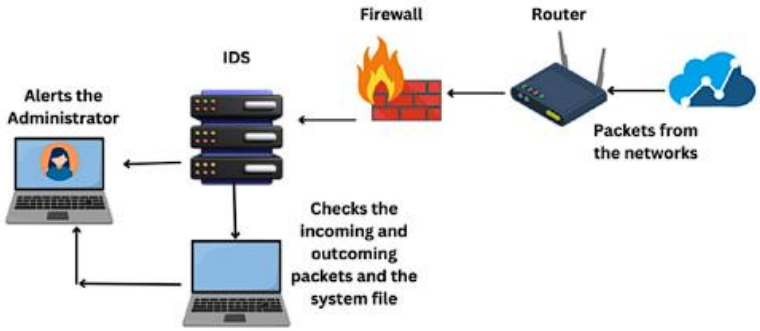
Metode analisis log merupakan pendekatan yang digunakan untuk mengkaji serta mengevaluasi data log sistem dengan tujuan mengidentifikasi aktivitas yang menyimpang dari pola normal maupun aktivitas yang berpotensi mengindikasikan adanya ancaman keamanan pada sistem jaringan. Data log sendiri merupakan catatan aktivitas yang dihasilkan secara otomatis oleh sistem operasi maupun layanan jaringan selama proses operasional berlangsung. Informasi yang tercatat dalam log meliputi proses autentikasi pengguna, aktivitas akses layanan jaringan, alamat IP pengguna, waktu akses, kegagalan login, hingga berbagai kesalahan sistem yang terjadi pada server. Seluruh data tersebut memiliki peran penting karena dapat digunakan sebagai sumber informasi utama dalam proses monitoring keamanan dan deteksi intrusi. Pada server Linux, *file log* seperti *auth.log* sering dimanfaatkan untuk mendeteksi adanya percobaan login ilegal yang dilakukan secara berulang terhadap layanan SSH.

Melalui proses analisis terhadap pola aktivitas yang terekam pada *file log*, sistem dapat mengenali adanya anomali atau perilaku mencurigakan yang berbeda dari aktivitas normal pengguna. Proses ini dilakukan dengan cara mengamati frekuensi login gagal, pola akses berulang dari alamat IP tertentu, serta interval waktu terjadinya aktivitas autentikasi yang tidak wajar. Ketika ditemukan pola yang mengarah pada indikasi serangan, seperti *brute force attack* atau *dictionary attack*, sistem keamanan dapat memberikan respons otomatis

berupa pemblokiran akses menggunakan firewall maupun pengiriman notifikasi kepada *administrator* server. Pendekatan ini memungkinkan proses deteksi ancaman dilakukan secara lebih cepat dan akurat dibandingkan pemantauan manual yang membutuhkan waktu lebih lama. Selain itu, analisis log juga membantu *administrator* dalam melakukan investigasi terhadap aktivitas serangan yang telah terjadi karena seluruh aktivitas sistem tercatat secara sistematis pada *file log*.

Penerapan metode log analysis pada sistem keamanan server Linux menjadi semakin penting seiring meningkatnya ancaman serangan siber yang menargetkan layanan autentikasi jarak jauh seperti SSH. Integrasi metode ini dengan tools keamanan seperti Fail2Ban memungkinkan sistem melakukan monitoring secara *real-time* terhadap aktivitas login pengguna dan secara otomatis memitigasi serangan ketika ditemukan percobaan autentikasi yang mencurigakan. Dengan adanya mekanisme tersebut, *administrator* dapat meningkatkan efektivitas keamanan server tanpa harus melakukan pengawasan secara terus-menerus. Oleh karena itu, metode analisis log dinilai sebagai salah satu pendekatan yang efektif dalam mendukung sistem keamanan jaringan modern karena mampu meningkatkan kemampuan deteksi, mempercepat respons terhadap ancaman, serta membantu menjaga stabilitas dan keamanan layanan server secara berkelanjutan (Komadina et al., 2024).

2.8 Intrusion Detection System (IDS)



Gambar 2.5 Intrusion Detection System

Intrusion Detection System (IDS) merupakan salah satu komponen penting dalam sistem keamanan jaringan yang berfungsi untuk mengidentifikasi adanya aktivitas mencurigakan maupun pelanggaran terhadap kebijakan keamanan yang telah ditetapkan. Sistem ini bekerja dengan cara melakukan pemantauan serta analisis terhadap berbagai sumber data, seperti lalu lintas jaringan dan file log sistem, guna menemukan pola-pola yang mengindikasikan adanya serangan. Dalam praktiknya, IDS mampu mengenali berbagai jenis ancaman, termasuk serangan brute force, port scanning, hingga eksploitasi layanan jaringan yang

rentan. Kemampuan ini menjadikan IDS sebagai alat deteksi dini yang sangat penting dalam menjaga keamanan sistem, karena *administrator* dapat memperoleh peringatan lebih awal sebelum serangan berkembang menjadi lebih serius.

Secara umum, IDS dibedakan menjadi dua kategori utama, yaitu Network-based *Intrusion Detection System* (NIDS) yang berfokus pada pemantauan paket data yang melintas di jaringan, serta Host-based *Intrusion Detection System* (HIDS) yang bekerja dengan menganalisis aktivitas pada tingkat host atau sistem secara langsung. Meskipun IDS memiliki kemampuan yang baik dalam mendeteksi ancaman, sistem ini pada dasarnya bersifat pasif karena hanya memberikan notifikasi atau alert tanpa melakukan tindakan langsung terhadap serangan yang terjadi. Oleh karena itu, IDS biasanya digunakan sebagai lapisan awal dalam sistem keamanan untuk memberikan visibilitas terhadap aktivitas jaringan. Keberadaan IDS dalam sistem keamanan modern sangat krusial karena mampu meningkatkan kesadaran situasional (*situational awareness*) terhadap ancaman serta membantu *administrator* dalam mengambil keputusan yang tepat dalam menangani insiden keamanan (Alsubhi et al., 2023).

2.9 Intrusion prevention system (IPS)

Intrusion Prevention System (IPS) merupakan pengembangan dari Intrusion Detection System (IDS) yang dirancang tidak hanya untuk mendeteksi ancaman, tetapi juga memberikan respons otomatis terhadap aktivitas yang dianggap berbahaya. Berbeda dengan IDS yang bersifat pasif dan hanya menghasilkan notifikasi, IPS memiliki kemampuan untuk melakukan tindakan langsung seperti memblokir alamat IP penyerang, menghentikan koneksi mencurigakan, maupun memperbarui aturan firewall secara otomatis. Mekanisme tersebut memungkinkan sistem keamanan merespons ancaman secara real-time sehingga risiko kerusakan akibat serangan dapat diminimalkan.

Dalam implementasi pada server Linux, konsep IPS dapat diterapkan menggunakan kombinasi Fail2Ban dan firewall berbasis iptables atau UFW. Sistem bekerja dengan membaca file log autentikasi seperti auth.log untuk mendeteksi pola login gagal secara berulang. Ketika jumlah percobaan login gagal melampaui parameter maxretry, Fail2Ban akan secara otomatis menjalankan firewall untuk memblokir alamat IP penyerang dalam jangka waktu tertentu. Mekanisme ini menunjukkan bahwa sistem keamanan berbasis log analysis mampu bekerja secara adaptif dalam mendeteksi sekaligus mencegah serangan brute force dan dictionary attack pada layanan SSH (Ariyadi et al., 2025).

Penelitian mengenai implementasi IPS berbasis log analysis menunjukkan bahwa integrasi antara IDS dan firewall dapat meningkatkan efektivitas mitigasi serangan secara signifikan. Sistem keamanan tidak hanya mampu mendeteksi pola autentikasi abnormal, tetapi juga melakukan respons otomatis tanpa intervensi administrator. Selain meningkatkan keamanan server Linux, pendekatan ini juga membantu mengurangi beban monitoring manual karena seluruh proses mitigasi berjalan secara otomatis berdasarkan analisis log sistem (Utomo et al., 2024).

2.10 System hardening

System hardening merupakan proses penguatan keamanan sistem yang bertujuan untuk mengurangi potensi kerentanan dengan cara melakukan konfigurasi yang lebih ketat dan terkontrol. Pendekatan ini berfokus pada pengurangan attack surface atau permukaan serangan, sehingga kemungkinan sistem dieksploitasi oleh pihak yang tidak bertanggung jawab dapat diminimalkan. Dalam lingkungan server Linux, hardening dilakukan dengan mengoptimalkan konfigurasi sistem agar hanya layanan yang benar-benar dibutuhkan saja yang aktif, serta memastikan bahwa setiap akses terhadap sistem memiliki batasan yang jelas.

Penerapan *System hardening* pada server Linux dapat dilakukan melalui berbagai langkah strategis, seperti menonaktifkan layanan yang tidak diperlukan, mengubah konfigurasi default seperti port SSH, menerapkan autentikasi berbasis kunci publik, serta membatasi hak akses pengguna sesuai dengan kebutuhan operasional. Langkah-langkah tersebut tidak hanya meningkatkan keamanan sistem, tetapi juga membantu *administrator*dalam mengontrol aktivitas yang terjadi pada server. Penelitian menunjukkan bahwa penerapan hardening yang dikombinasikan dengan mekanisme keamanan lain seperti firewall dan Fail2Ban mampu meningkatkan ketahanan sistem terhadap serangan brute force secara signifikan (Kumar & Singh, 2024).

2.11 *Real-time* Monitoring & Notifikasi Sistem

Real-time monitoring merupakan metode pemantauan sistem yang dilakukan secara terus-menerus dan langsung terhadap aktivitas yang terjadi pada server maupun jaringan. Pendekatan ini memungkinkan *administrator* untuk memperoleh informasi secara cepat mengenai kondisi sistem, termasuk adanya aktivitas yang tidak normal atau indikasi serangan siber. Dengan adanya pemantauan secara *real-time*, potensi ancaman dapat segera diketahui sehingga tindakan mitigasi dapat dilakukan tanpa harus menunggu analisis manual yang memerlukan waktu lebih lama.

Dalam implementasinya, sistem monitoring biasanya dilengkapi dengan fitur notifikasi otomatis yang berfungsi untuk menyampaikan informasi penting kepada *administrator* secara instan. Notifikasi ini dapat dikirim melalui berbagai media seperti email, *Telegram bot*, maupun dashboard monitoring berbasis web. Penggunaan *Telegram bot* menjadi salah satu solusi yang banyak digunakan karena kemudahan integrasi serta kecepatan dalam penyampaian informasi. Dengan adanya notifikasi *real-time*, *administrator* dapat segera mengetahui adanya aktivitas pemblokiran atau percobaan serangan sehingga proses penanganan dapat dilakukan dengan lebih cepat dan efektif (Batistuta et al., 2024).

2.12 *Response time*

Response time atau waktu respons merupakan parameter penting yang digunakan untuk mengukur kinerja sistem keamanan dalam merespons suatu ancaman. Parameter ini menunjukkan durasi waktu yang dibutuhkan sejak serangan terjadi hingga sistem memberikan tindakan tertentu, seperti pemblokiran alamat IP atau pengiriman notifikasi kepada *administrator*. Dalam konteks keamanan jaringan, kecepatan respons menjadi faktor krusial karena semakin cepat sistem bereaksi, maka semakin kecil kemungkinan penyerang berhasil mengeksploitasi celah keamanan yang ada.

Pengukuran *Response time* umumnya dilakukan dalam satuan milidetik untuk mendapatkan hasil yang lebih akurat dan detail. Sistem keamanan yang memiliki waktu respons cepat cenderung lebih efektif dalam mengurangi dampak serangan serta menjaga stabilitas layanan. Oleh karena itu, dalam penelitian ini, *Response time* digunakan sebagai indikator utama untuk mengevaluasi kinerja integrasi antara Fail2Ban dan firewall dalam mendeteksi serta memitigasi serangan secara otomatis (Chen et al., 2021).

2.13 *Centralized log management* (Manajemen Log Terpusat)

Centralized log management merupakan pendekatan dalam pengelolaan log yang mengintegrasikan data dari berbagai server ke dalam satu sistem pusat. Dengan adanya sistem ini, *administrator* dapat melakukan pemantauan dan analisis aktivitas secara lebih efisien karena seluruh informasi terkumpul dalam satu tempat. Pendekatan ini sangat penting dalam lingkungan multi-server, di mana aktivitas pada satu server dapat berkaitan dengan aktivitas pada server lainnya.

Melalui sistem log terpusat, *administrator* dapat mengidentifikasi pola serangan yang tersebar, memahami hubungan antar kejadian, serta mengambil tindakan mitigasi secara lebih terkoordinasi. Selain itu, informasi serangan yang terdeteksi pada satu server dapat digunakan untuk memicu mekanisme pemblokiran pada server lain secara otomatis. Hal ini

menjadikan *centralized log management* sebagai solusi yang efektif dalam meningkatkan kemampuan deteksi dan respons terhadap ancaman dalam sistem jaringan yang kompleks (Komadina et al., 2024).

2.14 Hydra



Gambar 2.6 Hydra

Hydra merupakan salah satu tools yang digunakan dalam pengujian keamanan sistem dengan metode *brute force* dan *dictionary attack*. *Tools* ini dirancang untuk melakukan percobaan autentikasi secara otomatis terhadap berbagai layanan jaringan, termasuk SSH, dengan mencoba berbagai kombinasi *username* dan *password* hingga menemukan pasangan yang valid. Proses ini dilakukan secara cepat dan berulang sehingga mampu mensimulasikan kondisi serangan yang sebenarnya terjadi di lingkungan jaringan.

Dalam penelitian keamanan jaringan, Hydra sering dimanfaatkan sebagai alat untuk menguji ketahanan sistem terhadap serangan *brute force*. Dengan menggunakan Hydra, peneliti dapat menghasilkan data log yang realistis sehingga dapat digunakan untuk menganalisis performa sistem keamanan dalam mendeteksi dan merespons serangan. Penggunaan *tools* ini juga membantu dalam mengidentifikasi kelemahan pada mekanisme autentikasi serta mengevaluasi efektivitas sistem seperti Fail2Ban dalam melakukan pemblokiran otomatis (Utomo et al., 2024).

2.15 *Dictionary Attack*

Dictionary attack merupakan metode serangan autentikasi yang dilakukan dengan mencoba kumpulan *password* yang berasal dari daftar kata tertentu (*wordlist*) secara otomatis hingga ditemukan kombinasi *username* dan *password* yang sesuai dengan sistem target. Berbeda dengan *brute force attack* yang mencoba seluruh kemungkinan kombinasi karakter, *dictionary attack* menggunakan kumpulan *password* umum yang sering digunakan

pengguna, seperti nama, tanggal lahir, maupun kombinasi kata sederhana. Metode ini dinilai lebih cepat dan efisien karena memanfaatkan kelemahan pengguna dalam membuat password yang mudah ditebak.

Penelitian yang dilakukan oleh Balqis dan Rahman menjelaskan bahwa *dictionary attack* menjadi salah satu ancaman utama pada layanan SSH karena banyak *administrator* masih menggunakan *password* sederhana dan mudah diprediksi. Dalam penelitian tersebut dilakukan simulasi serangan menggunakan *tools* Hydra dengan file *wordlist rockyou.txt* untuk menguji ketahanan autentikasi server Linux. Hasil penelitian menunjukkan bahwa *password* yang lemah dapat ditemukan dalam waktu singkat apabila sistem tidak memiliki mekanisme pembatasan login maupun pemblokiran otomatis. Implementasi Fail2Ban terbukti mampu mendeteksi pola login gagal secara berulang dan memblokir alamat IP penyerang secara otomatis melalui firewall sehingga risiko keberhasilan *dictionary attack* dapat dikurangi secara signifikan. (Balqis & Rahman, 2025).

Penelitian lain yang dilakukan oleh Farhannullah dan Hardjianto menjelaskan bahwa layanan SSH merupakan salah satu layanan jaringan yang paling sering menjadi target dictionary attack karena menggunakan autentikasi berbasis *username* dan *password* yang dapat diakses melalui jaringan publik. Dalam penelitian tersebut dijelaskan bahwa penggunaan kombinasi firewall, *system hardening*, dan log monitoring mampu meningkatkan keamanan layanan SSH terhadap percobaan *dictionary attack*. Sistem keamanan bekerja dengan menganalisis file log autentikasi untuk mendeteksi pola login mencurigakan serta membatasi percobaan autentikasi dari alamat IP tertentu. Hasil penelitian menunjukkan bahwa metode log analysis efektif digunakan untuk mendeteksi aktivitas serangan secara *real-time* dan membantu *administrator* dalam melakukan mitigasi ancaman keamanan server Linux. (Farhannullah & Hardjianto, 2022).

Tabel 2.2 Dataset Dictionary Attack

Dataset	**Total Data**	**Jenis Serangan**	**Karakteristik Dataset**	**Sumber**
CIC-IDS2017	±2,8 juta trafik jaringan	SSH-Patator, FTP-Patator, DoS	Simulasi dictionary attack SSH	Kaggle–CICIDS2017

CSE-CIC-IDS2018	±16 juta trafik jaringan	SSH Brute Force & FTP Brute Force	Percobaan login berbasis wordlist	Kaggle – CSE-CIC-IDS2018
TON_IoT Dataset	Telemetry & log jaringan	Password attack, SSH attack	Aktivitas autentikasi mencurigakan	Kaggle – TON_IoT Dataset

2.16 Website Monitoring Server

Website monitoring server merupakan aplikasi berbasis web yang digunakan untuk memantau kondisi server secara terpusat melalui antarmuka grafis (*Graphical User Interface*). Sistem ini menyajikan informasi mengenai kondisi server, aktivitas layanan, log sistem, serta status keamanan secara *real-time* sehingga memudahkan administrator dalam melakukan pengawasan dan pengambilan keputusan ketika terjadi gangguan maupun ancaman keamanan. Dengan adanya website monitoring, proses pemantauan server menjadi lebih efektif karena informasi dapat diakses melalui browser tanpa harus selalu masuk ke server menggunakan terminal SSH. Pada penelitian ini, website monitoring digunakan untuk menampilkan status server, aktivitas pada file auth.log, status Firewall, status Fail2Ban, serta daftar alamat IP yang melakukan serangan dan telah diblokir oleh sistem. (Bota & Setiawati, 2025).

2.17 Flask



Gambar 2.7 Flask

Flask merupakan *micro web framework* berbasis bahasa pemrograman Python yang dirancang untuk membangun aplikasi web dengan arsitektur yang ringan, sederhana, dan fleksibel. Framework ini menyediakan berbagai komponen dasar seperti mekanisme *routing*, *template engine*, pengelolaan sesi, serta kemudahan integrasi dengan berbagai pustaka eksternal sehingga banyak digunakan dalam pengembangan aplikasi monitoring maupun

sistem administrasi berbasis web. Pada penelitian ini, Flask digunakan sebagai framework utama untuk membangun website monitoring yang berfungsi menampilkan informasi keamanan server, membaca log autentikasi SSH, menampilkan status Firewall dan Fail2Ban, serta menyajikan data hasil monitoring secara *real-time* kepada administrator. (Silalahi et al., 2026).

2.18 Telegram Bot



Gambar 2.8 Telegram Bot

Telegram Bot merupakan layanan berbasis *Application Programming Interface* (API) yang memungkinkan aplikasi mengirimkan notifikasi secara otomatis kepada pengguna melalui aplikasi Telegram. Teknologi ini banyak dimanfaatkan pada sistem monitoring karena mampu menyampaikan informasi secara cepat dan *real-time* ketika terjadi suatu peristiwa tertentu pada server. Dalam penelitian ini, Telegram Bot diintegrasikan dengan sistem keamanan server untuk mengirimkan notifikasi secara otomatis ketika Fail2Ban berhasil mendeteksi dan memblokir alamat IP yang melakukan serangan brute force maupun dictionary attack terhadap layanan SSH. Melalui notifikasi tersebut, administrator dapat segera mengetahui adanya aktivitas mencurigakan tanpa harus terus-menerus memantau website monitoring. (Hasbir et al., 2026).

2.19 Penelitian Terdahulu

Referensi dari penelitian terdahulu menjadi landasan yang berharga untuk memperkaya teori dalam menyelidiki topik penelitian ini. Dengan merujuk pada penelitian terdahulu, penulis dapat mengidentifikasi berbagai aspek yang dapat dikembangkan lebih lanjut dalam konteks penelitian ini. Berikut adalah literatur dari penelitian terdahulu yang digunakan penulis sebagai referensi.

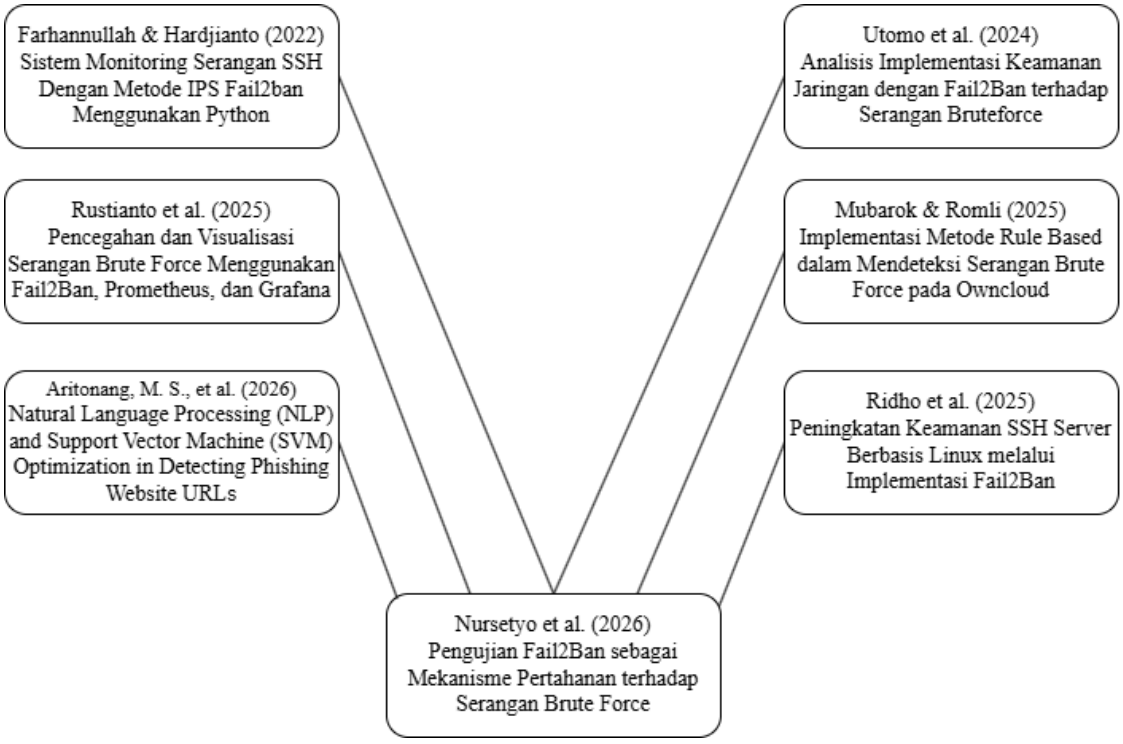
Tabel 2.3 Perbandingan Penelitian Terdahulu dan Penelitian Saat Ini

No	Peneliti	Tahun	Fokus Penelitian	Hasil Penelitian	Keterbatasan
1	Nursetyo, A., et al.	2026	Implementasi Fail2Ban berbasis *Log Analysis* untuk mitigasi serangan brute force pada layanan SSH Linux.	Fail2Ban mampu mendeteksi percobaan login berulang dan melakukan pemblokiran alamat IP secara otomatis berdasarkan parameter *maxretry*, *findtime*, dan *bantime*.	Belum menyediakan media monitoring berbasis web maupun notifikasi kepada administrator.
2	Rustianto, A., et al.	2025	Implementasi Fail2Ban dan Firewall untuk meningkatkan keamanan layanan SSH.	Integrasi Fail2Ban dengan Firewall berhasil mengurangi percobaan akses ilegal dan meningkatkan keamanan server Linux.	Monitoring masih dilakukan melalui terminal sehingga kurang efektif untuk pemantauan secara visual.
3	Ridho, M., et al.	2025	Analisis pengaruh serangan brute force terhadap performa server Linux.	Serangan brute force meningkatkan beban CPU dan memperlambat respons server, sedangkan penerapan Fail2Ban mampu menstabilkan performa server.	Penelitian hanya berfokus pada pengujian performa tanpa menyediakan sistem monitoring keamanan.

Tabel 2.4 Lanjutan Perbandingan Penelitian Terdahulu dan Penelitian Saat Ini

4	Mubarok, K., & Romli, M. A.	2025	Penerapan metode *Log Analysis* untuk mendeteksi aktivitas mencurigakan pada server.	Analisis log mampu mengidentifikasi pola serangan secara otomatis berdasarkan aktivitas autentikasi pengguna.	Belum mengintegrasikan mekanisme pemblokiran otomatis maupun notifikasi *real-time*.
5	Utomo, B. R., et al.	2024	Analisis kerentanan layanan SSH menggunakan Nmap dan Hydra.	Pengujian menunjukkan bahwa layanan SSH dengan konfigurasi standar rentan terhadap serangan brute force dan dictionary attack.	Penelitian hanya melakukan identifikasi kerentanan tanpa implementasi sistem mitigasi otomatis.
6	Farhannullah & Hardjianto	2022	Implementasi sistem monitoring keamanan SSH menggunakan Python dan Fail2Ban.	Dashboard monitoring memudahkan administrator dalam melihat aktivitas login dan log serangan pada server Linux.	Belum dilengkapi integrasi Firewall serta notifikasi instan melalui Telegram.

2.20 State of The Art



Gambar 2.9 State of The Art

1. Farhannullah dan Hardjianto (2022), meneliti tentang sistem pemantauan serangan pada layanan Secure Shell (SSH) menggunakan metode *Intrusion prevention system* (IPS) berbasis Fail2Ban dan bahasa pemrograman Python. Hasil penelitian ini menunjukkan bahwa pembuatan dashboard monitoring berbasis web dapat mempermudah *administrator* jaringan dalam mengamati data log secara berkala dan sistematis. Namun, sistem monitoring yang dirancang masih bersifat lokal dan pasif, serta belum mengintegrasikan mekanisme pengiriman notifikasi instan berbasis pesan seluler secara *real-time*.
2. Utomo et al. (2024), menganalisis implementasi keamanan jaringan dengan memanfaatkan Fail2Ban untuk memitigasi serangan *brute force*. Temuan penelitian membuktikan bahwa proses identifikasi *port* terbuka menggunakan Nmap dan simulasi serangan menggunakan Hydra merupakan tahapan yang sangat krusial sebelum menerapkan kebijakan pemblokiran alamat IP. Namun, fokus penelitian ini masih dominan pada tahap analisis kerentanan awal dan belum memberikan metrik evaluasi kuantitatif mengenai performa kecepatan respon sistem pertahanan setelah diimplementasikan.
3. Mubarok dan Romli (2025), mengimplementasikan metode *rule-based* untuk mendeteksi pola serangan *brute force* melalui pendekatan analisis log. Hasil

penelitian membuktikan bahwa analisis berkas log secara *real-time* memiliki tingkat akurasi yang sangat tinggi dalam mengenali pola aktivitas serangan berbasis kamus (*dictionary attack*). Namun, ruang lingkup pengujian dalam penelitian tersebut masih terbatas pada aplikasi penyimpanan berbasis *cloud* (*Owncloud*) dan belum diimplementasikan untuk mengamankan infrastruktur protokol SSH murni.

4. Ridho et al. (2025), mengevaluasi efektivitas penggunaan Fail2Ban dalam menjaga stabilitas kinerja server Linux dari serangan masif menggunakan tools Hydra. Temuan menunjukkan bahwa pemutusan koneksi penyerang secara otomatis oleh firewall dapat menahan lonjakan beban kerja *Central Processing Unit* (CPU) server secara signifikan. Meskipun demikian, skenario pengujian yang dilakukan masih bersifat single-server tunggal dan belum mengeksplorasi arsitektur analisis log terpusat yang melibatkan lingkungan multi-server.
5. Rustianto et al. (2025), mengembangkan sistem pencegahan dan visualisasi serangan *brute force* menggunakan kombinasi Fail2Ban, Prometheus, dan Grafana. Hasil penelitian menunjukkan bahwa integrasi komponen tersebut mempermudah proses audit keamanan melalui tampilan dasbor visual yang interaktif. Namun, implementasi stack monitoring pihak ketiga tersebut membutuhkan alokasi sumber daya komputasi (*resource*) yang cukup besar, sehingga kurang efisien jika diimplementasikan pada infrastruktur server dengan spesifikasi perangkat keras terbatas.
6. Nursetyo et al. (2026), melakukan pengujian eksperimental terhadap parameter komponen *jail* pada Fail2Ban, seperti *maxretry, findtime*, dan *bantime* untuk mengamankan layanan SSH di dalam lingkungan virtual machine. Hasil penelitian menunjukkan bahwa ketepatan penentuan nilai parameter tersebut menjadi penentu utama dalam efektivitas pemblokiran otomatis terhadap alamat IP penyerang. Namun, penelitian ini hanya berfokus pada mekanisme pertahanan mandiri pada satu node lokal tanpa menyertakan skema analisis log terdistribusi ataupun pengiriman laporan insiden ke pihak eksternal.
7. Aritonang et al. (2026), dalam penelitiannya yang berjudul *"Natural Language Processing (NLP) and Support Vector Machine (SVM) Optimization in Detecting Phishing Website URLs"*, berfokus pada optimasi deteksi ancaman keamanan siber berupa tautan (*URL*) *phishing*. Metodologi yang digunakan adalah dengan mengolah data teks (*string*) menggunakan teknik *Natural Language Processing* (NLP) untuk ekstraksi fitur, kemudian mengklasifikasikannya menggunakan

algoritma *Support Vector Machine* (SVM) guna membedakan antara URL normal dan URL organisasi yang termanipulasi. Hasil eksperimen membuktikan bahwa pendekatan analisis pola teks berbasis data digital memiliki tingkat akurasi yang tinggi dalam mengenali anomali serangan. Namun, ruang lingkup implementasi dan objek penelitian tersebut berada pada level aplikasi web (*application layer*) untuk menyaring URL, serta memerlukan proses komputasi pembelajaran mesin (*machine learning*) yang intensif. Penelitian tersebut belum menyentuh aspek otomasi mitigasi pertahanan infrastruktur pada *transport/network layer*, khususnya dalam mengamankan port otentikasi protokol SSH dari serangan berbasis paket massal seperti *brute force* dan *dictionary attack*.

BAB III
METODE PENELITIAN

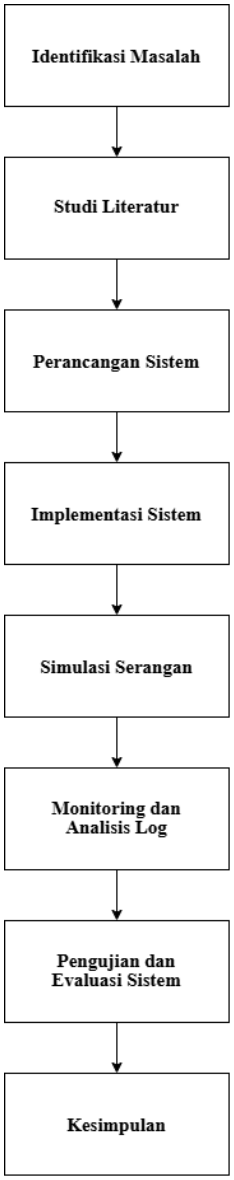
3.1 Metode Penelitian

Penelitian ini menggunakan metode eksperimen dengan pendekatan kuantitatif untuk mengimplementasikan sistem keamanan server Linux menggunakan Firewall dan Fail2Ban berbasis log analysis dalam mendeteksi serta memitigasi serangan brute force dan dictionary attack pada layanan Secure Shell (SSH). Penelitian dilakukan pada lingkungan virtual menggunakan Ubuntu Server sebagai server target, Kali Linux sebagai mesin penyerang, Firewall (UFW) sebagai mekanisme penyaringan akses jaringan, serta Fail2Ban sebagai sistem deteksi dan pemblokiran otomatis berdasarkan hasil analisis file auth.log. Selain itu, penelitian ini mengembangkan website monitoring berbasis Flask untuk menampilkan kondisi keamanan server secara real-time dan mengintegrasikan Telegram Bot sebagai media pengiriman notifikasi otomatis kepada administrator ketika terjadi aktivitas serangan maupun pemblokiran alamat IP.

Tahapan penelitian dimulai dari identifikasi permasalahan keamanan pada layanan SSH, kemudian dilanjutkan dengan studi literatur mengenai keamanan jaringan, Firewall, Fail2Ban, log analysis, website monitoring, dan Telegram Bot. Selanjutnya dilakukan perancangan dan implementasi sistem yang meliputi konfigurasi Firewall, konfigurasi Fail2Ban, pengembangan website monitoring, serta integrasi Telegram Bot. Website monitoring dirancang untuk menampilkan informasi status server, aktivitas autentikasi pada file auth.log, status Firewall, status Fail2Ban, dan daftar alamat IP yang melakukan percobaan login maupun yang telah diblokir oleh sistem.

Tahap pengujian dilakukan menggunakan Hydra pada Kali Linux untuk mensimulasikan serangan brute force dan dictionary attack terhadap layanan SSH pada Ubuntu Server. Aktivitas autentikasi yang dihasilkan dianalisis oleh Fail2Ban berdasarkan parameter maxretry, findtime, dan bantime sehingga sistem dapat melakukan pemblokiran alamat IP penyerang secara otomatis. Hasil pengujian kemudian dievaluasi berdasarkan waktu respons pemblokiran, tingkat keberhasilan deteksi serangan, keberhasilan pemblokiran alamat IP, keberhasilan pengiriman notifikasi Telegram, serta kemampuan website monitoring dalam menyajikan informasi keamanan server secara real-time.

3.2 Kerangka Penelitian



Gambar 3.1 Kerangka Penelitian

Kerangka penelitian digunakan sebagai pedoman sistematis dalam proses implementasi dan pengujian sistem keamanan server Linux menggunakan Firewall dan Fail2Ban berbasis log analysis. Kerangka penelitian ini disusun agar seluruh tahapan penelitian dapat berjalan secara terstruktur mulai dari identifikasi masalah hingga evaluasi efektivitas sistem keamanan terhadap serangan brute force dan dictionary attack pada layanan SSH.

1. Identifikasi Masalah

   Tahap identifikasi masalah dilakukan untuk menganalisis kerentanan keamanan pada layanan *Secure Shell* (*SSH*) di sistem operasi Linux terhadap serangan *brute force* dan *dictionary attack* yang memanfaatkan percobaan autentikasi secara berulang untuk

memperoleh akses tidak sah ke dalam server. Selain berpotensi mengganggu ketersediaan layanan, aktivitas tersebut juga dapat meningkatkan beban kerja server apabila tidak ditangani dengan mekanisme keamanan yang memadai. Di sisi lain, proses pemantauan keamanan yang masih bergantung pada pemeriksaan berkas *log* secara manual menyebabkan administrator membutuhkan waktu lebih lama dalam mendeteksi dan merespons ancaman yang terjadi. Berdasarkan kondisi tersebut, diperlukan suatu sistem yang mampu mendeteksi aktivitas mencurigakan melalui metode *log analysis*, melakukan pemblokiran alamat IP secara otomatis menggunakan *Firewall* dan Fail2Ban, serta menyediakan *website monitoring* berbasis Flask dan notifikasi *real-time* melalui Telegram Bot untuk mempermudah proses pemantauan keamanan server.

2. Studi Literatur

   Studi literatur dilakukan dengan mengumpulkan dan mempelajari berbagai referensi yang berkaitan dengan keamanan jaringan, sistem operasi Linux, *Firewall*, Fail2Ban, metode *log analysis*, *website monitoring*, *framework* Flask, Telegram Bot, serta teknik serangan *brute force* dan *dictionary attack*. Referensi diperoleh dari jurnal ilmiah, buku, prosiding, dan dokumentasi resmi yang relevan guna memperkuat landasan teoritis serta menjadi acuan dalam menentukan metode, perancangan sistem, implementasi, dan parameter pengujian yang digunakan pada penelitian ini.

3. Perancangan Sistem

   Tahap perancangan sistem bertujuan untuk menyusun arsitektur dan mekanisme kerja sistem keamanan yang akan diimplementasikan dengan mengintegrasikan Ubuntu Server sebagai server target, layanan *Secure Shell* (*SSH*), *Firewall* (*UFW*), Fail2Ban, *website monitoring* berbasis Flask, Telegram Bot, serta Kali Linux sebagai media simulasi serangan. Sistem dirancang agar mampu membaca aktivitas autentikasi pada berkas *auth.log*, mendeteksi pola percobaan login yang mencurigakan melalui metode *log analysis*, melakukan pemblokiran alamat IP secara otomatis menggunakan *Firewall*, menampilkan informasi keamanan server melalui *website monitoring*, serta mengirimkan notifikasi kepada administrator secara *real-time* melalui Telegram Bot.

4. Implementasi Sistem

   Tahap implementasi sistem dilakukan dengan menerapkan seluruh hasil perancangan ke dalam lingkungan penelitian yang meliputi instalasi Ubuntu Server, konfigurasi layanan *Secure Shell* (*SSH*), pemasangan *Firewall* (*UFW*), konfigurasi Fail2Ban, pengembangan *website monitoring* menggunakan *framework* Flask, serta

integrasi Telegram Bot sebagai media notifikasi. Seluruh komponen tersebut dikonfigurasi agar dapat bekerja secara terintegrasi dalam mendeteksi aktivitas autentikasi yang mencurigakan berdasarkan analisis berkas *auth.log*, melakukan pemblokiran alamat IP penyerang secara otomatis, menampilkan kondisi keamanan server melalui *website monitoring*, dan mengirimkan informasi hasil deteksi kepada administrator secara *real-time*.

5. Simulasi Serangan

   Tahap simulasi serangan dilakukan untuk menguji kemampuan sistem keamanan dalam mendeteksi dan memitigasi serangan terhadap layanan *Secure Shell* (*SSH*). Simulasi menggunakan *Hydra* pada sistem operasi Kali Linux dengan dua skenario, yaitu *brute force attack* dan *dictionary attack*. Seluruh aktivitas autentikasi yang dihasilkan selama proses pengujian akan tercatat pada berkas *auth.log* di Ubuntu Server. Data tersebut kemudian dianalisis oleh Fail2Ban untuk mendeteksi pola percobaan login yang melebihi batas parameter yang telah ditentukan. Apabila pola serangan teridentifikasi, sistem akan menjalankan aturan *Firewall* untuk memblokir alamat IP penyerang secara otomatis, menampilkan informasi pada *website monitoring*, serta mengirimkan notifikasi kepada administrator melalui Telegram Bot.

6. Monitoring dan Analisis Log

   Tahap monitoring dan analisis *log* dilakukan dengan memanfaatkan berkas *auth.log* sebagai sumber utama informasi aktivitas autentikasi layanan *SSH*. Seluruh data yang tercatat, seperti alamat IP, waktu kejadian, status autentikasi, dan jumlah kegagalan login, dianalisis menggunakan metode *log analysis* oleh Fail2Ban. Hasil analisis kemudian diintegrasikan ke dalam *website monitoring* berbasis Flask sehingga administrator dapat memantau kondisi server, aktivitas login, alamat IP yang diblokir, status *Firewall*, serta kondisi Fail2Ban secara *real-time*. Selain itu, setiap aktivitas yang memenuhi kriteria serangan akan menghasilkan notifikasi otomatis melalui Telegram Bot sebagai bentuk peringatan dini kepada administrator.

7. Pengujian dan Evaluasi Sistem

   Tahap pengujian dan evaluasi dilakukan untuk mengetahui tingkat efektivitas sistem keamanan yang telah dibangun dalam mendeteksi dan memitigasi serangan *brute force* dan *dictionary attack*. Pengujian dilakukan berdasarkan beberapa parameter, yaitu waktu respons pemblokiran alamat IP (*response time*), tingkat keberhasilan Fail2Ban dalam mendeteksi pola serangan, keberhasilan *Firewall* dalam melakukan pemblokiran, keakuratan informasi yang ditampilkan pada *website*

monitoring, serta keberhasilan pengiriman notifikasi melalui Telegram Bot. Data hasil pengujian selanjutnya dianalisis untuk menilai kinerja sistem secara keseluruhan dan memastikan seluruh komponen bekerja sesuai dengan rancangan.

8. Kesimpulan

   Tahap akhir penelitian dilakukan dengan menyusun kesimpulan berdasarkan hasil implementasi dan pengujian sistem keamanan yang telah dilakukan. Kesimpulan disusun untuk menjawab rumusan masalah penelitian mengenai efektivitas integrasi *Firewall*, Fail2Ban, *website monitoring* berbasis Flask, dan Telegram Bot dalam mendeteksi, memantau, serta memitigasi serangan *brute force* dan *dictionary attack* pada layanan *SSH*. Selain itu, pada tahap ini juga disampaikan beberapa rekomendasi yang dapat digunakan sebagai acuan dalam pengembangan sistem keamanan server pada penelitian selanjutnya.

3.3 Rancangan Penelitian

Rancangan penelitian disusun sebagai pedoman dalam proses perancangan, implementasi, dan pengujian sistem keamanan server Linux yang terintegrasi dengan *Firewall*, Fail2Ban, *website monitoring*, dan Telegram Bot. Penelitian diawali dengan identifikasi permasalahan keamanan pada layanan *Secure Shell* (*SSH*) yang rentan terhadap serangan *brute force* dan *dictionary attack*. Selanjutnya dilakukan studi literatur untuk memperoleh landasan teori mengenai keamanan jaringan, metode *log analysis*, mekanisme kerja Fail2Ban, konfigurasi *Firewall*, pengembangan aplikasi berbasis Flask, serta pemanfaatan Telegram Bot sebagai media notifikasi.

Tahap berikutnya adalah perancangan arsitektur sistem yang terdiri atas Ubuntu Server sebagai server target, Kali Linux sebagai mesin penyerang, Fail2Ban sebagai sistem pendeteksi serangan, *Firewall* (*UFW*) sebagai mekanisme pemblokiran alamat IP, *website monitoring* berbasis Flask sebagai media pemantauan keamanan server, serta Telegram Bot sebagai media penyampaian notifikasi kepada administrator. Seluruh komponen dirancang agar saling terintegrasi sehingga proses deteksi, pemblokiran, pemantauan, dan pemberitahuan dapat berjalan secara otomatis.

Setelah proses perancangan selesai, dilakukan implementasi sistem pada lingkungan *virtual machine*. Selanjutnya dilakukan simulasi serangan menggunakan *Hydra* untuk menghasilkan aktivitas *brute force* dan *dictionary attack* terhadap layanan *SSH*. Data autentikasi yang tercatat pada berkas *auth.log* dianalisis menggunakan metode *log analysis* oleh Fail2Ban untuk mendeteksi pola serangan. Apabila jumlah percobaan login gagal

melebihi parameter yang telah ditentukan, sistem akan melakukan pemblokiran alamat IP melalui *Firewall*, menampilkan informasi pada *website monitoring*, serta mengirimkan notifikasi secara *real-time* melalui Telegram Bot.

Tahap akhir penelitian dilakukan dengan mengevaluasi kinerja sistem berdasarkan beberapa parameter pengujian, yaitu waktu respons (*response time*) pemblokiran, keberhasilan deteksi serangan, keberhasilan pemblokiran alamat IP, akurasi informasi pada *website monitoring*, serta keberhasilan pengiriman notifikasi Telegram. Hasil evaluasi tersebut digunakan untuk menilai efektivitas sistem keamanan yang dikembangkan dalam memitigasi serangan *brute force* dan *dictionary attack* pada layanan *SSH*.

3.2.1 Penelitian Literatur

Penelitian literatur merupakan tahapan untuk memperoleh landasan teoritis yang mendukung proses perancangan dan implementasi sistem. Pada tahap ini dilakukan pengumpulan serta pengkajian berbagai referensi yang berkaitan dengan keamanan server Linux, *Firewall*, Fail2Ban, metode *log analysis*, *website monitoring* berbasis Flask, Telegram Bot, serta teknik serangan *brute force attack* dan *dictionary attack*. Referensi diperoleh dari jurnal ilmiah, buku, prosiding, dan dokumentasi resmi yang relevan dengan topik penelitian. Hasil studi literatur digunakan sebagai dasar dalam menentukan metode penelitian, perancangan sistem, serta mengidentifikasi celah (*research gap*) dari penelitian terdahulu sehingga sistem yang dikembangkan memiliki kontribusi yang lebih baik dalam aspek monitoring maupun mitigasi keamanan server.

3.2.2 Perancangan Sistem

Perancangan sistem dilakukan untuk membangun arsitektur keamanan server yang mampu mendeteksi, memantau, dan memitigasi serangan terhadap layanan *Secure Shell* (*SSH*). Sistem dirancang dengan mengintegrasikan Ubuntu Server sebagai server target, *Firewall* (*UFW*), Fail2Ban, *website monitoring* berbasis Flask, Telegram Bot, serta Kali Linux sebagai mesin penyerang. Fail2Ban bertugas menganalisis aktivitas autentikasi pada berkas *auth.log*, sedangkan *Firewall* digunakan untuk melakukan pemblokiran otomatis terhadap alamat IP yang terindikasi melakukan serangan. Informasi keamanan yang diperoleh kemudian ditampilkan melalui *website monitoring* sehingga administrator dapat memantau kondisi server secara *real-time*. Selain itu, sistem juga mengirimkan notifikasi otomatis melalui Telegram Bot ketika terjadi aktivitas serangan maupun proses pemblokiran alamat IP.

Adapun arsitektur sistem terdiri atas:

1. Ubuntu Server sebagai server target.
2. Layanan *Secure Shell* (*SSH*) sebagai objek pengujian.
3. *Firewall* (*UFW*) sebagai mekanisme pemfilteran jaringan.
4. Fail2Ban sebagai sistem deteksi dan pemblokiran otomatis.
5. Website Monitoring berbasis Flask sebagai media monitoring server.
6. Telegram Bot sebagai media notifikasi *real-time*.
7. Kali Linux sebagai mesin simulasi serangan.

3.2.3 Implementasi Firewall dan Fail2Ban

Tahap implementasi dilakukan dengan melakukan konfigurasi Firewall dan Fail2Ban pada Ubuntu Server. Firewall dikonfigurasi untuk membatasi akses terhadap layanan SSH berdasarkan aturan keamanan tertentu, sedangkan Fail2Ban digunakan untuk membaca serta menganalisis file *auth.log* secara otomatis.

Parameter utama yang digunakan pada Fail2Ban meliputi:

1. *maxretry* : jumlah maksimum percobaan login gagal
2. *findtime* : interval waktu pengamatan login gagal
3. *bantime* : durasi pemblokiran alamat IP attacker

Ketika jumlah percobaan login gagal melebihi nilai *maxretry* dalam periode *findtime*, maka Fail2Ban akan menjalankan aturan firewall untuk memblokir alamat IP *attacker* selama durasi *bantime*. Selain itu, sistem juga diintegrasikan dengan *Telegram Bot* menggunakan script otomatis agar administrator memperoleh notifikasi real-time ketika terjadi aktivitas serangan maupun pemblokiran alamat IP.

3.2.4 Implementasi Serangan Brute Force dan Dictionary Attack

Tahap ini dilakukan untuk menguji ketahanan sistem keamanan terhadap serangan autentikasi pada layanan SSH menggunakan tools Hydra dari Kali Linux.

1. Brute Force Attack

   Brute force attack dilakukan dengan mencoba berbagai kombinasi username dan password secara terus-menerus hingga ditemukan kredensial yang sesuai dengan sistem target. Pada penelitian ini, Hydra digunakan untuk menghasilkan percobaan login dalam jumlah besar terhadap layanan SSH server Ubuntu. Pengujian brute force bertujuan untuk mengetahui kemampuan Fail2Ban dalam mendeteksi pola login

gagal secara berulang serta mengukur kecepatan sistem dalam melakukan pemblokiran otomatis terhadap attacker.

2. Dictionary Attack

Dictionary attack dilakukan menggunakan file wordlist yang berisi kumpulan password umum seperti rockyou.txt. Hydra akan mencoba setiap kombinasi password yang terdapat pada wordlist terhadap akun target layanan SSH. Pengujian ini bertujuan untuk mengukur efektivitas metode log analysis dalam mengenali pola autentikasi mencurigakan berbasis dictionary attack serta mengevaluasi kemampuan Firewall dan Fail2Ban dalam mencegah akses ilegal terhadap server Linux.

3.2.5 Implementasi *Website Monitoring* dan Telegram Bot

Implementasi *website monitoring* dan Telegram Bot dilakukan untuk melengkapi mekanisme keamanan server dengan menyediakan fasilitas pemantauan dan penyampaian informasi secara terintegrasi. *Website monitoring* yang dibangun menggunakan *framework* Flask berfungsi menampilkan kondisi server secara *real-time*, meliputi status layanan *Secure Shell* (*SSH*), status *Firewall*, status Fail2Ban, aktivitas autentikasi pada berkas *auth.log*, serta daftar alamat IP yang berhasil diblokir akibat terdeteksi melakukan serangan. Selain itu, Telegram Bot diintegrasikan sebagai media notifikasi otomatis yang akan mengirimkan informasi kepada administrator setiap kali sistem mendeteksi aktivitas mencurigakan atau melakukan pemblokiran terhadap alamat IP penyerang. Integrasi kedua komponen tersebut bertujuan untuk mempermudah proses monitoring, mempercepat respons administrator terhadap insiden keamanan, serta meningkatkan efektivitas pengelolaan keamanan server Linux secara menyeluruh.

3.2.6 Analisis *Log* Sistem

Analisis *log* dilakukan dengan memanfaatkan berkas *auth.log* sebagai sumber utama informasi aktivitas autentikasi layanan *SSH*. Berkas tersebut mencatat berbagai aktivitas, seperti waktu kejadian, alamat IP sumber, status login berhasil maupun gagal, serta informasi pengguna yang melakukan autentikasi. Fail2Ban melakukan analisis menggunakan pola *regular expression* (*regex*) untuk mendeteksi percobaan login gagal yang terjadi secara berulang. Ketika pola serangan berhasil dikenali, sistem secara otomatis menjalankan proses pemblokiran melalui *Firewall*, menampilkan informasi pada *website monitoring*, serta mengirimkan notifikasi kepada administrator melalui Telegram Bot.

Data hasil analisis *log* digunakan untuk:

1. Mengidentifikasi aktivitas *brute force attack*.

2. Mengidentifikasi aktivitas *dictionary attack*.
3. Menghitung jumlah percobaan login gagal.
4. Menampilkan daftar alamat IP yang diblokir.
5. Menyajikan informasi keamanan server pada *website monitoring*.
6. Mengirimkan notifikasi keamanan secara *real-time* melalui Telegram Bot.

3.2.7 Pengujian dan Analisa Sistem

Pengujian dilakukan untuk mengevaluasi efektivitas sistem keamanan server Linux dalam mendeteksi serta memitigasi brute force dan dictionary attack pada layanan SSH.

1. Response Time Pemblokiran

   Response time digunakan untuk mengukur waktu yang dibutuhkan sistem sejak serangan terdeteksi hingga alamat IP attacker berhasil diblokir oleh firewall.

   Rumus response time:

$$Response\ Time = T_{block} - T_{detect}$$

   Keterangan:

   - T_{detect}= waktu sistem mendeteksi serangan
   - T_{block}= waktu firewall melakukan pemblokiran IP

   Semakin kecil nilai response time, maka semakin cepat sistem merespons ancaman keamanan.

2. Tingkat Keberhasilan Pemblokiran

   Parameter ini digunakan untuk mengukur persentase keberhasilan sistem dalam memblokir alamat IP attacker yang melakukan percobaan login ilegal.

$$Success\ Rate = \frac{Jumlah\ IP\ Berhasil\ Diblokir}{Jumlah\ Percobaan\ Serangan} \times 100\%$$

   Semakin tinggi nilai success rate, maka semakin efektif sistem keamanan yang diterapkan.

3. Keberhasilan Pengiriman Notifikasi Telegram

   Parameter ini digunakan untuk mengukur kemampuan sistem dalam mengirimkan alert secara real-time kepada administrator ketika terjadi aktivitas serangan.

$$Notification\ Rate = \frac{Jumlah\ Notifikasi\ Terkirim}{Jumlah\ Serangan\ Terdeteksi} \times 100\%$$

Keberhasilan notifikasi menunjukkan tingkat keandalan sistem monitoring keamanan berbasis Telegram Bot.

4. Akurasi Deteksi Serangan

Akurasi digunakan untuk mengukur kemampuan sistem dalam mendeteksi aktivitas serangan berdasarkan analisis log autentikasi.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Keterangan:

- TP (True Positive) = serangan berhasil terdeteksi
- TN (True Negative) = aktivitas normal berhasil dikenali
- FP (False Positive) = aktivitas normal terdeteksi sebagai serangan
- FN (False Negative) = serangan tidak terdeteksi sistem

3.4 Bahan dan Peralatan Yang Digunakan

Dalam mendukung pelaksanaan penelitian serta proses implementasi sistem keamanan jaringan server Linux, diperlukan berbagai bahan dan peralatan yang memadai. Bahan dan peralatan tersebut terdiri dari perangkat keras (*hardware*), perangkat lunak (*software*), serta data penelitian yang digunakan dalam proses analisis dan pengujian sistem. Pemilihan komponen-komponen ini didasarkan pada kebutuhan sistem yang akan dibangun, sehingga mampu mendukung seluruh tahapan penelitian mulai dari perancangan hingga evaluasi.

Tabel 3.1 Perangkat Keras (*Hardware*)

Nama	Spesifikasi
Personal Computer HP 245 G8 Notebook PC	1. Sistem Operasi : *Windows 11 Home Single Language* 2. *Processor : AMD Ryzen 3 3250U with Radeon Graphics (2.60 GHz)* 3. *Ram : 12 GB* 4. *Storage : 237 GB* 5. *Network : Virtual Network Adapter*

Tabel 3.2 Perangkat Lunak (*Software*)

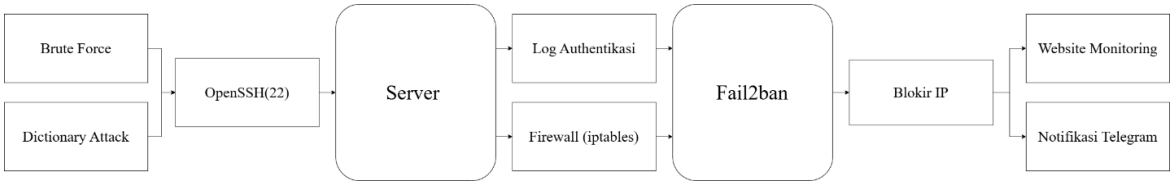
No	Nama *Software*	Fungsi
1	Ubuntu Server 24.04 LTS	Sistem operasi server yang menjadi objek penelitian
2	VMware Workstation	Media virtualisasi lingkungan pengujian
3	Kali Linux	Sistem operasi untuk melakukan simulasi serangan
4	UFW (*Uncomplicated Firewall*)	Mengatur aturan keamanan jaringan dan pemblokiran alamat IP
5	Fail2Ban	Mendeteksi aktivitas login mencurigakan dan melakukan pemblokiran otomatis
6	Hydra	Melakukan simulasi *brute force attack* dan *dictionary attack*
7	Python 3	Bahasa pemrograman untuk pengembangan sistem
8	Flask	*Framework* pengembangan *website monitoring*
9	Telegram Bot API	Mengirim notifikasi keamanan secara *real-time* kepada administrator
10	*Auth.log*	Sumber data aktivitas autentikasi untuk proses *log analysis*
11	Web Browser (Google Chrome/Mozilla Firefox)	Mengakses dan menguji *website monitoring*

BAB IV
ANALISIS DAN PERANCANGAN

4.1 Analisa Sistem

Analisis sistem merupakan tahapan krusial untuk mengidentifikasi, memetakan, dan mengevaluasi seluruh kebutuhan fungsional maupun non-fungsional dari sistem keamanan server Linux yang dibangun. Pada tahap ini, kelemahan bawaan dari layanan OpenSSH pada port default (Port 22) dianalisis guna merumuskan mekanisme pertahanan terintegrasi. Penambahan sistem pemantauan berbasis *website monitoring* menggunakan mikro-*framework* Flask ditujukan untuk menyelesaikan keterbatasan administrator dalam mengamati aktivitas server secara konvensional. Melalui analisis sistem ini, ditentukan bagaimana data mentah dari berkas log autentikasi, aturan *firewall*, dan metrik performa perangkat keras diolah secara otomatis menjadi informasi visual yang responsif bagi administrator jaringan.

4.1.1 Blok Diagram



Gambar 4.1 Blok Diagram

Adapun fungsi dan interaksi masing-masing blok dijelaskan sebagai berikut.

1. Brute Force berfungsi sebagai simulasi serangan yang dilakukan dengan mencoba berbagai kombinasi *username* dan *password* secara berulang terhadap layanan *Secure Shell* (*SSH*). Serangan ini digunakan untuk menguji kemampuan sistem keamanan dalam mendeteksi percobaan autentikasi yang tidak sah.
2. Dictionary Attack berfungsi sebagai simulasi serangan yang memanfaatkan *wordlist* berisi kumpulan kata sandi umum untuk melakukan percobaan login ke layanan *SSH*. Serangan ini digunakan untuk menguji efektivitas sistem dalam mengenali pola autentikasi yang mencurigakan berdasarkan metode *log analysis*.
3. OpenSSH (Port 22) berfungsi sebagai layanan *Secure Shell* (*SSH*) yang menerima seluruh permintaan koneksi dari pengguna maupun penyerang.

Layanan ini menjadi objek utama pengujian karena merupakan media akses administrasi server dari jarak jauh yang rentan terhadap serangan *brute force* dan *dictionary attack*.

4. Server berfungsi sebagai pusat pemrosesan sistem yang menjalankan sistem operasi Ubuntu Server beserta layanan keamanan yang terdiri atas *Firewall*, Fail2Ban, *website monitoring*, dan Telegram Bot. Seluruh proses autentikasi, pencatatan log, analisis keamanan, serta pengiriman informasi dilakukan pada server ini.
5. Log Autentikasi berfungsi sebagai media pencatatan seluruh aktivitas autentikasi pada layanan *SSH*. Berkas *auth.log* menyimpan informasi seperti alamat *IP* sumber, waktu kejadian, status login berhasil maupun gagal, yang selanjutnya digunakan sebagai sumber data dalam proses *log analysis*.
6. Firewall (*iptables*) berfungsi sebagai mekanisme pengamanan jaringan yang mengatur lalu lintas data berdasarkan aturan (*rule*) yang telah ditentukan. *Firewall* akan menerima instruksi dari Fail2Ban untuk menambahkan aturan pemblokiran terhadap alamat *IP* yang terindikasi melakukan serangan sehingga akses menuju layanan *SSH* dapat dihentikan secara otomatis.
7. Fail2Ban berfungsi sebagai sistem deteksi dan pencegahan serangan yang melakukan pemantauan terhadap berkas *auth.log*. Fail2Ban menganalisis jumlah percobaan login gagal berdasarkan parameter *maxretry*, *findtime*, dan *bantime*. Apabila jumlah percobaan login gagal melebihi batas yang telah ditentukan, sistem akan menjalankan proses pemblokiran melalui *Firewall*.
8. Blokir IP berfungsi sebagai mekanisme mitigasi terhadap serangan yang telah terdeteksi. Pada tahap ini, alamat *IP* penyerang dimasukkan ke dalam aturan *Firewall* sehingga seluruh koneksi dari alamat *IP* tersebut ditolak selama periode pemblokiran sesuai konfigurasi Fail2Ban.
9. Website Monitoring berfungsi sebagai antarmuka berbasis web yang dibangun menggunakan *framework* Flask untuk menampilkan kondisi keamanan server secara *real-time*. Informasi yang ditampilkan meliputi penggunaan sumber daya server (*CPU*, *RAM*, dan penyimpanan), status *Firewall*, status Fail2Ban, aktivitas autentikasi, serta daftar alamat *IP* yang berhasil diblokir sehingga administrator dapat memantau kondisi server secara terpusat.

10. Notifikasi Telegram berfungsi sebagai media penyampaian informasi keamanan kepada administrator secara *real-time*. Ketika Fail2Ban berhasil mendeteksi dan memblokir alamat *IP* penyerang, sistem secara otomatis mengirimkan notifikasi melalui Telegram Bot yang berisi informasi mengenai waktu kejadian, alamat *IP* penyerang, layanan yang diserang, dan status pemblokiran sehingga administrator dapat segera melakukan tindakan apabila diperlukan.

4.1.2 Analisa Sistem Berjalan dan Permasalahan

Layanan *Secure Shell* (*SSH*) pada sistem operasi Linux merupakan layanan yang umum digunakan oleh administrator untuk melakukan administrasi server secara jarak jauh melalui jaringan. Secara *default*, layanan *SSH* berjalan pada port 22 dengan mekanisme autentikasi menggunakan *username* dan *password*. Meskipun telah menerapkan proses autentikasi, layanan tersebut masih memiliki potensi kerentanan terhadap berbagai bentuk serangan, terutama *brute force attack* dan *dictionary attack*, apabila tidak didukung oleh mekanisme keamanan yang mampu mendeteksi dan merespons aktivitas mencurigakan secara otomatis. Berdasarkan hasil analisis terhadap sistem yang berjalan, ditemukan beberapa permasalahan yang dapat memengaruhi tingkat keamanan server, yaitu sebagai berikut.

1. Belum adanya mekanisme pembatasan percobaan login secara otomatis. Layanan *SSH* masih menerima percobaan autentikasi secara berulang tanpa melakukan pemblokiran terhadap alamat *IP* yang melakukan kegagalan login dalam jumlah tertentu. Kondisi ini memberikan peluang bagi penyerang untuk melakukan *brute force attack* maupun *dictionary attack* hingga memperoleh kredensial yang valid.
2. Belum tersedia sistem mitigasi otomatis terhadap aktivitas serangan. Sistem hanya mencatat seluruh aktivitas autentikasi pada berkas *auth.log* tanpa disertai mekanisme yang mampu menganalisis log dan melakukan tindakan pencegahan secara langsung. Akibatnya, administrator harus melakukan pemeriksaan log secara manual sehingga respons terhadap insiden keamanan menjadi kurang efektif.
3. Belum tersedia media monitoring keamanan secara terpusat. Informasi mengenai kondisi server, status *Firewall*, status Fail2Ban, aktivitas autentikasi, serta daftar alamat *IP* yang diblokir belum disajikan dalam

bentuk antarmuka yang mudah dipantau. Hal tersebut menyebabkan administrator mengalami kesulitan dalam melakukan pemantauan kondisi keamanan server secara menyeluruh dan *real-time*.

4. Belum adanya mekanisme notifikasi keamanan secara langsung kepada administrator. Ketika terjadi percobaan serangan maupun proses pemblokiran alamat *IP*, sistem tidak memberikan pemberitahuan secara otomatis. Kondisi ini menyebabkan administrator berpotensi terlambat mengetahui adanya insiden keamanan karena harus melakukan pemeriksaan server secara berkala.
5. Pemanfaatan data log belum optimal sebagai sumber informasi keamanan. Berkas *auth.log* hanya berfungsi sebagai media pencatatan aktivitas autentikasi tanpa dimanfaatkan untuk menghasilkan informasi yang dapat mendukung proses analisis maupun pengambilan keputusan. Padahal, data log tersebut dapat digunakan sebagai dasar deteksi serangan, proses pemblokiran otomatis, penyajian informasi pada *website monitoring*, serta pengiriman notifikasi *real-time* kepada administrator.

4.1.3 Analisa Mekanisme Serangan (*Brute Force & Dictionary Attack*)

Untuk merancang sistem pertahanan yang efektif, diperlukan analisa mendalam mengenai karakteristik dan pola dari metode serangan yang kerap menargetkan layanan SSH:

1. *Brute Force Attack*: Serangan ini bekerja dengan mengombinasikan karakter, angka, dan simbol secara acak, sekuensial, dan terus-menerus untuk menebak kata sandi yang tepat. Dari aspek analisis sistem, serangan ini dicirikan dengan adanya lonjakan permintaan koneksi yang sangat masif dan padat dari satu atau beberapa alamat IP dalam rentang waktu yang sangat singkat.
2. *Dictionary Attack*: Serangan ini memiliki pola yang lebih terarah dan efisien dibandingkan *brute force*. Penyerang memanfaatkan berkas daftar kata (*wordlist*) yang berisi ribuan hingga jutaan kombinasi kata sandi populer, frasa yang umum digunakan, atau data kebocoran kredensial yang ada di internet. Pola serangan ini sering kali berhasil menembus server karena kecenderungan pengguna atau administrator dalam memilih kata sandi yang mudah diingat namun lemah secara metrik keamanan.

4.1.4 Analisa Log Layanan SSH

Sistem operasi Linux memiliki subsistem pencatatan (*logging*) yang sangat terstruktur. Setiap aktivitas yang berkaitan dengan proses autentikasi, baik yang berhasil maupun yang gagal, akan dicatat secara otomatis oleh *daemon* layanan ke dalam berkas log spesifik. Pada sistem operasi berbasis Debian atau Ubuntu, log tersebut disimpan di /var/log/auth.log, sedangkan pada sistem berbasis RHEL atau CentOS tersimpan di /var/log/secure.

Ketika serangan *Brute Force* atau *Dictionary Attack* berlangsung, layanan SSH daemon (*sshd*) akan menghasilkan baris-baris pesan kegagalan (*failed events*) dengan pola string yang konsisten. Berikut adalah contoh visualisasi dari potongan baris log autentikasi saat terjadi serangan:

```
Jun 25 07:35:12 homeserver sshd[2182]: Failed password for ramadhanazua from 192.168.186.133 port 38138 ssh2
Jun 25 07:35:12 homeserver sshd[2186]: Failed password for ramadhanazua from 192.168.186.133 port 38144 ssh2
Jun 25 07:35:13 homeserver sshd[2194]: Failed password for ramadhanazua from 192.168.186.133 port 38238 ssh2
Jun 25 07:35:14 homeserver sshd[2188]: Failed password for ramadhanazua from 192.168.186.133 port 38176 ssh2
Jun 25 07:35:14 homeserver sshd[2189]: Failed password for ramadhanazua from 192.168.186.133 port 38190 ssh2
Jun 25 07:35:14 homeserver sshd[2191]: Failed password for ramadhanazua from 192.168.186.133 port 38218 ssh2
Jun 25 07:35:14 homeserver sshd[2193]: Failed password for ramadhanazua from 192.168.186.133 port 38236 ssh2
Jun 25 07:35:14 homeserver sshd[2190]: Failed password for ramadhanazua from 192.168.186.133 port 38200 ssh2
Jun 25 07:35:14 homeserver sshd[2192]: Failed password for ramadhanazua from 192.168.186.133 port 38234 ssh2
Jun 25 07:35:14 homeserver sshd[2202]: Failed password for ramadhanazua from 192.168.186.133 port 38268 ssh2
```

Gambar 4.2 Authentication Log

Berdasarkan analisis struktur teks di atas, setiap baris log mengandung informasi krusial yang dapat diekstraksi, yaitu: stempel waktu kejadian (*timestamp*), nama mesin host, nama layanan (*sshd*) beserta nomor *Process ID* (PID), status atau alasan kegagalan (*Failed password*), nama akun yang ditargetkan (*invalid user admin* atau *root*), serta alamat IP asal berikut nomor port dinamis yang digunakan oleh penyerang (*from 192.168.186.133 port 38238*). Pola *string* konsisten inilah yang nantinya dimanfaatkan sebagai basis deteksi oleh Fail2Ban menggunakan ekspresi reguler.

4.1.5 Analisa Kebutuhan Perangkat Keras dan Perangkat Lunak

Implementasi sistem keamanan pada penelitian ini dilakukan pada lingkungan *virtual machine* sehingga proses konfigurasi, simulasi serangan, dan pengujian dapat dilaksanakan secara terisolasi tanpa memengaruhi jaringan produksi. Kebutuhan sistem terdiri atas perangkat keras sebagai media implementasi, perangkat lunak server sebagai sistem yang diamankan, serta perangkat lunak penguji yang digunakan untuk melakukan simulasi serangan dan pemantauan sistem. Adapun kebutuhan tersebut dijelaskan sebagai berikut.

1. Perangkat Keras (*Hardware*)

Perangkat keras yang digunakan sebagai *host* utama berupa komputer atau laptop dengan spesifikasi minimal prosesor AMD Ryzen 3 3250U atau setara, memori utama (*RAM*) sebesar 12 GB, media penyimpanan berbasis Solid State Drive (SSD) berkapasitas 256 GB, serta *Virtual Network Adapter* untuk mendukung komunikasi antar *virtual machine*. Spesifikasi tersebut dipilih agar seluruh mesin virtual dapat berjalan secara bersamaan dengan performa yang stabil selama proses implementasi dan pengujian.

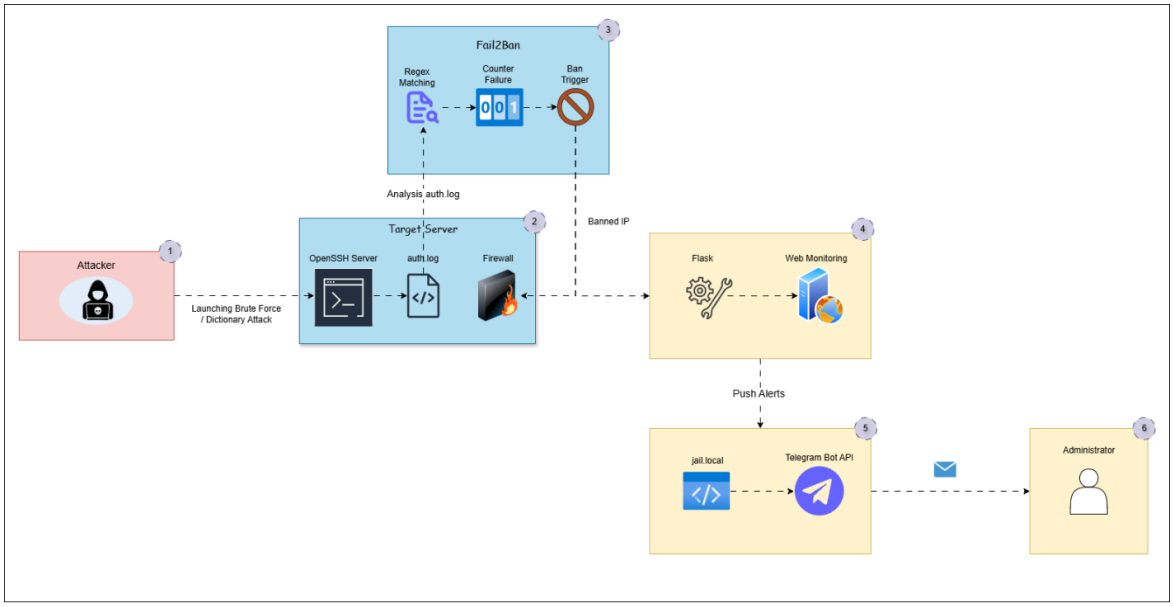
2. Perangkat Lunak Server (*Target*)

Perangkat lunak pada sisi server terdiri atas Ubuntu Server 22.04 LTS sebagai sistem operasi utama yang menjalankan layanan OpenSSH Server pada port 22. Sistem keamanan diimplementasikan menggunakan UFW (*Uncomplicated Firewall*) atau *iptables* sebagai mekanisme penyaringan lalu lintas jaringan, serta Fail2Ban sebagai sistem deteksi dan pemblokiran otomatis berdasarkan analisis berkas *auth.log*. Selain itu, server juga menjalankan Python 3, Flask, dan SQLite sebagai pendukung pembangunan *website monitoring*, sedangkan Telegram Bot API digunakan sebagai media pengiriman notifikasi keamanan secara *real-time* kepada administrator.

3. Perangkat Lunak Penguji (*Attacker & Monitoring*)

Perangkat lunak penguji menggunakan Kali Linux sebagai mesin *attacker* yang berfungsi untuk melakukan simulasi *brute force attack* dan *dictionary attack* terhadap layanan *SSH* menggunakan aplikasi Hydra. Aktivitas serangan yang dihasilkan akan memicu pencatatan pada berkas *auth.log* sehingga dapat dianalisis oleh Fail2Ban. Selanjutnya, hasil deteksi serangan, status *Firewall*, daftar alamat *IP* yang diblokir, serta informasi kondisi server akan ditampilkan melalui *website monitoring* berbasis Flask dan dikirimkan secara otomatis kepada administrator melalui Telegram Bot sebagai notifikasi *real-time*.

4.2 Perancangan Topologi Jaringan Simulasi



Gambar 4.3 Topologi Jaringan Simulasi

Dari Topologi diatas, berikut uraiannya:

1. *Attacker* merupakan pihak yang melakukan simulasi serangan terhadap layanan *SSH* pada server target. Serangan dilakukan menggunakan aplikasi Hydra yang dijalankan pada sistem operasi Kali Linux dengan metode *brute force* maupun *dictionary attack*. Tujuan simulasi ini adalah menghasilkan percobaan autentikasi secara berulang sehingga dapat menguji kemampuan sistem keamanan dalam mendeteksi dan menangani aktivitas yang mencurigakan.
2. *Target Server* merupakan server Linux yang menyediakan layanan *OpenSSH Server* sebagai media akses jarak jauh. Setiap percobaan login yang diterima akan dicatat ke dalam berkas *auth.log*. Selain itu, server telah dikonfigurasi menggunakan *Firewall* (*iptables*) untuk menerapkan aturan pemblokiran terhadap alamat IP yang dianggap melakukan aktivitas berbahaya. Server ini menjadi objek utama yang diamankan pada penelitian.
3. *Fail2Ban* berfungsi sebagai sistem deteksi dan pencegahan serangan berbasis *log analysis*. Fail2Ban secara berkala membaca berkas *auth.log* menggunakan mekanisme *regex matching* untuk mengidentifikasi pola kegagalan autentikasi. Apabila jumlah percobaan login gagal telah melebihi nilai parameter *maxretry* dalam rentang waktu *findtime*, maka Fail2Ban akan menjalankan aksi *ban trigger* dengan menambahkan aturan pemblokiran pada *Firewall*. Mekanisme ini

memungkinkan sistem melakukan mitigasi serangan secara otomatis tanpa memerlukan intervensi administrator.

4. *Website Monitoring* merupakan aplikasi berbasis *Flask* yang berfungsi sebagai media pemantauan kondisi keamanan server secara *real-time*. Website ini menampilkan berbagai informasi penting, seperti status *Firewall*, status *Fail2Ban*, daftar alamat IP yang diblokir, riwayat aktivitas serangan, serta data hasil analisis log. Informasi tersebut diperoleh dari server melalui proses pembacaan log dan status layanan sehingga administrator dapat memantau kondisi keamanan server melalui antarmuka web secara lebih mudah dan terpusat.
5. *Telegram Bot* berfungsi sebagai sistem notifikasi *real-time* yang mengirimkan informasi kepada administrator ketika terjadi aktivitas keamanan tertentu. Setelah Website Monitoring memperoleh data hasil deteksi dari Fail2Ban, sistem akan menjalankan skrip *Python* (*jail.local*) untuk mengirimkan pesan melalui Telegram Bot API. Notifikasi yang dikirim meliputi informasi waktu kejadian, alamat IP penyerang, layanan yang diserang, serta status pemblokiran yang telah dilakukan oleh sistem.
6. *Administrator* merupakan pengguna yang bertugas melakukan pengawasan terhadap keamanan server. Administrator menerima notifikasi secara langsung melalui aplikasi Telegram dan dapat melakukan pemantauan kondisi server melalui Website Monitoring. Dengan adanya kedua media tersebut, administrator dapat mengetahui aktivitas serangan, status pemblokiran alamat IP, serta kondisi layanan keamanan secara cepat sehingga proses pengambilan keputusan dapat dilakukan dengan lebih efektif.

4.3 Perancangan Arsitektur dan Alur Sistem Keamanan

Perancangan arsitektur sistem keamanan pada penelitian ini dilakukan dengan mengintegrasikan beberapa komponen utama, yaitu OpenSSH Server**,** Fail2Ban, Firewall (iptables), berkas log auth.log**,** Website Monitoring berbasis *Flask*, dan Telegram Bot. Integrasi antar komponen tersebut dirancang untuk membentuk sistem keamanan yang mampu mendeteksi, menganalisis, memitigasi, serta memantau aktivitas serangan *brute force* dan *dictionary attack* pada layanan *Secure Shell* (*SSH*) secara otomatis. Selain berfungsi sebagai mekanisme pertahanan, sistem juga menyediakan media monitoring berbasis web dan notifikasi *real-time* sehingga administrator dapat memantau kondisi keamanan server dengan lebih efektif.

Alur kerja sistem diawali ketika server menerima permintaan autentikasi *SSH* dari pengguna maupun penyerang. Seluruh proses autentikasi diproses oleh layanan *OpenSSH Server*, kemudian setiap aktivitas login, baik berhasil maupun gagal, dicatat ke dalam berkas /var/log/auth.log. Berkas log tersebut dipantau secara *real-time* oleh Fail2Ban menggunakan mekanisme *regex matching* untuk mengidentifikasi pola kegagalan autentikasi serta menghitung jumlah percobaan login dari setiap alamat IP berdasarkan parameter *maxretry*, *findtime*, dan *bantime*. Apabila jumlah percobaan login gagal melebihi batas yang telah ditentukan, Fail2Ban akan memberikan perintah kepada *Firewall* (*iptables*) untuk memblokir alamat IP penyerang secara otomatis.

Setelah proses pemblokiran berhasil dilakukan, informasi mengenai aktivitas serangan, status layanan keamanan, serta daftar alamat IP yang diblokir akan ditampilkan pada Website Monitoring berbasis *Flask*. Website ini berfungsi sebagai media visualisasi yang menyajikan kondisi keamanan server secara *real-time*, sehingga administrator dapat memantau aktivitas sistem melalui antarmuka web. Selain itu, sistem juga akan mengirimkan notifikasi melalui Telegram Bot yang berisi informasi mengenai waktu kejadian, alamat IP penyerang, serta tindakan pemblokiran yang telah dilakukan. Dengan mekanisme tersebut, proses deteksi, analisis, mitigasi, monitoring, dan penyampaian informasi kepada administrator dapat berlangsung secara terintegrasi sehingga mampu meningkatkan efektivitas pengelolaan keamanan layanan *SSH* pada server Linux.

4.3.1 Mekanisme Log Analysis (Regex Matching)

Perangkat lunak Fail2Ban bekerja di latar belakang sistem (*system daemon*) dan memanfaatkan submodul INOTIFY yang disediakan oleh kernel Linux untuk memantau setiap adanya perubahan atau penambahan baris teks baru pada file berkas log autentikasi secara *real-time*. Proses identifikasi serangan dilakukan melalui mekanisme pencocokan string menggunakan Ekspresi Reguler (*Regular Expression* atau Regex). Pola regex yang dirancang dan diletakkan pada berkas konfigurasi filter sshd.conf adalah sebagai berikut:

```
^%(__prefix_line)sFailed \S+ for .* from (?: port \d+)?(?: ssh2)?\s*$
```

Sistem akan memindai berkas log baris demi baris. Ketika ditemukan sebuah baris teks kegagalan login yang memiliki kesesuaian pola dengan aturan ekspresi reguler di atas, komponen *filter* akan mengekstrak alamat IP penyerang yang berada

pada variabel token <HOST>. Alamat IP tersebut kemudian dikirimkan ke komponen *jail* untuk dihitung jumlah akumulasi frekuensi kegagalannya.

4.3.2 Perancangan Aturan Firewall dan Parameter Fail2Ban

Ketika sebuah alamat IP terdeteksi melakukan pelanggaran, Fail2Ban tidak langsung melakukan pemblokiran secara permanen, melainkan melakukan kalkulasi berdasarkan parameter-parameter pembatasan yang telah didefinisikan di dalam berkas konfigurasi jail.local. Parameter-parameter tersebut dirancang untuk memberikan keseimbangan antara keamanan server dan kenyamanan akses pengguna agar tidak terjadi salah blokir (*false positive*).

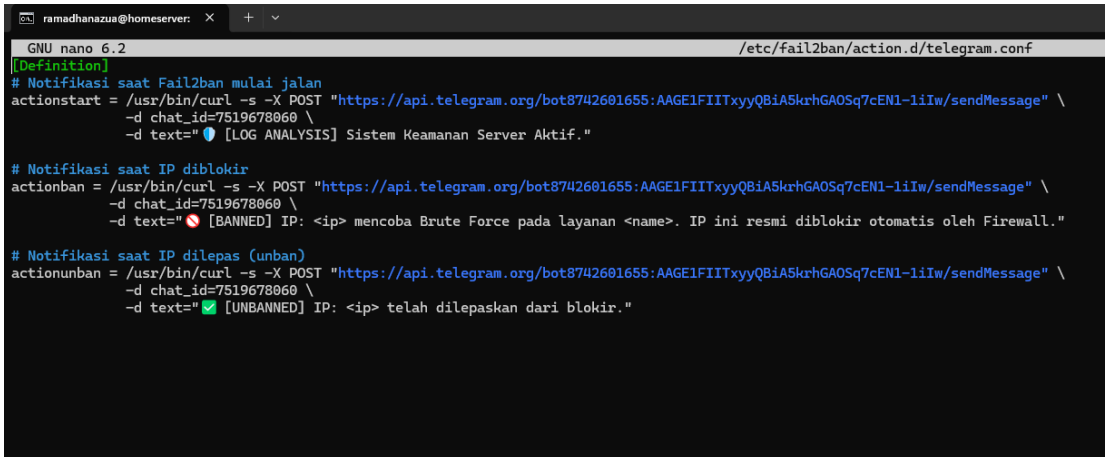
Tabel 4.1 Parameter Konfigurasi Jail Fail2Ban

Parameter	Nilai Konfigurasi	Deskripsi Fungsi
[sshd]	true	Mengaktifkan status proteksi dan pengawasan khusus untuk layanan SSH daemon.
port	22	Menentukan nomor port target yang diawasi aktivitasnya oleh sistem keamanan.
filter	sshd	Merujuk pada berkas filter sshd.conf yang berisi aturan ekspresi reguler.
logpath	/var/log/auth.log	Menentukan lokasi absolut dari berkas log autentikasi yang wajib dipindai.
maxretry	5	Batas toleransi maksimal frekuensi kesalahan login sebelum tindakan pemblokiran dijatuhkan.
findtime	5m (5 Menit)	Jendela waktu pengawasan untuk menghitung akumulasi total kegagalan login yang terjadi.
bantime	1h (1 Jam)	Durasi waktu isolasi atau penolakan paket data alamat IP penyerang di dalam firewall.
Banaction	iptables-multiport	Aksi taktis firewall untuk menyisipkan IP penyerang ke dalam tabel DROP Netfilter/iptables.

Logika operasional dari Tabel 4.1 di atas dirancang dengan skenario sebagai berikut: Jika ada sebuah alamat *IP Address* melakukan kesalahan autentikasi kata sandi sebanyak 3 kali (*maxretry*) dalam rentang jendela waktu kurang dari atau sama dengan

5 menit (*findtime*), maka *Rule Engine* Fail2Ban akan memutuskan bahwa aktivitas tersebut adalah serangan. Secara otomatis, Fail2Ban akan mengeksekusi perintah sistem untuk memerintahkan iptables *firewall* menjatuhkan aturan (*DROP*) pada setiap paket data yang berasal dari IP tersebut selama durasi 1 jam (*bantime*).

4.3.3 Integrasi Notifikasi Telegram Bot



Gambar 4.4 Integrasi Notifikasi Telegram Bot

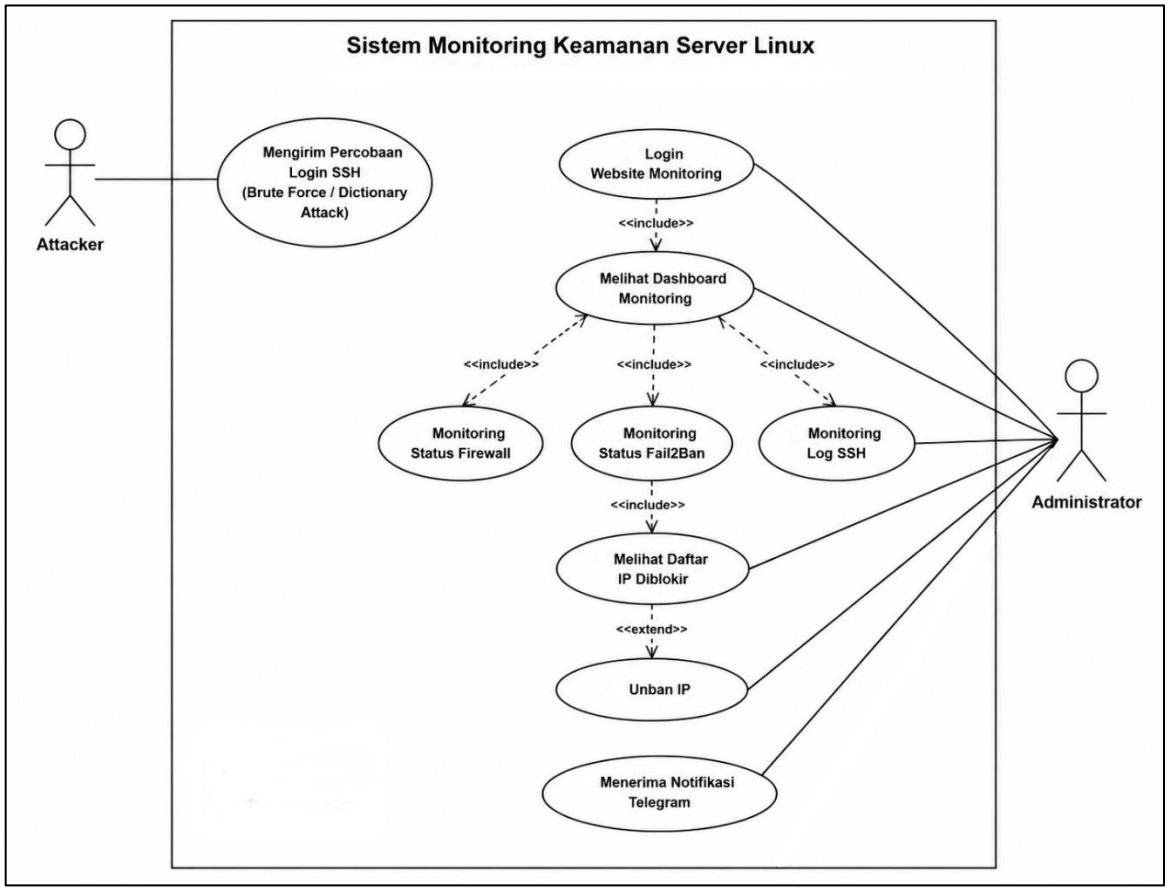
Sebagai upaya untuk meningkatkan visibilitas dan aspek kemudahan pengawasan bagi administrator jaringan yang tidak selalu berada di depan konsol server, arsitektur ini mengimplementasikan subsistem pemberitahuan otomatis. Mekanisme ini dirancang dengan memodifikasi parameter actionban pada Fail2Ban agar memicu pengeksekusian skrip Python eksternal saat proses pemblokiran IP berhasil dijalankan oleh *firewall*.

Skrip tersebut akan melakukan panggilan metode *POST* ke URL Telegram Bot API menggunakan token bot dan ID obrolan (*Chat ID*) milik administrator yang unik. Informasi di dalam struktur pesan dikonfigurasi secara dinamis, mencakup: stempel waktu pemblokiran, nama layanan yang diserang (SSH), alamat IP penyerang yang berhasil diisolasi, jumlah total percobaan login gagal yang dilakukan, serta durasi waktu pemblokiran.

4.4 Perancangan Sistem

Untuk memberikan visualisasi yang logis dan terstruktur mengenai bagaimana sistem penunjang keputusan keamanan ini bekerja, dirancang pemodelan sistem menggunakan diagram UML (*Unified Modeling Language*) dan diagram alir (*flowchart*). Pemodelan ini memetakan hubungan antara aktor eksternal dengan proses-proses internal sistem.

4.4.1 *Use Case* Diagram Sistem Keamanan



Gambar 4.5 Use Case Diagram

Dari diagram konteks diatas, berikut uraiannya:

1. Mengirim Percobaan Login SSH (*Brute Force* / *Dictionary Attack*)

Use case ini dilakukan oleh aktor Attacker dengan mengirimkan percobaan autentikasi SSH secara berulang menggunakan aplikasi Hydra. Aktivitas tersebut menghasilkan sejumlah kegagalan login yang dicatat pada berkas *auth.log* dan selanjutnya diproses oleh Fail2Ban sebagai dasar pendeteksian serangan.

2. Login Website Monitoring

Use case ini dilakukan oleh Administrator untuk mengakses aplikasi website monitoring menggunakan akun yang telah terdaftar. Proses login diperlukan sebagai mekanisme autentikasi agar hanya pengguna yang memiliki hak akses yang dapat mengelola dan memantau kondisi keamanan server.

3. Melihat Dashboard Monitoring

Setelah berhasil melakukan login, administrator dapat mengakses halaman dashboard yang menampilkan ringkasan informasi keamanan server secara *real-time*.

Dashboard menyajikan informasi mengenai aktivitas Fail2Ban, status firewall, log autentikasi SSH, serta daftar alamat IP yang telah diblokir.

4. Monitoring Status Firewall

Use case ini memungkinkan administrator memantau kondisi firewall yang digunakan untuk melakukan pemblokiran terhadap alamat IP yang terindikasi melakukan serangan. Informasi yang ditampilkan meliputi status layanan firewall dan hasil penerapan aturan pemblokiran.

5. Monitoring Status Fail2Ban

Administrator dapat melihat kondisi layanan Fail2Ban, termasuk status aktif layanan, jumlah percobaan login yang terdeteksi, serta proses pemblokiran yang telah dijalankan berdasarkan hasil analisis log autentikasi.

6. Monitoring Log SSH

Use case ini digunakan untuk menampilkan isi berkas *auth.log* yang berisi aktivitas autentikasi pada layanan SSH, seperti login berhasil, login gagal, alamat IP sumber, serta waktu terjadinya aktivitas. Informasi tersebut digunakan sebagai bahan pemantauan dan analisis keamanan server.

7. Melihat Daftar IP Diblokir

Administrator dapat melihat daftar alamat IP yang telah diblokir oleh Fail2Ban beserta informasi pendukung seperti waktu pemblokiran, durasi blokir (*bantime*), serta jumlah percobaan login yang menyebabkan pemblokiran dilakukan.

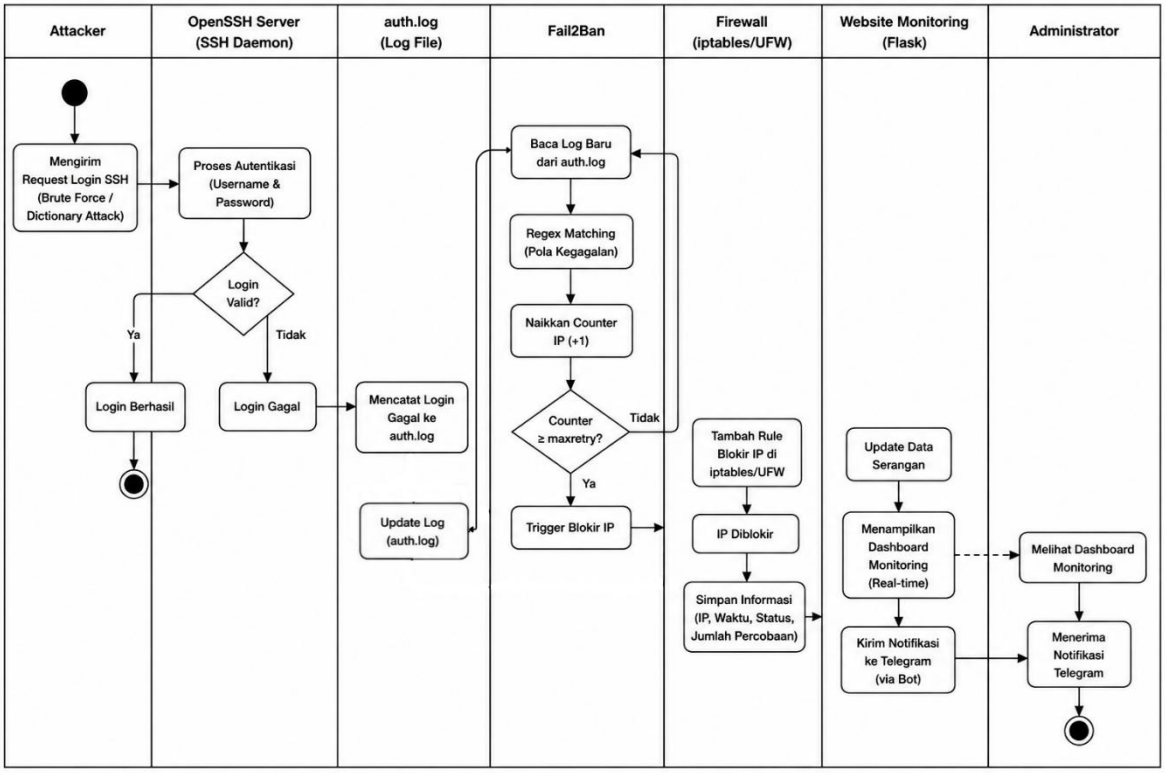
8. Unban IP

Use case ini digunakan apabila administrator ingin membuka kembali akses terhadap alamat IP tertentu yang sebelumnya telah diblokir oleh sistem. Fitur ini umumnya digunakan apabila terjadi *false positive* atau terdapat kebutuhan administratif tertentu.

9. Menerima Notifikasi Telegram

Sistem secara otomatis mengirimkan notifikasi kepada administrator melalui Telegram Bot ketika Fail2Ban berhasil mendeteksi dan memblokir alamat IP yang melakukan percobaan login secara berulang. Notifikasi berisi informasi seperti alamat IP penyerang, waktu kejadian, jumlah percobaan login, dan status pemblokiran sehingga administrator dapat segera mengetahui adanya insiden keamanan.

4.4.2 *Activity Diagram* Analisis Log dan Pemblokiran



Gambar 4.6 Activity Diagram Analisis Log dan Pemblokiran

Dari diagram konteks diatas, berikut uraiannya:

1. Attacker Mengirimkan Permintaan Login SSH

Proses diawali ketika attacker mengirimkan sejumlah permintaan autentikasi menuju layanan SSH pada server Linux. Serangan dilakukan menggunakan metode Brute Force atau Dictionary Attack dengan mencoba berbagai kombinasi username dan password secara berulang hingga memperoleh akses ke sistem.

2. OpenSSH Server Melakukan Proses Autentikasi

Seluruh permintaan login diterima oleh layanan OpenSSH (SSH Daemon). Server kemudian melakukan proses verifikasi terhadap kredensial yang diberikan dengan membandingkan username dan password yang diterima dengan data autentikasi yang tersimpan pada sistem.

3. Validasi Hasil Login

Apabila kredensial yang diberikan benar, proses autentikasi dinyatakan berhasil sehingga pengguna memperoleh akses ke server dan proses berakhir. Sebaliknya, apabila autentikasi gagal, OpenSSH mencatat informasi kegagalan tersebut ke dalam berkas log autentikasi (auth.log).

4. Pencatatan Aktivitas pada Berkas auth.log

Setiap kegagalan autentikasi direkam pada berkas auth.log yang memuat informasi penting, seperti alamat IP sumber, waktu kejadian, nama pengguna, serta jenis kegagalan login. Berkas log ini menjadi sumber utama yang digunakan dalam proses analisis keamanan sistem.

5. Fail2Ban Membaca dan Menganalisis Berkas Log

Setelah terdapat pembaruan pada auth.log, Fail2Ban secara otomatis membaca baris log terbaru kemudian melakukan proses Regex Matching untuk mencocokkan pola kegagalan autentikasi berdasarkan aturan (filter) yang telah dikonfigurasi.

6. Perhitungan Jumlah Percobaan Login Gagal

Apabila pola kegagalan berhasil dikenali, Fail2Ban akan meningkatkan nilai penghitung (counter) untuk alamat IP yang bersangkutan. Nilai ini digunakan untuk mengetahui jumlah percobaan login gagal yang dilakukan oleh satu alamat IP dalam periode waktu tertentu (findtime).

7. Pemeriksaan Nilai Maxretry

Sistem selanjutnya membandingkan jumlah percobaan login gagal dengan parameter maxretry. Jika jumlah percobaan belum mencapai batas yang telah ditentukan, proses kembali melakukan pemantauan terhadap pembaruan log berikutnya. Namun, apabila jumlah kegagalan telah mencapai atau melebihi nilai maxretry, Fail2Ban akan menganggap aktivitas tersebut sebagai indikasi serangan dan melanjutkan proses mitigasi.

8. Eksekusi Pemblokiran Alamat IP

Setelah serangan terdeteksi, Fail2Ban mengirimkan perintah kepada Firewall (iptables/UFW) untuk menambahkan aturan (rule) pemblokiran terhadap alamat IP penyerang. Dengan demikian, seluruh koneksi berikutnya yang berasal dari alamat IP tersebut akan ditolak selama periode bantime yang telah ditentukan.

9. Penyimpanan Informasi Serangan

Setelah alamat IP berhasil diblokir, sistem menyimpan informasi mengenai insiden keamanan, seperti alamat IP penyerang, waktu kejadian, status pemblokiran, serta jumlah percobaan login gagal. Data tersebut digunakan sebagai arsip sekaligus bahan analisis keamanan server.

10. Pembaruan Dashboard Website Monitoring

Informasi serangan yang telah tersimpan selanjutnya dikirimkan ke aplikasi Website Monitoring berbasis Flask. Dashboard melakukan pembaruan data

secara real-time sehingga administrator dapat memantau kondisi keamanan server, daftar alamat IP yang diblokir, serta riwayat aktivitas serangan melalui antarmuka web.

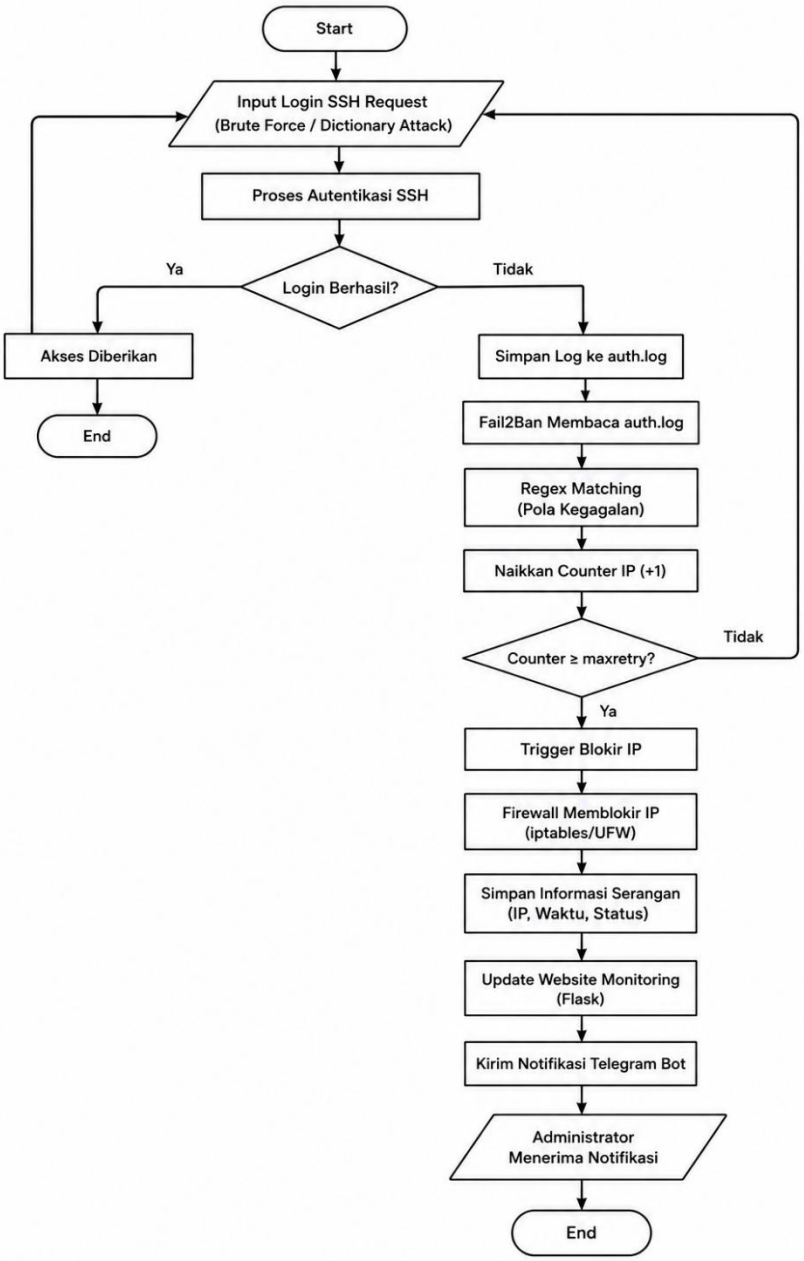
11.Pengiriman Notifikasi Telegram

Selain memperbarui dashboard monitoring, sistem juga mengirimkan notifikasi otomatis melalui Telegram Bot kepada administrator. Pesan yang dikirimkan berisi informasi mengenai alamat IP penyerang, waktu kejadian, jumlah percobaan login, serta status pemblokiran sehingga administrator dapat segera mengetahui adanya insiden keamanan.

12.Administrator Melakukan Monitoring Sistem

Administrator dapat mengakses Website Monitoring untuk melihat kondisi keamanan server secara real-time serta menerima notifikasi melalui aplikasi Telegram. Informasi tersebut membantu administrator dalam melakukan pemantauan, evaluasi, dan tindakan lanjutan apabila diperlukan. Setelah informasi diterima, proses mitigasi terhadap serangan pada sesi tersebut dinyatakan selesai.

4.4.3 Flowchart Proses Mitigasi Serangan



Gambar 4.7 FlowChart Proses Mitigasi Serangan

Penjelasan mengenai alur dari algoritma *flowchart* di atas adalah sebagai berikut:

1. Start

Tahap ini merupakan kondisi awal ketika sistem keamanan server Linux telah diaktifkan. Pada tahap ini layanan OpenSSH, Firewall (iptables/UFW), Fail2Ban, serta aplikasi website monitoring berbasis Flask telah berjalan dan berada dalam kondisi siap untuk menerima permintaan autentikasi SSH, melakukan

pemantauan terhadap berkas auth.log, serta mendukung proses deteksi dan mitigasi serangan secara otomatis.

2. Input Login SSH (Brute Force / Dictionary Attack)

Sistem menerima permintaan autentikasi menuju layanan SSH dari suatu alamat IP. Permintaan tersebut dapat berasal dari pengguna yang sah maupun attacker yang mencoba memperoleh akses tidak sah melalui metode brute force attack atau dictionary attack dengan melakukan percobaan login secara berulang.

3. Proses Autentikasi SSH

OpenSSH Server melakukan proses autentikasi dengan memverifikasi username dan password yang diterima terhadap data pengguna yang tersimpan pada sistem. Hasil verifikasi ini menentukan apakah permintaan koneksi dapat diterima atau ditolak.

4. Keputusan Login Valid

Pada tahap ini sistem menentukan hasil proses autentikasi. Apabila kredensial yang dimasukkan sesuai, pengguna diberikan hak akses ke server dan proses berakhir. Sebaliknya, apabila autentikasi gagal, sistem akan melanjutkan proses dengan mencatat informasi kegagalan login ke dalam berkas auth.log untuk keperluan analisis keamanan.

5. Mencatat Login Gagal ke auth.log

Setiap kegagalan autentikasi secara otomatis dicatat ke dalam berkas /var/log/auth.log. Informasi yang direkam meliputi waktu kejadian, alamat IP sumber, nama pengguna, serta status autentikasi. Berkas log ini menjadi sumber utama yang digunakan dalam proses deteksi aktivitas mencurigakan.

6. Update Log (auth.log)

Setelah entri baru ditambahkan, berkas auth.log diperbarui sehingga informasi terbaru mengenai aktivitas autentikasi dapat segera dipantau oleh Fail2Ban. Pembaruan ini memungkinkan proses analisis log dilakukan secara real-time tanpa memerlukan intervensi administrator.

7. Membaca Berkas auth.log

Fail2Ban secara otomatis membaca perubahan yang terjadi pada berkas auth.log. Setiap entri baru hasil autentikasi akan diproses untuk mengidentifikasi adanya pola aktivitas yang mengarah pada indikasi serangan terhadap layanan SSH.

8. Regex Matching (Pola Kegagalan)

Fail2Ban menerapkan metode regular expression (regex) untuk mencocokkan setiap entri log dengan pola kegagalan autentikasi yang telah didefinisikan pada filter. Tahap ini bertujuan untuk membedakan aktivitas login normal dengan aktivitas yang terindikasi sebagai serangan.

9. Naikkan Counter IP (+1)

Apabila pola kegagalan berhasil dikenali, Fail2Ban akan menambahkan nilai penghitung (counter) terhadap alamat IP yang melakukan percobaan login gagal. Nilai counter ini digunakan sebagai dasar dalam menentukan apakah suatu alamat IP telah melewati batas toleransi kegagalan autentikasi.

10. Keputusan Counter ⩾ maxretry

Sistem membandingkan jumlah kegagalan autentikasi dengan nilai parameter maxretry yang telah ditetapkan. Apabila jumlah kegagalan belum mencapai batas tersebut, sistem akan kembali melakukan pemantauan terhadap aktivitas login berikutnya. Namun, apabila jumlah kegagalan telah mencapai atau melebihi nilai maxretry, sistem akan melanjutkan proses mitigasi dengan melakukan pemblokiran alamat IP.

11. Trigger Blokir IP

Setelah batas maxretry terpenuhi, Fail2Ban menjalankan mekanisme ban dengan mengirimkan perintah kepada Firewall untuk memblokir alamat IP yang teridentifikasi melakukan percobaan login secara berulang.

12. Firewall Memblokir IP (iptables/UFW)

Firewall menambahkan aturan (rule) baru sehingga seluruh koneksi yang berasal dari alamat IP penyerang akan ditolak selama periode bantime yang telah dikonfigurasi. Dengan mekanisme ini, serangan lanjutan dari alamat IP yang sama dapat dicegah secara otomatis.

13. Simpan Informasi Serangan

Setelah proses pemblokiran berhasil dilakukan, sistem menyimpan informasi mengenai alamat IP penyerang, waktu kejadian, status pemblokiran, serta jumlah percobaan login ke dalam basis data monitoring. Informasi tersebut digunakan sebagai dokumentasi sekaligus bahan evaluasi terhadap aktivitas keamanan server.

14. Update Website Monitoring

Data hasil deteksi dan pemblokiran diperbarui secara otomatis pada aplikasi website monitoring berbasis Flask. Melalui antarmuka web tersebut, administrator dapat memantau kondisi keamanan server, daftar alamat IP yang diblokir, serta riwayat aktivitas serangan secara real-time.

15. Kirim Notifikasi Telegram

Selain memperbarui data pada website monitoring, sistem juga mengirimkan notifikasi melalui Telegram Bot kepada administrator. Notifikasi tersebut berisi informasi mengenai alamat IP penyerang, waktu kejadian, jenis aktivitas yang terdeteksi, serta status tindakan mitigasi yang telah dilakukan.

16. Administrator Menerima Notifikasi

Administrator menerima pesan peringatan melalui aplikasi Telegram sebagai bentuk *early warning* terhadap insiden keamanan yang terjadi. Informasi tersebut memungkinkan administrator untuk segera melakukan pemantauan lanjutan atau tindakan tambahan apabila diperlukan.

17. End

Tahap ini menandai berakhirnya satu siklus proses penanganan autentikasi. Setelah seluruh tahapan selesai dijalankan, sistem kembali ke kondisi siap (*standby*) untuk menerima dan memproses permintaan autentikasi SSH berikutnya secara berkelanjutan.

4.4.4 Perancangan Antarmuka Website



Gambar 4.8 Perancangan Website

Perancangan antarmuka website monitoring dirancang menggunakan konsep dashboard administrator yang bertujuan untuk memudahkan proses pemantauan kondisi keamanan server Linux secara *real-time*. Antarmuka terdiri atas *sidebar* navigasi di sisi kiri yang menyediakan menu Dashboard, Monitoring Log, Firewall, Fail2Ban, Telegram, dan Logout. Pada halaman utama ditampilkan informasi penggunaan sumber daya server meliputi CPU, RAM, Disk, serta status server sebagai indikator kesehatan sistem. Selain itu, dashboard juga menyajikan informasi detail server seperti *hostname*, sistem operasi, kernel, *uptime*, alamat IP, status Firewall, status layanan SSH, dan status Fail2Ban. Untuk mendukung proses analisis keamanan, halaman ini dilengkapi dengan grafik aktivitas login SSH serta tabel monitoring log yang menampilkan waktu kejadian, alamat IP, nama pengguna, status autentikasi, dan tindakan yang dilakukan sistem terhadap setiap aktivitas login. Perancangan antarmuka tersebut diharapkan mampu memberikan informasi keamanan server secara terpusat sehingga administrator dapat melakukan pemantauan dan pengambilan keputusan secara cepat ketika terjadi indikasi serangan.

4.5 Parameter Metrik Evaluasi Performa Server

Tahap evaluasi dilakukan untuk mengukur tingkat efektivitas sistem keamanan server Linux yang telah dirancang dan diimplementasikan. Pengujian dilakukan melalui simulasi serangan Brute Force Attack dan Dictionary Attack menggunakan aplikasi Hydra pada layanan OpenSSH. Selanjutnya, sistem diamati untuk mengetahui kemampuan Fail2Ban dalam mendeteksi aktivitas login yang mencurigakan berdasarkan analisis berkas auth.log, melakukan pemblokiran alamat IP secara otomatis melalui Firewall (iptables/UFW), menampilkan informasi pada website monitoring, serta mengirimkan notifikasi kepada administrator melalui Telegram Bot. Selain itu, evaluasi juga dilakukan terhadap penggunaan sumber daya server untuk memastikan bahwa mekanisme keamanan yang diterapkan tidak memberikan beban yang berlebihan terhadap sistem.

Pengukuran dilakukan secara kuantitatif dengan membandingkan kondisi server sebelum serangan, selama proses serangan berlangsung, dan setelah mekanisme mitigasi dijalankan. Hasil pengujian dianalisis menggunakan beberapa parameter yang meliputi keberhasilan deteksi serangan, waktu respons pemblokiran, tingkat keberhasilan notifikasi, pembaruan informasi pada website monitoring, serta konsumsi sumber daya server. Parameter-parameter tersebut digunakan sebagai dasar untuk menilai tingkat keandalan sistem keamanan yang dibangun.

Tabel 4.2 Matriks Pengujian dan Metrik Evaluasi Performa

No	Jenis Pengujian	Skenario Pengujian	Indikator Keberhasilan
1	Pengujian Deteksi Serangan	Melakukan simulasi *Brute Force* dan *Dictionary Attack* menggunakan Hydra terhadap layanan SSH.	Fail2Ban mampu mendeteksi seluruh aktivitas login gagal berdasarkan analisis *auth.log* sesuai konfigurasi *regex* dan *maxretry*.
2	Pengujian Pemblokiran Otomatis	Jumlah percobaan login gagal telah mencapai batas *maxretry*.	Firewall berhasil memblokir alamat IP penyerang secara otomatis sesuai nilai *bantime* yang telah ditentukan.
3	Pengujian Waktu Respons	Mengukur waktu sejak login gagal terakhir terdeteksi hingga alamat IP berhasil diblokir.	Waktu respons sistem berada dalam kategori cepat sehingga proses mitigasi berlangsung secara *real-time*.
4	Pengujian Website Monitoring	Mengamati perubahan data pada dashboard setelah terjadi serangan dan pemblokiran IP.	Dashboard menampilkan informasi log serangan, status Fail2Ban, status Firewall, dan daftar IP yang diblokir secara otomatis.
5	Pengujian Notifikasi Telegram	Terjadi proses pemblokiran alamat IP oleh Fail2Ban.	Telegram Bot berhasil mengirimkan informasi serangan beserta alamat IP, waktu kejadian, dan status pemblokiran kepada administrator.
6	Pengujian Konsumsi Sumber Daya	Membandingkan penggunaan CPU, RAM, dan penyimpanan sebelum dan sesudah sistem keamanan aktif.	Penggunaan sumber daya server tetap stabil sehingga mekanisme keamanan tidak mengganggu kinerja layanan server.

BAB V
HASIL & PEMBAHASAN

5.1 Lingkungan Pengujian

Pengujian dilakukan menggunakan lingkungan virtual yang terdiri atas dua mesin utama, yaitu mesin attacker dan mesin server. Rincian lingkungan pengujian disajikan pada Tabel 5.1.

Tabel 5.1 Lingkungan Pengujian

Komponen	Keterangan
Attacker	Kali Linux
Server	Linux Ubuntu
IP Server	192.168.186.134
Layanan yang diuji	SSH
Port SSH	22
Firewall	UFW / iptables
Intrusion Prevention	Fail2Ban
Audit monitoring	Auditd
Website monitoring	Flask (Python)
Log SSH	/var/log/auth.log
Data sensitif	File ISO pada /var/data/perusahaan/

Pada server disediakan beberapa file ISO sebagai data dummy untuk simulasi data sensitif, dengan struktur direktori sebagai berikut.

```
/var/data/perusahaan/
├── laporan_keuangan.iso
├── database_karyawan.iso
├── dokumen_proyek.iso
└── backup_database.iso
```

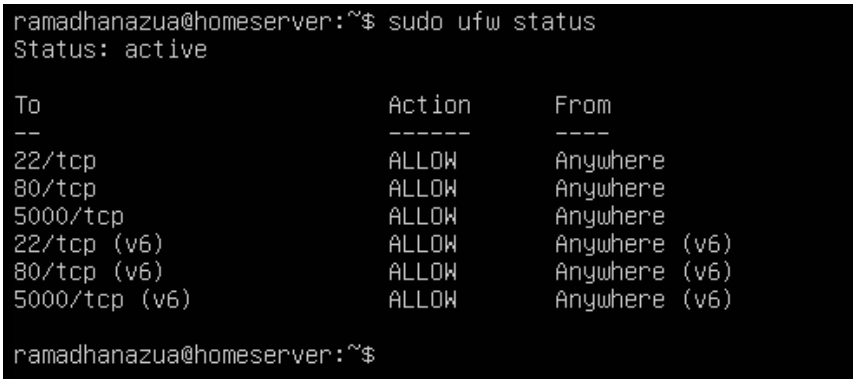
Data tersebut digunakan sebagai objek pengujian akses berkas dan bukan merupakan data perusahaan yang sebenarnya.

5.2 Hasil Implementasi Sistem

Tahap implementasi dilakukan untuk membangun sistem keamanan server Linux yang mampu mendeteksi, memitigasi, dan memantau aktivitas serangan pada layanan Secure Shell (SSH). Sistem yang dikembangkan mengintegrasikan OpenSSH sebagai layanan utama, Firewall UFW sebagai mekanisme pemblokiran alamat IP, Fail2Ban sebagai sistem

deteksi dan mitigasi otomatis berdasarkan analisis auth.log, Telegram Bot sebagai media notifikasi real-time, serta website monitoring berbasis Flask sebagai antarmuka pemantauan keamanan. Seluruh komponen bekerja secara terintegrasi sehingga setiap aktivitas login, proses pemblokiran IP, status Firewall, dan notifikasi dapat dikirimkan kepada administrator melalui Telegram untuk mendukung respons keamanan yang lebih cepat dan efektif.

5.2.1 Implementasi Firewall



Gambar 5.1 Implementasi Firewall

Implementasi Firewall pada penelitian ini menggunakan Uncomplicated Firewall (UFW) sebagai mekanisme pengendalian akses jaringan. Berdasarkan Gambar 5.1, Firewall berhasil diaktifkan dengan status active dan dikonfigurasi untuk mengizinkan akses hanya pada port 22/TCP (SSH), 80/TCP (HTTP), dan 5000/TCP (website monitoring) untuk koneksi IPv4 maupun IPv6. Konfigurasi tersebut membatasi akses hanya pada layanan yang diperlukan sehingga meningkatkan keamanan server. Selain itu, Firewall terintegrasi dengan Fail2Ban untuk melakukan pemblokiran otomatis terhadap alamat IP yang terdeteksi melakukan aktivitas serangan.

5.2.2 Implementasi Fail2Ban

Implementasi Fail2Ban diawali dengan memastikan bahwa layanan fail2ban.service telah terpasang dan berjalan dengan baik pada server. Verifikasi status layanan dilakukan menggunakan perintah *systemctl*. Berdasarkan Gambar 5.2, layanan *fail2ban.service* berhasil dijalankan dengan status *active* (*running*) sejak 15 September 2026 pukul 23:55 WIB. Layanan ini menggunakan Python 3 sebagai runtime dengan PID utama 984 (*fail2ban-server*), serta mengonsumsi memori sebesar 18.0M dan CPU sebesar 7.819s. Status *enabled*; vendor preset: *enabled* menunjukkan bahwa layanan Fail2Ban akan otomatis berjalan kembali setiap kali server dinyalakan ulang (*reboot*),

sehingga mekanisme pertahanan tetap aktif secara berkelanjutan tanpa memerlukan intervensi manual dari administrator.

```
ramadhanazua@homeserver:~$ sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
     Loaded: loaded (/lib/systemd/system/fail2ban.service; enabled; vendor preset: enabled)
     Active: active (running) since Tue 2026-09-15 23:55:35 WIB; 19min ago
       Docs: man:fail2ban(1)
   Main PID: 984 (fail2ban-server)
      Tasks: 7 (limit: 2172)
     Memory: 18.0M
        CPU: 7.819s
     CGroup: /system.slice/fail2ban.service
             └─984 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Sep 15 23:55:35 homeserver systemd[1]: Started Fail2Ban Service.
Sep 15 23:55:54 homeserver fail2ban-server[984]: Server ready
ramadhanazua@homeserver:~$
```

Gambar 5.2 Status Layanan Fail2Ban

Baris log terakhir, yaitu Server ready, menandakan bahwa seluruh konfigurasi jail telah berhasil dimuat dan Fail2Ban siap melakukan pemantauan terhadap berkas auth.log secara real-time.

Setelah memastikan layanan utama berjalan, langkah berikutnya adalah mengonfigurasi jail pada berkas /etc/fail2ban/jail.local. Berdasarkan Gambar 5.3, terdapat dua jail utama yang digunakan, yaitu sshd-dictionary dan sshd-bruteforce. Kedua jail tersebut dikonfigurasi dengan parameter *maxretry, findtime*, dan *bantime* yang berbeda sesuai dengan karakteristik serangan yang ditangani.

```
ramadhanazua@homeserver:
  GNU nano 6.2
[DEFAULT] ignoreip = 127.0.0.1/8 ::1

[sshd]
enabled = false

[sshd-dictionary]

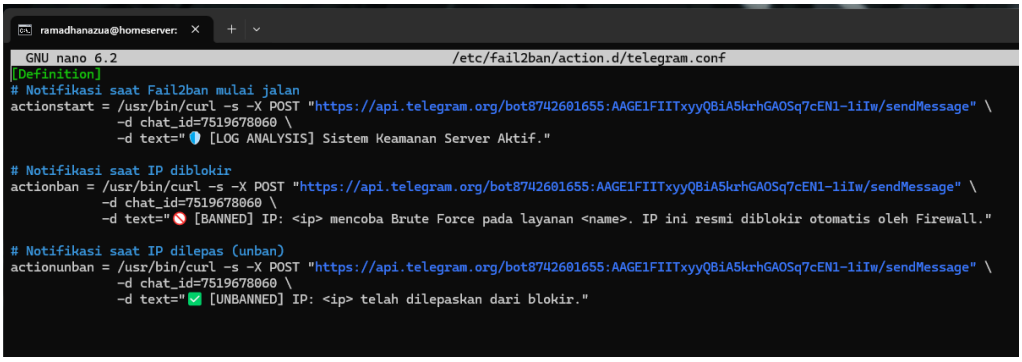
enabled = true
port = ssh
filter = sshd-dictionary
logpath = /var/log/auth.log
maxretry = 20
findtime = 120
bantime = 3600
action = telegram-dict

[sshd-bruteforce]

enabled = true
port = ssh
filter = sshd-bruteforce
logpath = /var/log/auth.log
maxretry = 10
findtime = 30
bantime = 3600
action = telegram-brute
```

Gambar 5.3 Implementasi Fail2Ban

5.2.3 Implementasi Telegram Bot



Gambar 5.4 Implementasi Telegram Bot

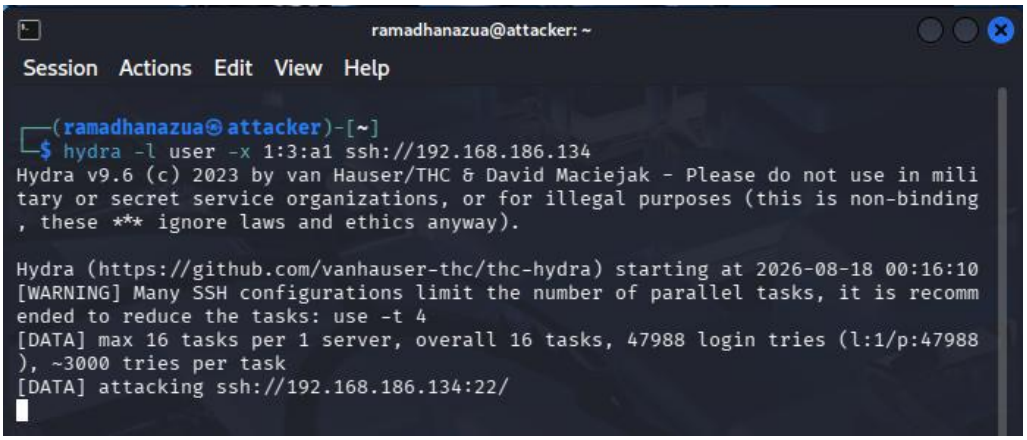
Mekanisme notifikasi Telegram Bot diaktifkan melalui parameter actionban pada konfigurasi Fail2Ban. Berdasarkan Gambar 5.4, setiap kali terjadi pemblokiran IP, sistem akan menjalankan skrip yang mengirimkan pesan ke Telegram Bot API menggunakan token dan Chat ID administrator.

5.3 Hasil Pengujian Sistem

Pengujian sistem pada sub-bab ini merupakan pengujian utama yang menjadi fokus penelitian sesuai dengan judul, yaitu Brute Force dan Dictionary Attack terhadap layanan *Secure Shell* (SSH). Kedua serangan diuji menggunakan tools Hydra pada Kali Linux dengan target server Ubuntu Server yang telah dikonfigurasi menggunakan Firewall (UFW) dan Fail2Ban. Setiap percobaan serangan dicatat pada berkas auth.log dan dianalisis oleh Fail2Ban menggunakan mekanisme regex matching untuk menentukan tindakan pemblokiran otomatis.

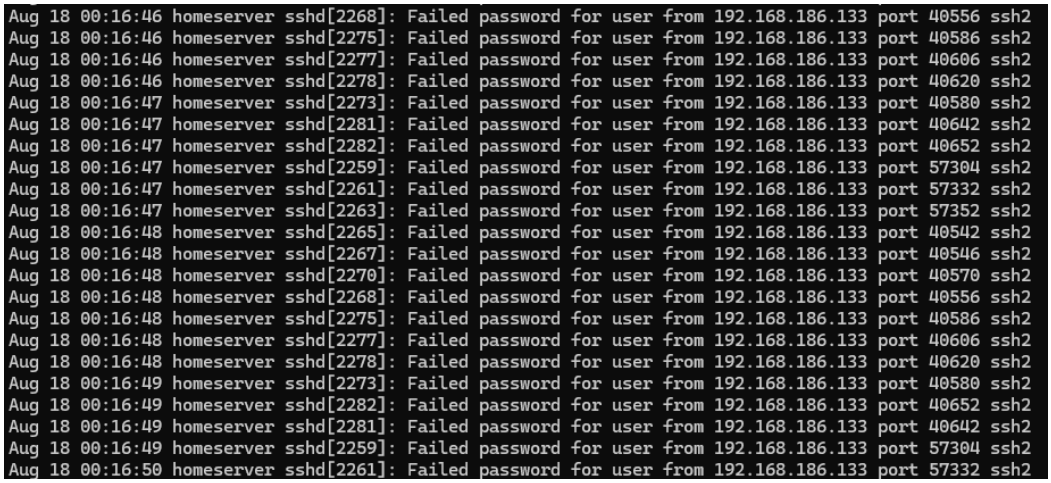
5.3.1 Pengujian Serangan Brute Force

Pengujian brute force bertujuan mengukur efektivitas integrasi Firewall dan Fail2Ban dalam mendeteksi serta memitigasi serangan autentikasi terhadap layanan SSH. Serangan ini bekerja dengan mencoba berbagai kombinasi username dan password secara berulang hingga ditemukan kredensial yang valid. Pengujian dilakukan dari Kali Linux menggunakan Hydra dengan perintah sebagai berikut.



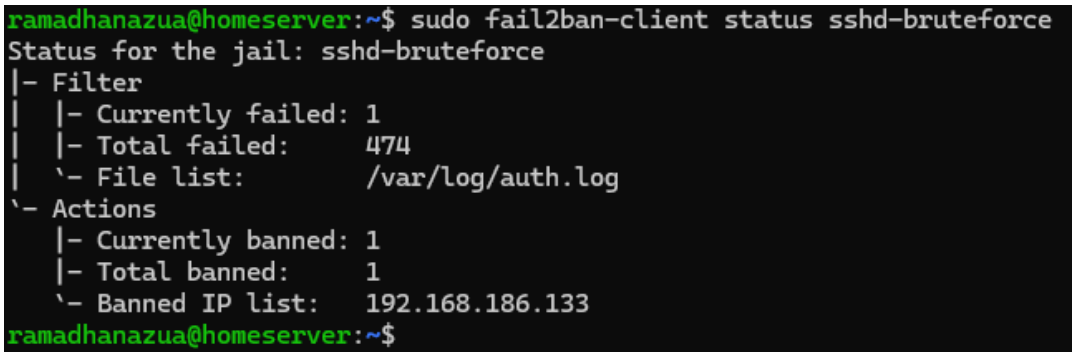
Gambar 5.5 Perintah Serangan Brute Force

Gambar 5.5 memperlihatkan Hydra yang dijalankan dari mesin penyerang (Kali Linux) dengan mode brute force. Hydra secara otomatis menghasilkan kombinasi password berdasarkan rentang yang ditentukan (1–3 karakter, huruf kecil dan angka), sehingga total terdapat 47.988 kemungkinan kombinasi yang akan dicoba. Setiap percobaan login yang gagal akan dicatat pada berkas */var/log/auth.log* di sisi server.



Gambar 5.6 Proses *Auth.log* Brute Force

Gambar 5.6 menunjukkan potongan isi berkas */var/log/auth.log* selama serangan berlangsung. Terlihat pola kegagalan autentikasi yang berulang dengan format yang konsisten, yaitu Failed password for user from 192.168.186.133 port [nomor port] ssh2. Pola inilah yang dikenali oleh Fail2Ban melalui mekanisme regex matching. Setiap baris log yang cocok dengan pola kegagalan akan meningkatkan nilai counter untuk alamat IP 192.168.186.133.



Gambar 5.7 Banned IP List sshd Bruteforce

Gambar 5.7 memperlihatkan status jail sshd-bruteforce setelah serangan berhasil dideteksi. Terlihat bahwa total kegagalan autentikasi mencapai 474 kali, dan alamat IP 192.168.186.133 berhasil masuk ke dalam daftar Banned IP list. Hal ini membuktikan bahwa Fail2Ban berhasil mendeteksi pola serangan dan menjalankan aksi pemblokiran melalui Firewall.

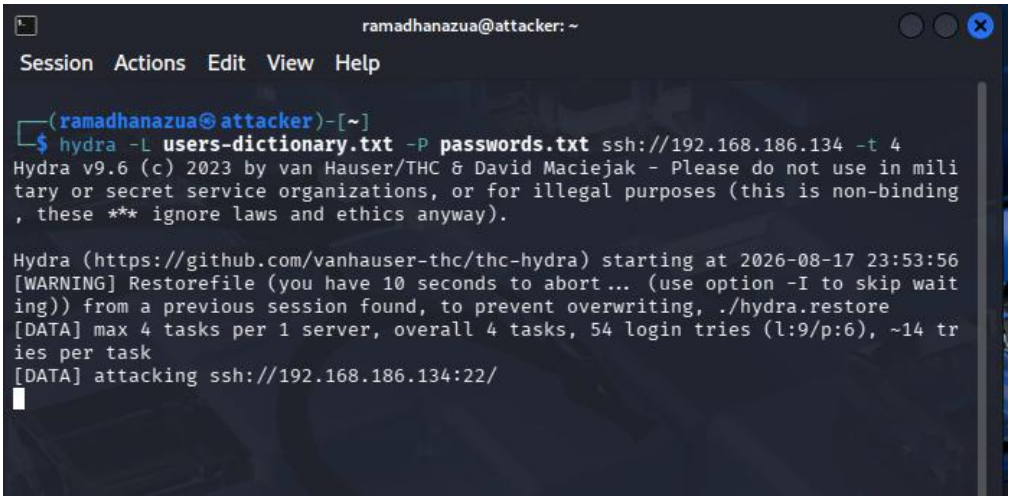
Setiap percobaan dilakukan dengan prosedur serang → blokir → unban → serang lagi, diulang sebanyak 5 kali untuk mengukur konsistensi sistem. Setiap percobaan dicatat waktu serangan, waktu blokir, dan waktu notifikasi diterima untuk menghitung response time dan delay notifikasi. Response time dihitung dari selisih antara waktu blokir dan waktu serangan, sedangkan delay notifikasi dihitung dari selisih antara waktu notif diterima dan waktu blokir. Hasil pengujian disajikan pada Tabel 5.2.

Tabel 5.2 Hasil Pengujian Brute Force Attack

Percobaan	Waktu Serangan	Waktu Blokir	Respon Time (dtk)	Waktu Notif Diterima	Delay Notif (dtk)	Status Ban
1	00:16:10	00:16:24	14	00:16:26	2	Berhasil
2	00:22:18	00:22:30	12	00:22:32	2	Berhasil
3	00:25:33	00:25:51	18	00:25:54	3	Berhasil
4	00:28:47	00:29:00	13	00:29:02	2	Berhasil
5	00:31:55	00:32:11	16	00:32:13	2	Berhasil

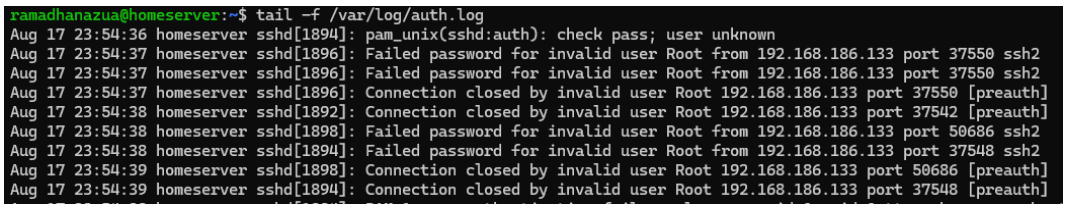
5.3.2 Pengujian Standard Dictionary Attack

Pengujian dictionary attack bertujuan mengukur efektivitas integrasi Firewall dan Fail2Ban dalam mendeteksi serta memitigasi serangan autentikasi yang memanfaatkan daftar kata (wordlist) , baik untuk username maupun password. Berbeda dengan brute force yang mencoba kombinasi karakter secara acak, dictionary attack menggunakan kumpulan kredensial umum yang sering digunakan sehingga prosesnya lebih terarah dan efisien. Pengujian dilakukan dari Kali Linux menggunakan Hydra dengan perintah sebagai berikut.



Gambar 5.8 Perintah Serangan Dictionary Attack

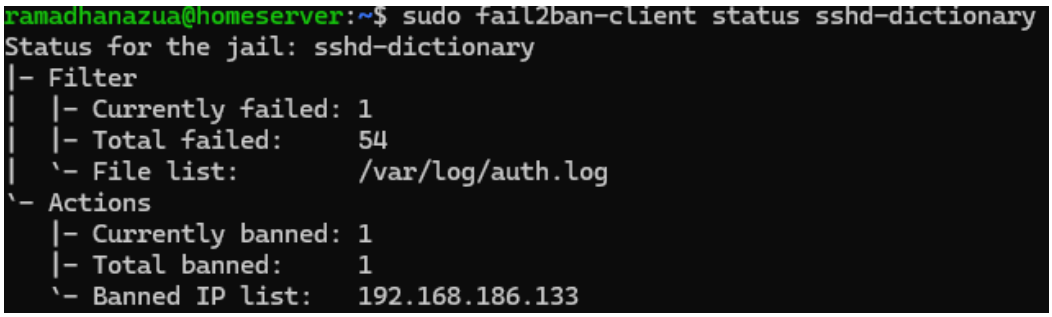
Gambar 5.8 memperlihatkan Hydra yang dijalankan dengan mode dictionary menggunakan dua berkas *wordlist*, yaitu users-dictionary.txt untuk daftar *username* dan passwords.txt untuk daftar *password*. Opsi -t 4 menunjukkan bahwa Hydra menjalankan 4 koneksi paralel secara bersamaan. Berbeda dengan brute force yang hanya menargetkan satu *username*, dictionary attack mencoba berbagai kombinasi *username* dan *password* dari *wordlist*, sehingga pola log yang dihasilkan pun berbeda.



Gambar 5.9 Proses *Auth.log* Dictionary

Gambar 5.9 menunjukkan potongan isi berkas /var/log/auth.log selama serangan dictionary berlangsung. Terlihat pola kegagalan autentikasi dengan format *Failed password for invalid user* [*username*] *from* 192.168.186.133 port [nomor port] ssh2. Ciri khas serangan ini adalah adanya kata invalid user yang menandakan bahwa

username yang dicoba tidak terdaftar di sistem, serta username yang berbeda-beda pada setiap baris log. Pola inilah yang dikenali oleh filter sshd-dictionary melalui mekanisme *regex matching*.



Gambar 5.10 Banned IP List sshd Dictionary

Gambar 5.10 memperlihatkan status jail sshd-dictionary setelah serangan berhasil dideteksi. Terlihat bahwa alamat IP 192.168.186.133 berhasil masuk ke dalam daftar Banned IP list. Hal ini membuktikan bahwa Fail2Ban berhasil mendeteksi pola serangan dictionary attack dan menjalankan aksi pemblokiran melalui Firewall.

Setiap percobaan dilakukan dengan prosedur serang → blokir → unban → serang lagi, diulang sebanyak 5 kali untuk mengukur konsistensi sistem. Setiap percobaan dicatat waktu serangan, waktu blokir, dan waktu notifikasi diterima untuk menghitung response time dan delay notifikasi. Response time dihitung dari selisih antara waktu blokir dan waktu serangan, sedangkan delay notifikasi dihitung dari selisih antara waktu notif diterima dan waktu blokir. Hasil pengujian disajikan pada Tabel 5.3.

Tabel 5.3 Hasil Pengujian Dictionary Attack

Percobaan	Waktu Serangan	Waktu Blokir	Respon Time (dtk)	Waktu Notif Diterima	Delay Notif (dtk)	Status Ban
1	23:53:56	23:54:08	12	23:54:10	2	Berhasil
2	23:59:45	23:59:57	12	23:59:59	2	Berhasil
3	00:02:52	00:03:03	11	00:03:06	3	Berhasil
4	00:05:10	00:05:25	15	00:05:27	2	Berhasil
5	00:08:33	00:08:47	14	00:08:49	2	Berhasil

5.3.3 Perbandingan Hasil Serangan Brute Force & Dictionary Attack

Berdasarkan hasil pengujian pada Tabel 5.2 dan Tabel 5.3, terdapat beberapa perbedaan karakteristik antara brute force attack dan dictionary attack yang berhasil dideteksi oleh sistem. Perbandingan kedua jenis serangan disajikan pada Tabel 5.4.

Tabel 5.4 Perbandingan Brute Force dan Dictionary Attack

Aspek	Brute Force Attack	Dictionary Attack
Username Target	Satu (user)	Multi user
Sumber Password	Digenerate (-x 1:3:a1)	*Wordlist* (-P passwords.txt)
Pola Log	*Failed password for user*	*Failed password for invalid user*
Filter	sshd-bruteforce	sshd-dictionary
Jumlah Percobaan	5	5
Rata-rata Response Time	14,6 detik	12,8 detik
Rata-rata Delay Notif	2,2 detik	2,2 detik
Success Rate	100%	100%
Notification Rate	100%	100%

Berdasarkan Tabel 5.4, sistem berhasil membedakan kedua jenis serangan melalui pola log yang dihasilkan. Brute force attack ditandai dengan *username* yang sama berulang (user), sedangkan dictionary attack ditandai dengan *username* yang berbeda-beda dan mengandung kata *invalid user*. Diferensiasi ini memungkinkan Fail2Ban untuk menerapkan konfigurasi jail yang berbeda sesuai karakteristik masing-masing serangan, sehingga proses deteksi dan pemblokiran menjadi lebih efektif.

Dari sisi response time, dictionary attack memiliki rata-rata 12,8 detik, lebih cepat dibandingkan brute force yang memiliki rata-rata 14,6 detik. Hal ini disebabkan oleh dua faktor utama. Pertama, jail sshd-dictionary dikonfigurasi dengan maxretry = 5 dan findtime = 20, yang berarti hanya diperlukan 5 kegagalan dalam 20 detik untuk memicu pemblokiran. Kedua, pola log *invalid user* lebih mudah dikenali oleh *filter regex* sehingga proses deteksi berlangsung lebih cepat. Sementara itu, brute force attack memerlukan 5 kegagalan sebelum memicu pemblokiran, dan seluruh percobaan menggunakan *username* yang sama sehingga membutuhkan waktu lebih lama untuk mencapai ambang batas.

5.3.4 Pengujian Login Sah (Pengukuran False Positive)

Selain pengujian serangan, dilakukan pula pengujian tambahan berupa login sah menggunakan kredensial yang benar dari user resmi yang terdaftar di sistem. Pengujian ini bertujuan untuk mengukur True Negative (TN) dan False Positive (FP) , yaitu sejauh mana sistem mampu membedakan aktivitas normal dari aktivitas serangan.

Pada penelitian ini, terdapat 3 user resmi yang terdaftar di sistem, yaitu ramadhanazua, user, dan admin. Pengujian dilakukan sebanyak 15 kali dengan variasi jumlah salah password sebelum berhasil login. Variasi ini bertujuan untuk menguji apakah sistem tetap mampu mengenali aktivitas normal meskipun user melakukan kesalahan input password beberapa kali. Hasil pengujian disajikan pada Tabel 5.5.

Tabel 5.5 Hasil Pengujian Login Sah

No	Waktu	Username	Jumlah Salah Password	Status Login	Status Jail	Klasifikasi
1	08:02:15	ramadhanazua	0	Berhasil	Tidak diblokir	TN
2	08:05:40	user	1	Berhasil	Tidak diblokir	TN
3	08:10:22	admin	0	Berhasil	Tidak diblokir	TN
4	08:14:07	ramadhanazua	2	Berhasil	Tidak diblokir	TN
5	08:19:51	user	0	Berhasil	Tidak diblokir	TN
6	08:23:33	admin	1	Berhasil	Tidak diblokir	TN
7	08:28:10	ramadhanazua	0	Berhasil	Tidak diblokir	TN
8	08:33:45	user	0	Berhasil	Tidak diblokir	TN
9	08:37:29	admin	2	Berhasil	Tidak diblokir	TN

Tabel 5.6 Lanjutan Tabel Hasil Pengujian Login Sah

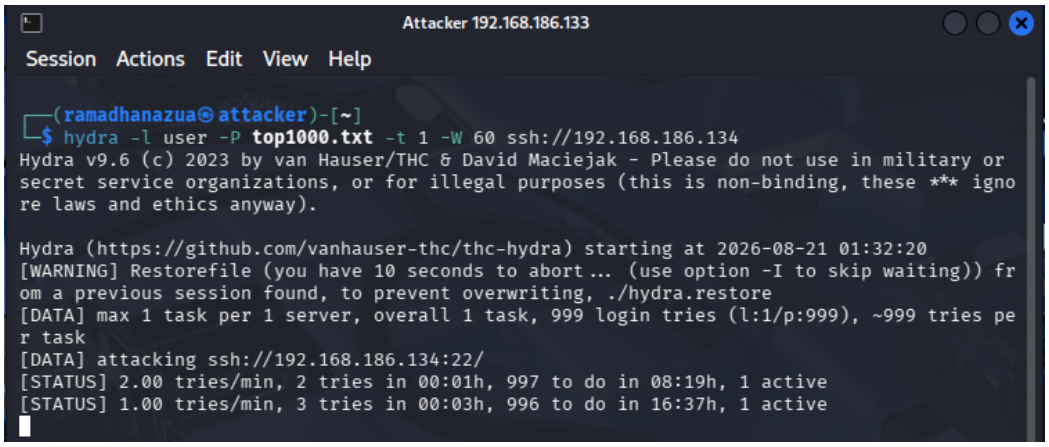
10	08:42:58	ramadhanazua	0	Berhasil	Tidak diblokir	TN
11	08:47:16	admin	5	Gagal 5x	Diblokir	FP
12	08:53:02	user	0	Berhasil	Tidak diblokir	TN
13	08:58:40	ramadhanazua	1	Berhasil	Tidak diblokir	TN
14	09:04:12	admin	0	Berhasil	Tidak diblokir	TN
15	09:09:55	user	3	Berhasil	Tidak diblokir	TN

Berdasarkan Tabel 5.5, dari 15 percobaan login sah yang dilakukan menggunakan 3 user resmi (ramadhanazua, user, admin), 14 percobaan berhasil dikenali sebagai aktivitas normal (TN), sedangkan 1 percobaan (baris ke-11) diblokir oleh sistem karena user admin salah memasukkan password sebanyak 5 kali berturut-turut sehingga mencapai ambang *maxretry* = 5 pada jail sshd-bruteforce. Kondisi ini diklasifikasikan sebagai *False Positive* (FP) , yaitu aktivitas normal yang salah terdeteksi sebagai serangan.

Meskipun demikian, jumlah FP yang hanya 1 dari 15 percobaan menunjukkan bahwa sistem memiliki tingkat presisi yang cukup baik dalam membedakan aktivitas normal dan serangan. Sebagian besar user yang salah memasukkan password 1–3 kali masih dapat dikenali sebagai aktivitas normal karena jumlah kegagalannya belum mencapai ambang *maxretry*. *False positive* baru muncul ketika user salah memasukkan *password* hingga 5 kali, yang memang secara statistik menyerupai pola serangan brute force.

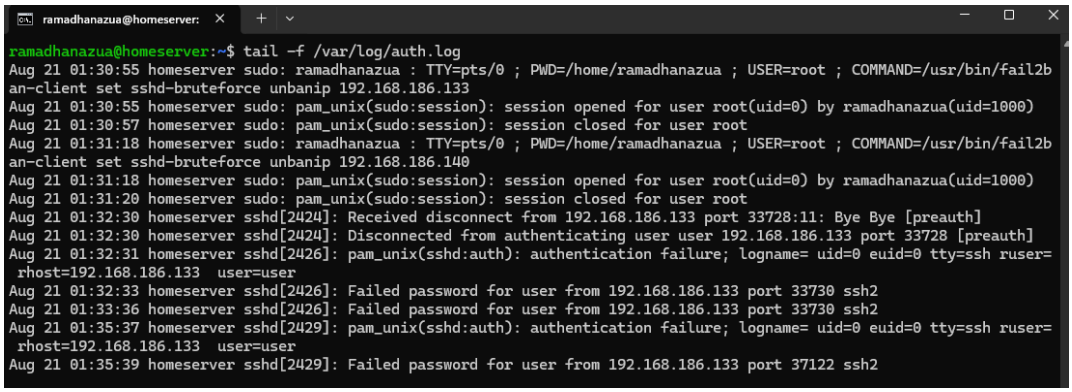
5.3.5 Pengujian *Low-and-Slow* Brute Force (Pengukuran *False Negative*)

Low-and-slow brute force dilakukan dengan memberi jeda waktu antarpercobaan agar jumlah kegagalan login yang tercatat dalam satu rentang waktu (*findtime*) tetap berada di bawah ambang batas *maxretry* pada Fail2Ban, sehingga percobaan sulit dibedakan dari aktivitas login normal yang sesekali gagal. Pengujian dilakukan dengan mengatur jeda waktu antarpercobaan pada Hydra menggunakan opsi -*W* (wait time).



Gambar 5.11 Perintah Serangan *Low-and-slow* Brute Force

Gambar 5.11 memperlihatkan Hydra yang menjalankan percobaan login dengan jeda satu percobaan setiap menit. Rekaman *auth.log* pada Gambar 5.12 menunjukkan pola kegagalan login yang sama namun tersebar pada rentang waktu yang panjang (pukul 01:30–01:35), berbeda dengan pola padat pada dictionary attack maupun brute force sebelumnya.



Gambar 5.12 Proses *Auth.log* Serangan *Low-and-Slow* Brute Force

Untuk mengukur False Negative (FN) , dilakukan pengujian tambahan berupa serangan low-and-slow brute force, yaitu serangan yang dilakukan dengan jeda waktu panjang antar percobaan agar jumlah kegagalan dalam rentang findtime tidak pernah mencapai maxretry. Pengujian dilakukan sebanyak 5 kali dengan jeda ±60 detik antar percobaan. Hasil pengujian disajikan pada Tabel 5.6.

Tabel 5.7 Hasil Pengujian *Low-and-Slow* Brute Force

No	Waktu Percobaan	Username	Jeda Antar Percobaan	Currently Failed	Status Diblokir	Klasifikasi
1	01:30:55	user	-	0	Tidak	FN
2	01:32:33	user	~2 menit	1	Tidak	FN
3	01:35:37	user	~3 menit	1	Tidak	FN

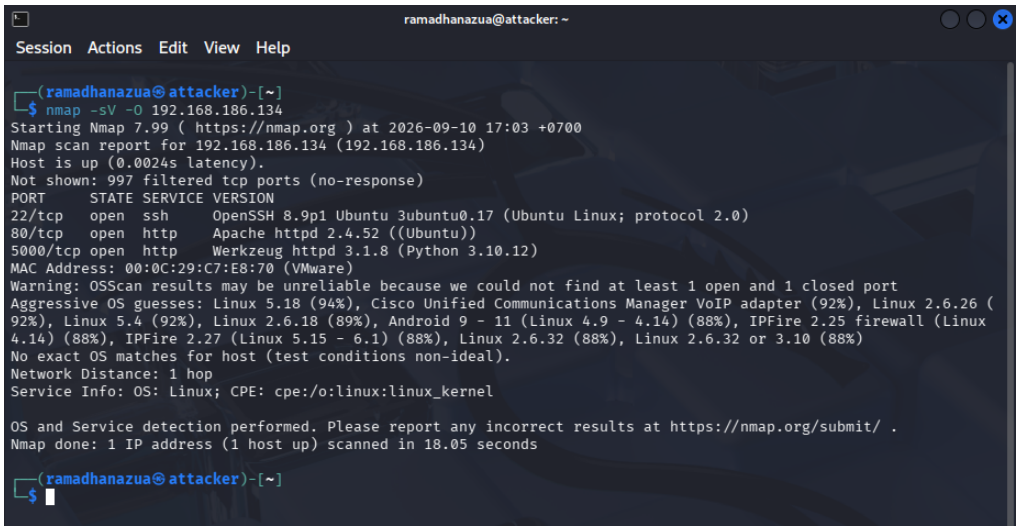
Berdasarkan Tabel 5.7, seluruh 5 percobaan low-and-slow brute force tidak terdeteksi oleh Fail2Ban. Hal ini disebabkan oleh jeda waktu yang panjang antar percobaan sehingga jumlah kegagalan dalam rentang findtime = 60 detik tidak pernah mencapai ambang maxretry = 5. Kondisi ini diklasifikasikan sebagai *False Negative* (FN) , yaitu serangan yang tidak terdeteksi oleh sistem. Hasil ini menunjukkan bahwa sistem masih memiliki keterbatasan dalam menghadapi serangan bertipe evasion yang dirancang untuk menghindari deteksi berbasis *threshold*.

5.4 Hasil Pengujian Serangan Lainnya

Selain dictionary attack dan brute force attack yang menjadi fokus utama penelitian ini pada layanan SSH, dilakukan pula sejumlah pengujian tambahan terhadap jenis serangan lain guna melihat sejauh mana cakupan dan keterbatasan sistem keamanan yang dibangun. Hasil dari masing-masing pengujian tersebut diuraikan secara ringkas pada sub-bab berikut.

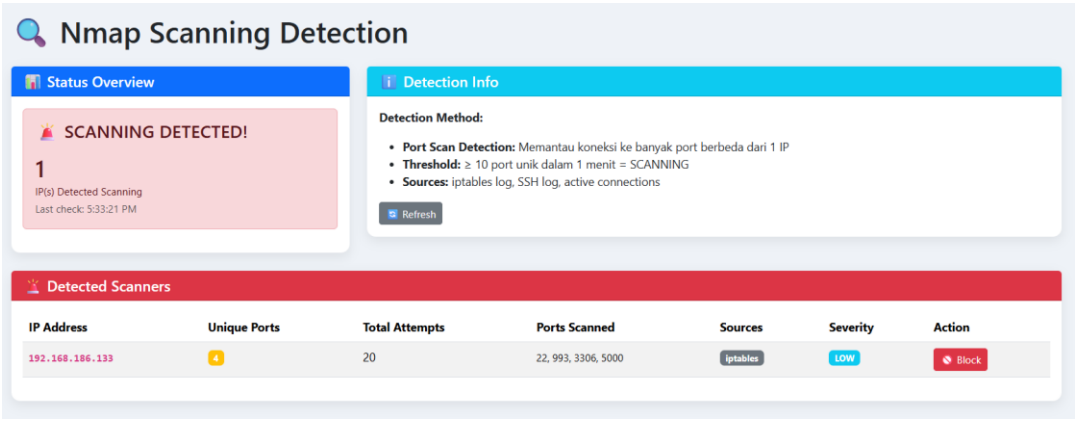
5.4.1 Pengujian Reconnaissance & Service Enumeration

Tahap pertama dilakukan dengan melakukan identifikasi terhadap server menggunakan mesin Kali Linux. Pengujian bertujuan untuk mengetahui layanan yang tersedia pada server target.



Gambar 5.13 Hasil Reconnaissance Server

Gambar 5.13 memperlihatkan hasil pemindaian Nmap terhadap server target. Teridentifikasi tiga layanan terbuka, yaitu SSH pada port 22, HTTP pada port 80, dan Werkzeug pada port 5000. Informasi ini menjadi dasar bagi penyerang untuk menentukan target serangan lanjutan.



Gambar 5.14 Hasil Deteksi Scanning pada Website Monitoring

Gambar 5.14 memperlihatkan bahwa aktivitas pemindaian tersebut turut terdeteksi oleh modul keamanan pada website monitoring. Modul ini memantau jumlah port unik yang diakses oleh satu alamat IP dalam rentang waktu tertentu; apabila sebuah IP mengakses sepuluh port unik atau lebih dalam satu menit, aktivitas tersebut diklasifikasikan sebagai scanning. Sistem berhasil mengenali IP 192.168.186.133 sebagai sumber scanning dengan tingkat keparahan LOW.

Tabel 5.8 Hasil Pengujian Reconnaissance & Service Enumeration

No	Command/Tools	Jumlah Port Dipindai	Status Deteksi Website	Severity
1	nmap -sV -O	4	SCANNING DETECTED	Low
2	nmap -p-	96	SCANNING DETECTED	Medium
3	nmap -A	106	SCANNING DETECTED	Medium

5.4.2 Pengujian SQL Injection

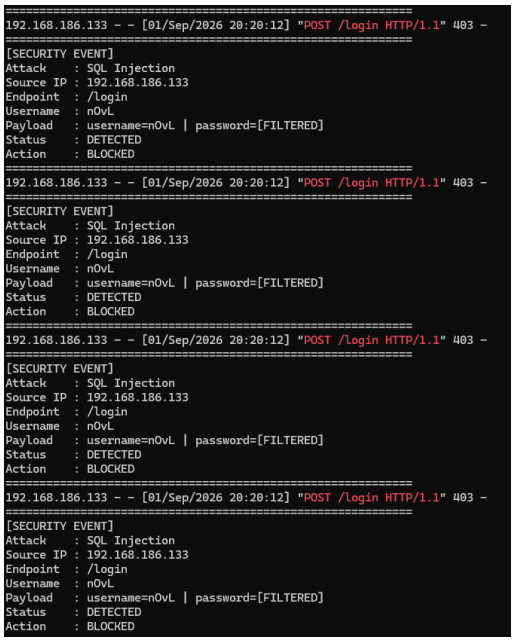
SQL Injection merupakan teknik serangan yang menyisipkan perintah SQL berbahaya ke dalam input formulir web (misalnya kolom username/password) dengan tujuan memanipulasi query database di belakang layar, sehingga penyerang dapat melewati autentikasi atau mengambil data yang seharusnya tidak dapat diakses. Serangan ini menyasar aplikasi *website monitoring* berbasis Flask, berbeda dari sembilan skenario lain yang menyasar layanan SSH.



Gambar 5.15 Proses Penyerangan sqlmap

Gambar 5.15 memperlihatkan proses sqlmap yang secara otomatis mendeteksi formulir login pada halaman dan mencoba menyisipkan payload SQL pada parameter username. Parameter --forms membuat sqlmap otomatis mendeteksi formulir, --batch

menjalankan proses tanpa konfirmasi manual, dan --dump mencoba mengekstraksi isi tabel database apabila celah ditemukan.



Gambar 5.16 Deteksi Serangan dari Server

Gambar 5.16 menunjukkan bahwa modul keamanan pada aplikasi Flask berhasil mengenali payload SQL Injection pada parameter *username* dan langsung memberikan respons HTTP 403 (Forbidden) disertai status *DETECTED* dan *BLOCKED*, yang berarti setiap permintaan berbahaya ditolak sebelum sempat memanipulasi query database. Hasil yang sama juga direkam pada sisi website monitoring, sebagaimana ditunjukkan pada Gambar 5.17.

No	Waktu	IP Address	Username	Jenis Serangan	Endpoint	Payload	File	Process	Syscall	Access	Status	Action	Risk
1	01-09-2026 20:20:12	192.168.186.133	nOvL	SQL Injection	/login	username=nOvL | password=[FILTERED]	-	-	-	DENIED	DETECTED	BLOCKED	CRITICAL
2	01-09-2026 20:20:12	192.168.186.133	nOvL	SQL Injection	/login	username=nOvL | password=[FILTERED]	-	-	-	DENIED	DETECTED	BLOCKED	CRITICAL
3	01-09-2026 20:20:12	192.168.186.133	nOvL	SQL Injection	/login	username=nOvL | password=[FILTERED]	-	-	-	DENIED	DETECTED	BLOCKED	CRITICAL
4	01-09-2026 20:20:12	192.168.186.133	nOvL	SQL Injection	/login	username=nOvL | password=[FILTERED]	-	-	-	DENIED	DETECTED	BLOCKED	CRITICAL
5	01-09-2026 20:20:12	192.168.186.133	nOvL	SQL Injection	/login	username=nOvL | password=[FILTERED]	-	-	-	DENIED	DETECTED	BLOCKED	CRITICAL
6	01-09-2026 20:20:12	192.168.186.133	nOvL	SQL Injection	/login	username=nOvL | password=[FILTERED]	-	-	-	DENIED	DETECTED	BLOCKED	CRITICAL
7	01-09-2026 20:20:12	192.168.186.133	nOvL	SQL Injection	/login	username=nOvL | password=[FILTERED]	-	-	-	DENIED	DETECTED	BLOCKED	CRITICAL

Gambar 5.17 Deteksi Serangan dari Website

Tabel pada Gambar 5.17 mencatat setiap percobaan SQL Injection beserta payload, status deteksi, aksi yang diambil (BLOCKED), dan tingkat risiko

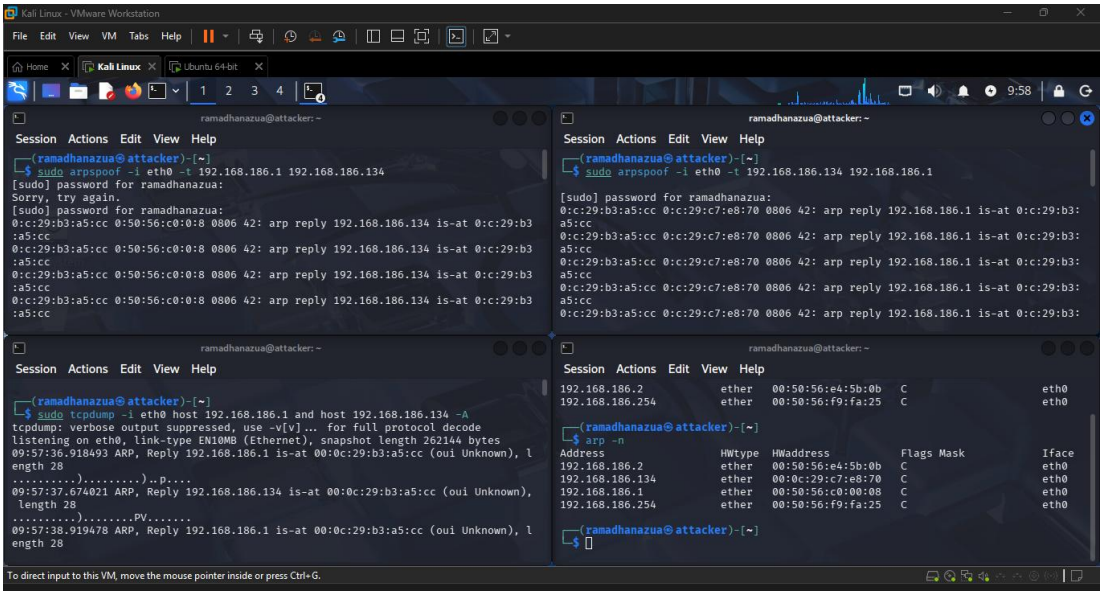
(CRITICAL). Dengan demikian, dapat disimpulkan bahwa mekanisme pertahanan pada aplikasi web berhasil mendeteksi dan memblokir seluruh percobaan SQL Injection secara konsisten.

Tabel 5.9 Hasil Pengujian SQL Injection

No	Waktu Percobaan	Endpoint	Payload	Status Deteksi	Aksi
1	20:20:12	/login	username=nOvL | password=[FILTERED]	DETECTED	BLOCKED (403)
2	20:20:12	/login	username=nOvL | password=[FILTERED]	DETECTED	BLOCKED (403)
3	20:20:12	/login	username=nOvL | password=[FILTERED]	DETECTED	BLOCKED (403)

5.4.3 Pengujian Man-in-the-Middle (MITM) / ARP Spoofing

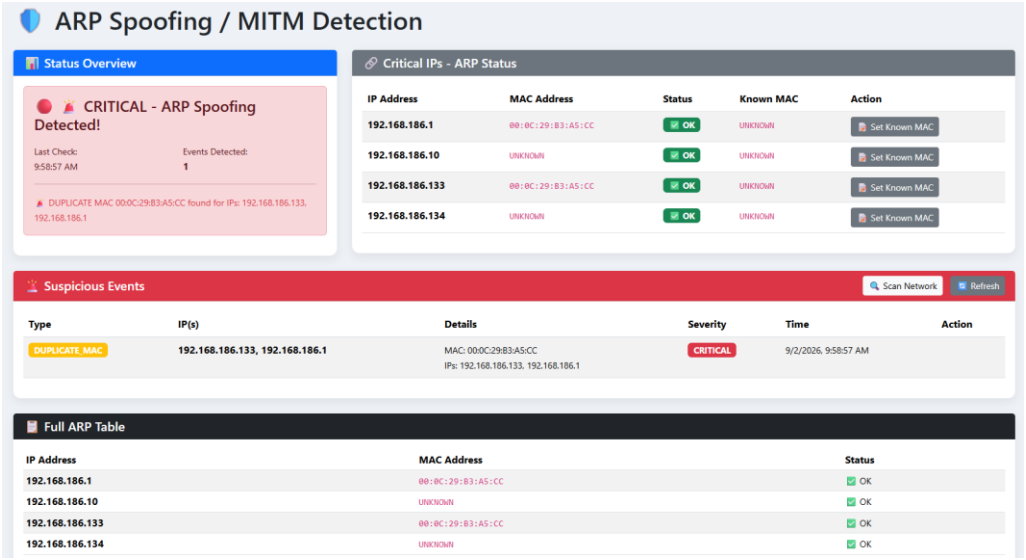
Man-in-the-Middle (MITM) adalah teknik serangan yang menempatkan penyerang di antara komunikasi dua pihak (misalnya administrator dan server) melalui teknik *ARP spoofing*, yaitu meracuni tabel ARP korban agar seluruh traffic jaringannya melewati mesin penyerang terlebih dahulu.



Gambar 5.18 Proses Serangan MITM

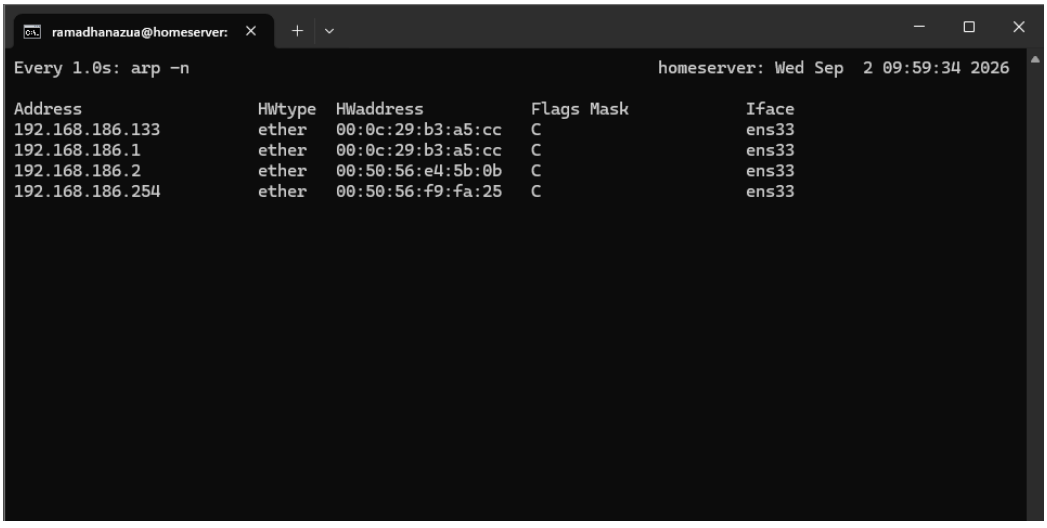
Gambar 5.18 memperlihatkan proses *arpspoof* yang membanjiri jaringan dengan paket ARP palsu ke dua arah, serta hasil tangkapan *tcpdump* yang menunjukkan lalu

lintas ARP tersebut. Aktivitas ini kemudian dianalisis oleh modul keamanan pada website monitoring untuk mendeteksi indikasi ARP spoofing/MITM.



Gambar 5.19 Hasil website deteksi serangan MITM

Gambar 5.19 menunjukkan hasil pertahanan berupa peringatan *CRITICAL - ARP Spoofing Detected* pada website, di mana sistem mengenali adanya MAC address duplikat (*duplicate MAC*) yang mengklaim dua alamat IP berbeda (192.168.186.1 dan 192.168.186.133), sehingga indikasi ARP spoofing berhasil diketahui secara dini. Sebagai bukti tambahan bahwa peracunan ARP benar-benar terjadi pada sisi server, Gambar 5.20 menampilkan tabel ARP pada server yang mencatat perubahan MAC address tersebut.



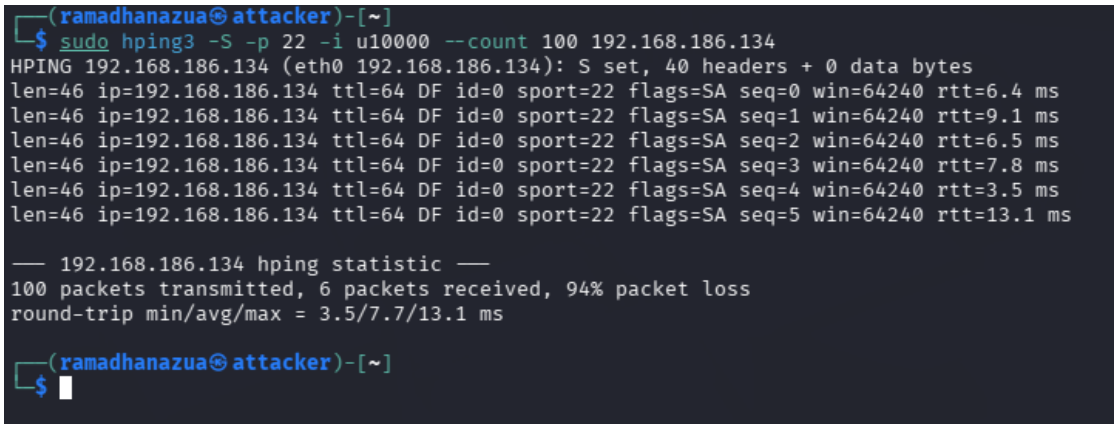
Gambar 5.20 Perubahan MAC pada server

Tabel 5.10 Hasil Pengujian Man-in-the-Middle (MITM) / ARP Spoofing

No	Waktu	IP Korban	MAC Sebelum Serangan	MAC Terdeteksi (Duplikat)	Status Deteksi
1	09:57:36	192.168.186.1	00:50:56:c0:00:08	00:0c:29:b3:a5:cc	CRITICAL
2	09:58:57	192.168.186.134	00:0c:29:c7:e8:70	00:0c:29:b3:a5:cc	CRITICAL

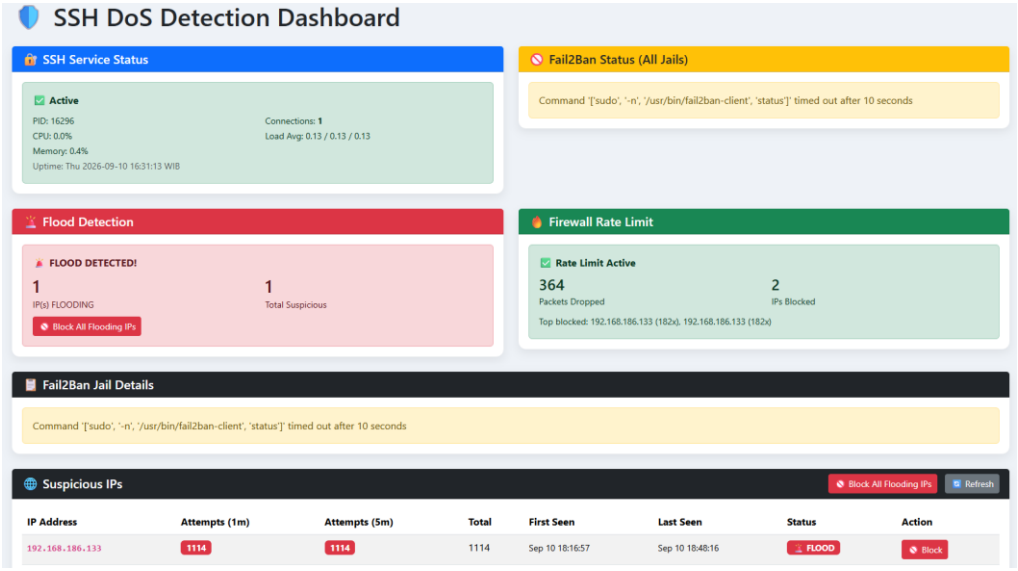
5.4.4 Pengujian Denial of Service (DoS)

Denial of Service (DoS) adalah serangan yang bertujuan membuat layanan tidak dapat diakses oleh pengguna sah, dengan cara membanjiri server menggunakan *volume traffic* atau permintaan koneksi yang sangat besar hingga sumber daya server habis. Berbeda dengan skenario lain yang menyasar kerahasiaan kredensial, DoS menyasar aspek ketersediaan (*availability*) layanan.



Gambar 5.21 Proses Serangan DoS

Gambar 5.21 memperlihatkan hping3 yang mengirimkan paket SYN secara terus-menerus ke port 22, dengan tingkat kegagalan paket yang tinggi (94% packet loss) sebagai indikasi bahwa server mulai kewalahan menangani volume traffic tersebut.



Gambar 5.22 Hasil Website Serangan DoS

Gambar 5.22 menunjukkan hasil pertahanan yang diambil oleh sistem: modul *Flood Detection* mengenali IP 192.168.186.133 sebagai sumber flooding, sementara modul *Firewall Rate Limit* mencatat ratusan paket yang berhasil dijatuhkan (packet dropped) dan dua IP yang diblokir. Dengan demikian, meskipun jumlah paket serangan sangat besar, mekanisme pembatasan laju pada firewall tetap dapat menjaga ketersediaan layanan SSH.

Tabel 5.11 Hasil Pengujian Denial of Service (DoS)

Percobaan	Command	Packet Trasmitted	Packet Loss	Packet Recieved	Status Deteksi	Aksi Firewall
1	sudo hping3 -S -p 22 -i u10000 --count 100 192.168.186.134	100	94%	6	*FLOOD DETECTED*	94 paket dijatuhkan, IP diblokir
2	sudo hping3 -S -p 22 -i u500000 --count 294 192.168.186.134	294	95%	15	*FLOOD DETECTED*	279 paket dijatuhkan, IP diblokir
3	sudo hping3 -S -p 22 -i u10000 --count 528 192.168.186.134	528	99%	6	*FLOOD DETECTED*	522 paket dijatuhkan, IP diblokir

Berdasarkan pengujian *Denial of Service* (DoS) menggunakan metode *SYN Flood* pada Tabel 5.11, analisis korelasi antara parameter serangan (hping3), jumlah paket dikirim (*transmitted*), paket diterima (*received*), dan nilai persentase *packet loss* dijelaskan sebagai berikut:

1. Percobaan 1 (Serangan Cepat):
    a. Parameter & Paket Dikirim: Pengujian menggunakan perintah hping3 dengan interval antarpaket -i u10000 (10 milidetik) sebanyak 100 paket.
    b. Paket Diterima & *Packet Loss*: Server hanya merespons 6 paket awal sebelum sistem mengidentifikasi ancaman dan mengubah status menjadi *FLOOD DETECTED*. Setelah *firewall* memasukkan IP penyerang ke daftar pemblokiran (*blocklist*), 94 paket sisanya langsung dijatuhkan (*DROP*). Hal ini menghasilkan tingkat *packet loss* sebesar 94% berdasarkan kalkulasi:

$$\text{Packet Loss} = \left(1 - \frac{6}{100}\right) \times 100\% = 94\%$$

2. Percobaan 2 (Serangan Diperlambat):
    a. Parameter & Paket Dikirim: Pengujian dilakukan dengan memperlambat interval pengiriman menjadi -i u500000 (500 milidetik / 0,5 detik) sebanyak 294 paket.
    b. Paket Diterima & *Packet Loss*: Karena interval pengiriman lebih lambat, server memiliki jeda waktu untuk memproses dan merespons lebih banyak paket awal sebelum *threshold* pemblokiran dipicu secara total, sehingga paket yang diterima meningkat menjadi 15 paket. Namun, begitu pemblokiran aktif, seluruh sisa paket (279 paket) dijatuhkan oleh *firewall*, menghasilkan nilai *packet loss* sebesar 95%:

$$\text{Packet Loss} = \left(1 - \frac{15}{294}\right) \times 100\% = 94.89\% \approx 95\%$$

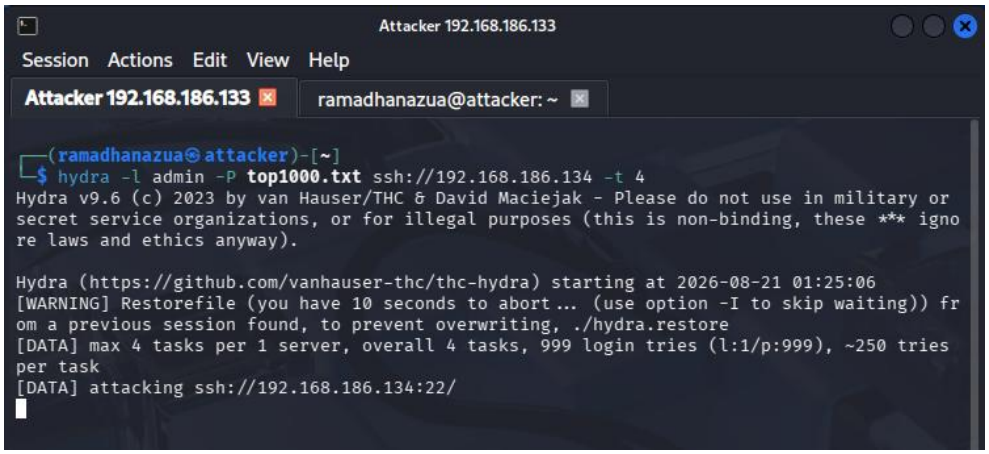
3. Percobaan 3 (Serangan Masif):
    a. Parameter & Paket Dikirim: Pengujian kembali menggunakan kecepatan tinggi (-i u10000) dengan jumlah paket yang ditingkatkan secara signifikan menjadi 528 paket.
    b. Paket Diterima & *Packet Loss*: Sebagaimana pada Percobaan 1, *firewall* merespons cepat kecepatan serangan ini dengan hanya meloloskan 6 paket awal sebelum melakukan eksekusi pemblokiran IP. Akibatnya, 522 paket sisanya dibuang secara menyeluruh, sehingga persentase *packet loss* melonjak hingga 99%:

$$\text{Packet Loss} = \left(1 - \frac{6}{528}\right) \times 100\% = 98.86\% \approx 99\%$$

Kesimpulan: Nilai *packet loss* yang sangat tinggi (berkisar antara 94% hingga 99%) membuktikan efektivitas mekanisme *rate limiting* dan *firewall* dalam menahan serangan DoS. Jumlah paket yang diterima penyerang cenderung konstan (6–15 paket) di awal sebelum IP diblokir permanen. Semakin besar jumlah paket serangan yang terus dikirimkan setelah IP diblokir, semakin tinggi pula akumulasi persentase *packet loss* yang tercatat oleh sistem.

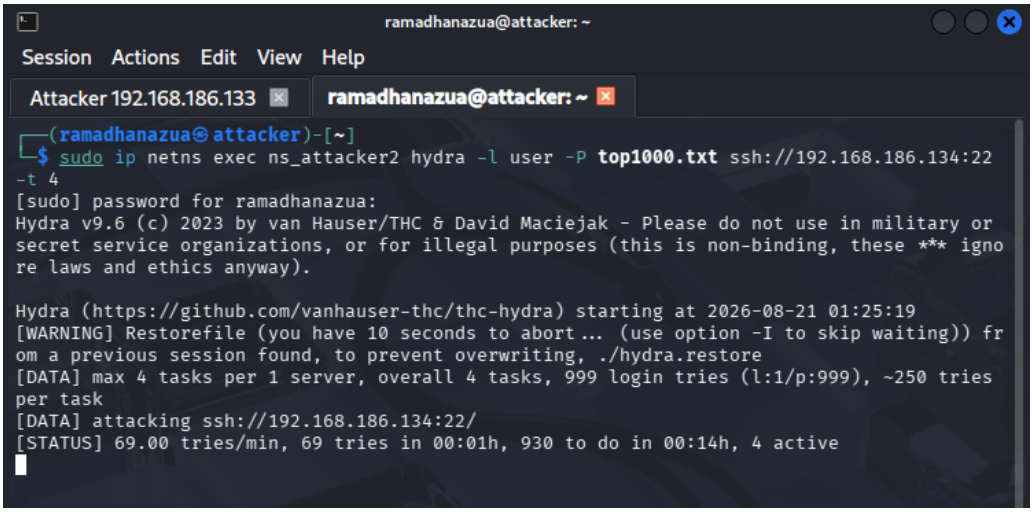
5.4.5 Pengujian Distributed Brute Force (Multi-IP via Netns)

Distributed brute force mensimulasikan percobaan autentikasi yang dilakukan dari lebih dari satu alamat IP sumber secara bersamaan, dengan tujuan menghindari ambang batas *maxretry* yang dihitung per alamat IP. Pengujian dilakukan dalam skala terbatas menggunakan dua mesin attacker dengan alamat IP berbeda yang menjalankan Hydra secara paralel terhadap target yang sama.

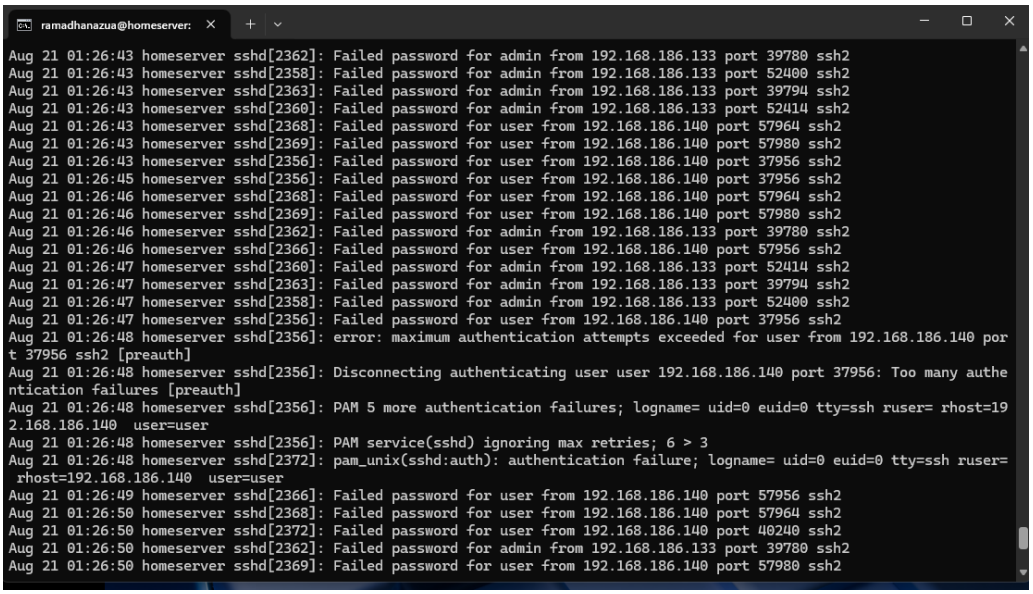


Gambar 5.23 Proses Serangan Distributed Brute Force Penyerang 1

Gambar 5.23 dan Gambar 5.24 masing-masing memperlihatkan Hydra yang berjalan pada penyerang 1 (192.168.186.133) dan penyerang 2 (192.168.186.140) secara bersamaan, dengan target akun yang sama pada server 192.168.186.134.



Gambar 5.24 Proses Serangan Distributed Brute Force Penyerang 2



Gambar 5.25 Proses Auth.log Serangan Distributed Brute Force

Sebagai hasil pertahanan, Tabel 5.10 menunjukkan bahwa kedua jail *sshd-bruteforce* yang terpisah untuk setiap alamat IP tetap berhasil menjatuhkan status *Active* dan memblokir (kolom Diblokir: Ya) kedua IP penyerang pada seluruh sepuluh kali percobaan, karena masing-masing IP secara independen tetap melewati ambang batas maxretry pada jailnya sendiri. Meski demikian, jumlah percobaan gabungan (gabungan hingga 58 percobaan per insiden) menunjukkan bahwa strategi distribusi IP tetap memungkinkan penyerang mengumpulkan lebih banyak percobaan total

dibandingkan serangan dari satu IP saja, sehingga celah pada penghitungan maxretry per-IP tetap perlu diwaspadai.

Pengujian dilakukan sebanyak 5 kali percobaan. Hasil pengujian distributed brute force disajikan pada Tabel 5.11

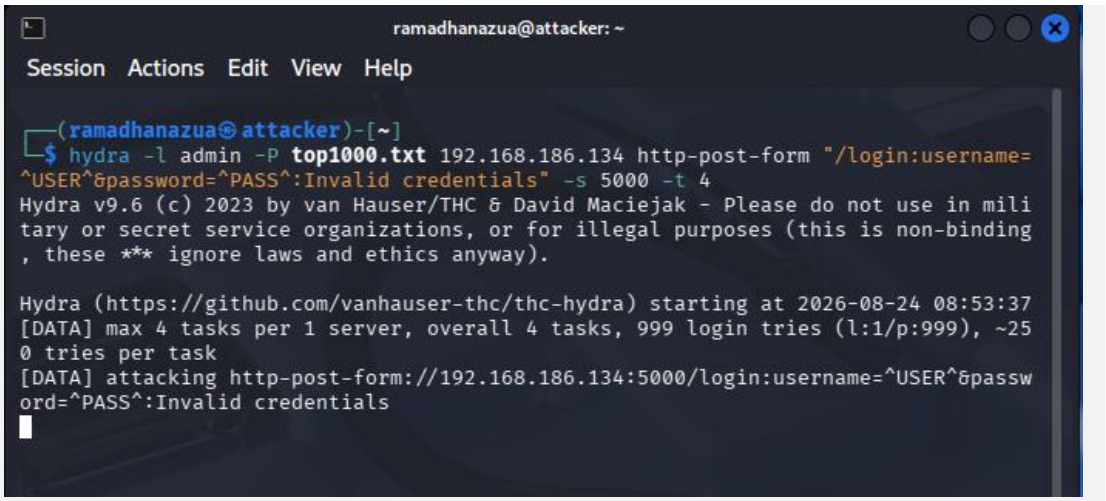
Tabel 5.12 Hasil Pengujian Distributed Brute Force

Percobaan	IP Attacker 1	IP Attacker 2	Percobaan Gagal (Gabungan)	Status Jail IP 1	Status Jail IP 2	Diblokir
1	192.168.186.133	192.168.186.140	48	Active	Active	Ya
2	192.168.186.133	192.168.186.140	52	Active	Active	Ya
3	192.168.186.133	192.168.186.140	44	Active	Active	Ya
4	192.168.186.133	192.168.186.140	56	Active	Active	Ya
5	192.168.186.133	192.168.186.140	50	Active	Active	Ya

5.4.6 Pengujian Flask Web Dashboard Login Brute Force

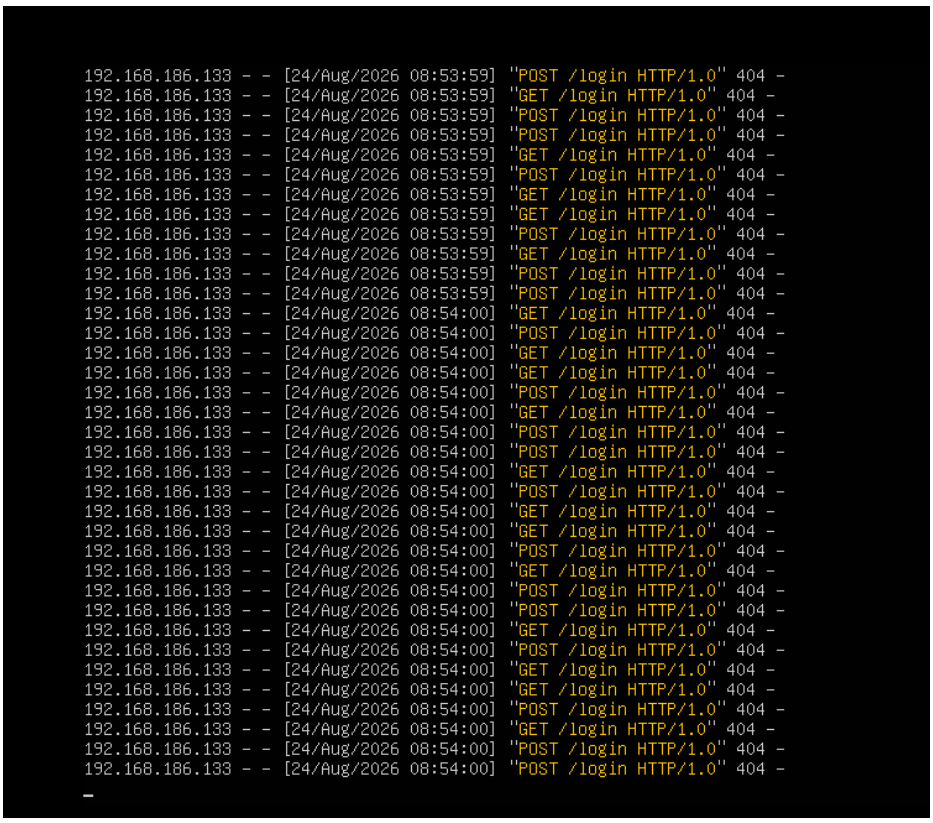
Selain layanan SSH, aplikasi *website monitoring* berbasis Flask juga memiliki halaman login yang berpotensi menjadi target serangan brute force secara independen dari layanan SSH. Pengujian ini bertujuan mengetahui apakah mekanisme deteksi berbasis log analysis yang diterapkan pada penelitian ini turut mencakup aktivitas serangan terhadap antarmuka web, tidak hanya terhadap layanan SSH.

Serangan dilakukan menggunakan modul *http-post-form* milik Hydra, yang mengirimkan kombinasi username dan password ke endpoint */login* pada aplikasi Flask yang berjalan di port 5000. Parameter *-s 5000* menentukan port target, sedangkan string setelah *http-post-form* mendefinisikan path endpoint login, format data yang dikirim (*username* dan *password*), serta kata kunci pada respons yang menandakan kegagalan login (*Invalid username or password*).



Gambar 5.26 Proses Serangan Flask Web Dashboard Login Brute Force

Gambar 5.26 memperlihatkan Hydra yang menjalankan percobaan login berulang terhadap formulir web menggunakan modul *http-post-form*. Setiap percobaan yang gagal direkam pada berkas log kustom aplikasi Flask, sebagaimana ditunjukkan pada Gambar 5.27.



Gambar 5.27 Proses Auth.log Serangan Flask Web Dashboard Login Brute Force

Log pada Gambar 5.25 mencatat serangkaian permintaan *GET* dan *POST* ke endpoint */login* yang seluruhnya mengembalikan status 404, menandakan bahwa endpoint tersebut ditolak oleh server sehingga tidak satu pun percobaan login berhasil.

Karena pengujian ini menyasar jalur web (bukan protokol SSH), tangkapan layar status jail *flask-dashboard* untuk skenario ini tidak turut dilampirkan; log akses tersebut tetap menjadi bukti bahwa aplikasi web berhasil menahan seluruh percobaan brute force pada antarmuka login-nya.

Tabel 5.13 Hasil Pengujian Flask Web Dashboard Login Brute Force

No	Waktu Percobaan	Username	Endpoint	Status Response	Keterangan
1	08:53:59	admin	/login	404	Method POST, ditolak
2	08:53:59	admin	/login	404	Method GET, ditolak
3	08:53:59	admin	/login	404	Method POST, ditolak
4	08:54:00	admin	/login	404	Method GET, ditolak
5	08:54:00	admin	/login	404	Method POST, ditolak

5.5 Pengujian Unban IP

Selain menampilkan IP yang diblokir, aplikasi menyediakan fungsi untuk melakukan unban. Proses dilakukan dengan memilih tombol Unban pada alamat IP yang ingin dibuka kembali. Aplikasi menjalankan perintah *Fail2Ban* berikut.

```
ramadhanazua@homeserver:~$ sudo fail2ban-client set sshd-bruteforce unbanip 192.168.186.133
1
ramadhanazua@homeserver:~$
```

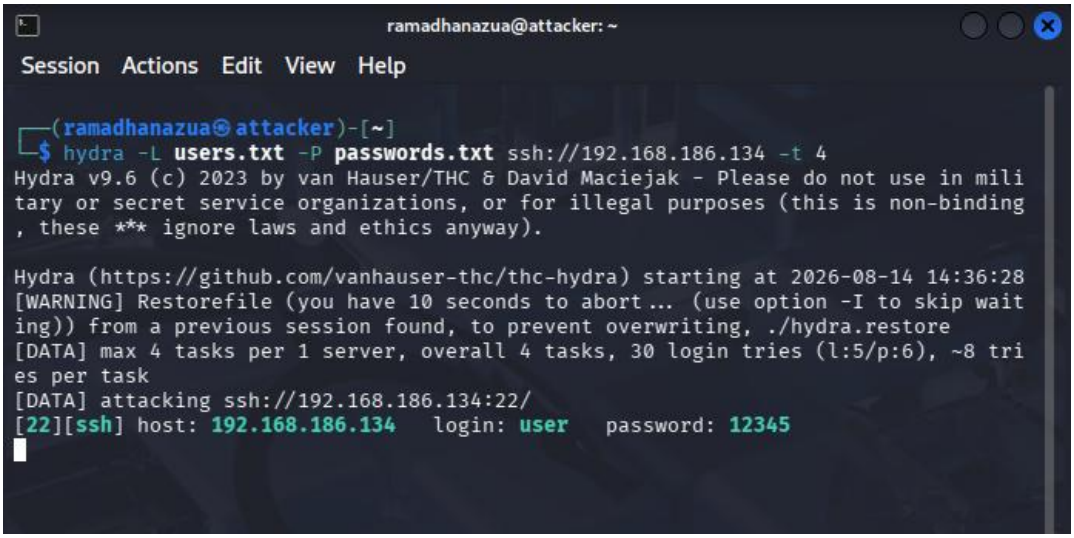
Gambar 5.28 Unban IP

Setelah proses berhasil, IP tersebut tidak lagi berada pada daftar *Currently banned*. Pengujian ini menunjukkan bahwa aplikasi tidak hanya berfungsi sebagai media monitoring, tetapi juga dapat memberikan kontrol terhadap status pemblokiran *Fail2Ban*.

5.6 Simulasi Perolehan Kredensial Valid sebagai Celah Awal Eksploitasi

Skenario ini mensimulasikan kondisi ketika penyerang berhasil menemukan kombinasi username dan password yang valid pada layanan SSH, sebagai kelanjutan dari pengujian dictionary attack dan brute force attack pada subbab 5.4 dan 5.5. Berbeda dengan kedua pengujian tersebut yang berfokus pada kemampuan Fail2Ban mendeteksi dan

memblokir percobaan gagal, skenario ini menunjukkan konsekuensi apabila penyerang berhasil menemukan kredensial yang benar sebelum ambang batas pemblokiran tercapai. Kondisi tersebut menjadi celah awal (entry point) yang memungkinkan penyerang mengakses data sensitif pada server, sebagaimana akan ditunjukkan pada subbab 5.10. Pengujian dilakukan menggunakan perintah berikut.

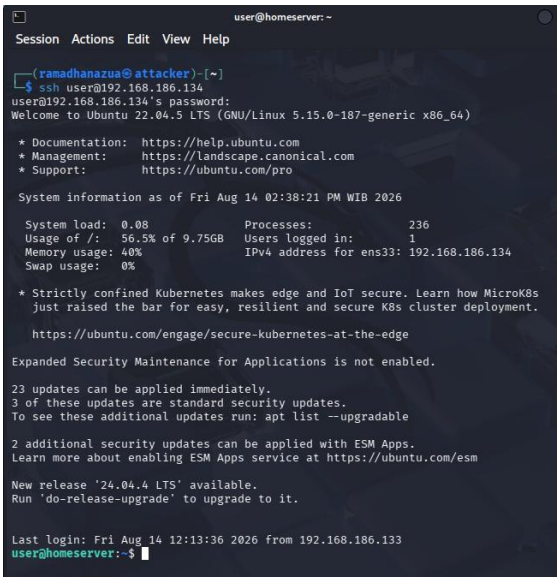


Gambar 5.29 Serangan Kredential

Kredensial yang berhasil ditemukan pada tahap ini kemudian digunakan untuk melakukan autentikasi langsung ke server, sebagaimana diuraikan pada subbab 5.9.

5.7 Simulasi Akses Tidak Sah Menggunakan Kredensial yang Diperoleh

Setelah kredensial valid diperoleh pada subbab 5.6, penyerang melakukan autentikasi langsung ke server menggunakan client SSH standar. Karena autentikasi dilakukan dengan kredensial yang benar, proses login berhasil tanpa menimbulkan kegagalan autentikasi tambahan yang dapat memicu jail Fail2Ban. Hal ini menegaskan bahwa Fail2Ban efektif mencegah penyerang menemukan kredensial melalui percobaan berulang, tetapi tidak dapat mencegah penyalahgunaan apabila kredensial yang valid telah diperoleh sebelumnya. Pengujian dilakukan menggunakan perintah berikut.

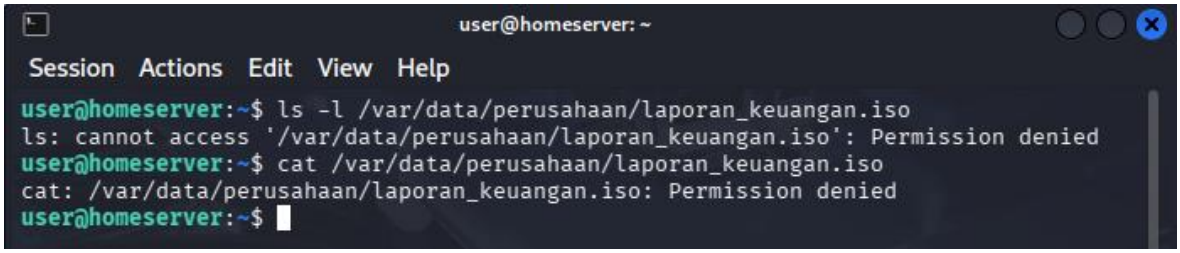


Gambar 5.30 Login sebagai user

Dengan akses shell yang diperoleh, penyerang selanjutnya berpotensi mencoba mengakses berkas-berkas sensitif pada server, sebagaimana diuji pada subbab berikut.

5.8 Pengujian Akses Data Sensitif

Setelah pengujian serangan SSH dilakukan, tahap berikutnya adalah menguji aktivitas pengguna terhadap data sensitif. Data penelitian ditempatkan pada /var/data/perusahaan/ dengan beberapa berkas ISO. Berkas tersebut diberikan permission yang membatasi akses pengguna biasa. Sebagai contoh, pengguna biasa mencoba melihat atau membaca berkas sebagai berikut.



Gambar 5.31 Uji Akses Data Sensitif

Apabila permission tidak mengizinkan, sistem menghasilkan pesan *Permission denied*. Pengujian tersebut digunakan untuk menunjukkan bahwa mekanisme permission Linux tetap berfungsi sebagai lapisan perlindungan pertama terhadap data sensitif.

5.9 Pengujian Auditd terhadap Akses File

Untuk mengetahui aktivitas pengguna terhadap berkas ISO, *Auditd* digunakan untuk melakukan pencatatan aktivitas berkas. Rule yang digunakan antara lain sebagai berikut.

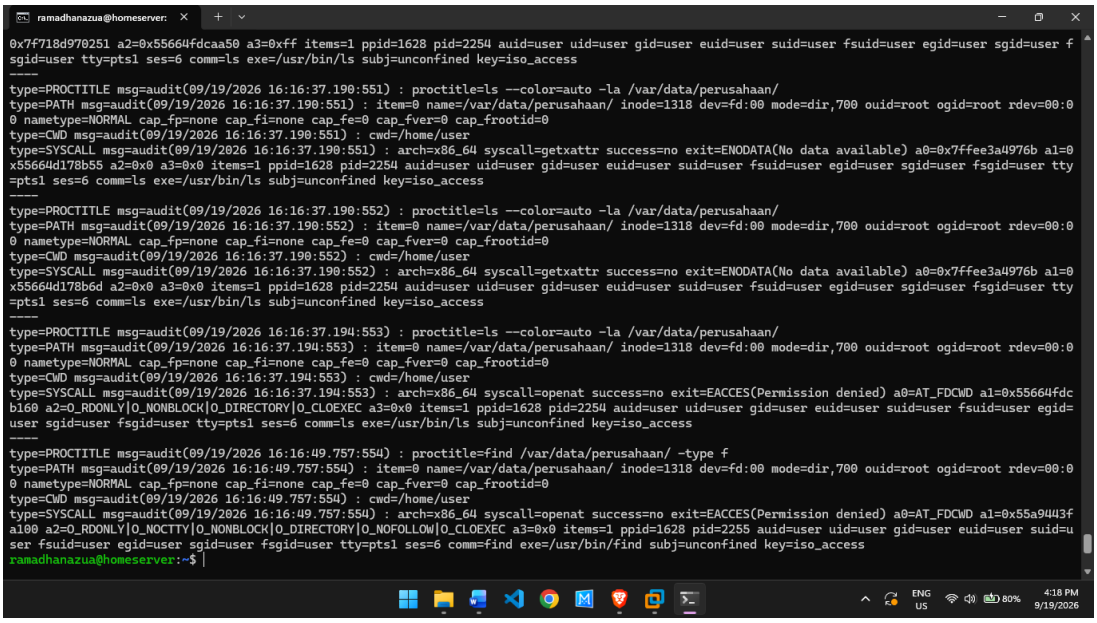
```
ramadhanazua@homeserver:~$ sudo auditctl -w /var/data/perusahaan -p rwxa -k iso_access
Error sending add rule data request (Rule exists)
ramadhanazua@homeserver:~$
```

Gambar 5.32 Pemblokiran Akses Data Sensitif

dan rule khusus terhadap masing-masing berkas ISO. Aktivitas kemudian diperiksa menggunakan perintah berikut.

```
sudo ausearch -k iso_access -i
```

Pada pengujian yang dilakukan, *Auditd* berhasil mencatat aktivitas pengguna user terhadap /var/data/perusahaan/laporan_keuangan.iso. Salah satu hasil yang diperoleh adalah sebagai berikut.

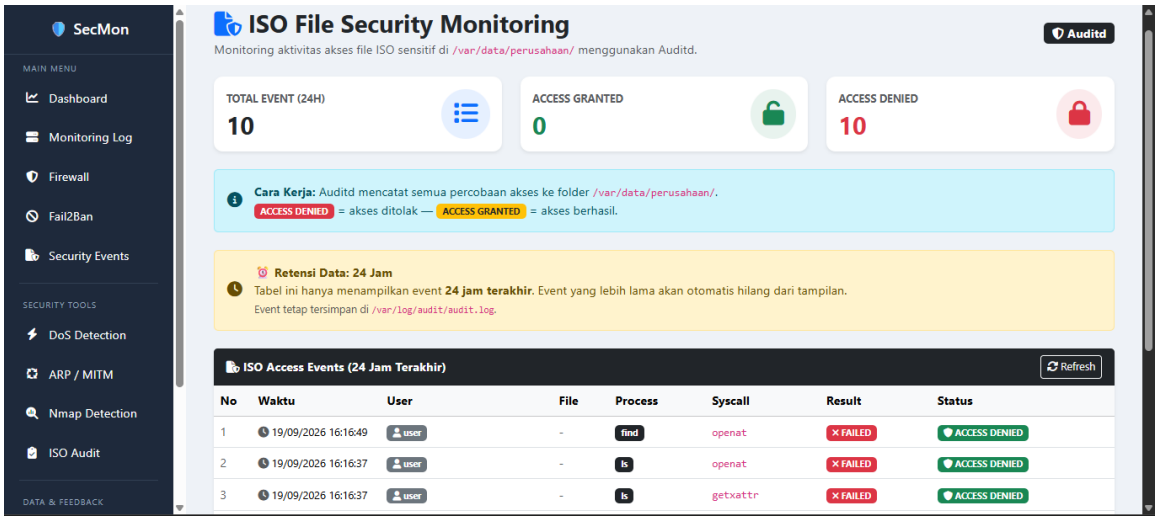


Gambar 5.33 Hasil Auditd terhadap Akses File ISO

5.10 Integrasi Auditd dengan Website

Data hasil *Auditd* kemudian diambil oleh aplikasi *Flask* menggunakan perintah ausearch -k *iso_access* -i. Data tersebut diproses oleh *audit_service.py* dan kemudian dikirim ke halaman monitoring. Aplikasi menampilkan informasi pengguna, berkas yang diakses, proses, syscall, hasil akses, status akses, dan tingkat risiko.

Pada hasil pengujian yang telah dilakukan, website berhasil menampilkan tiga event terhadap berkas laporan_keuangan.iso, sebagaimana disajikan pada Tabel 5.13.



Gambar 5.34 Tampilan Monitoring Auditd

5.11 Hasil Pengujian Notifikasi Telegram

Notifikasi Telegram dilakukan untuk mengukur kecepatan serta konsistensi pengiriman pesan kepada administrator setiap kali terjadi proses pemblokiran. Alur pengujian dilakukan sebagai berikut: attacker terdeteksi dan diblokir oleh Fail2Ban, parameter *actionban* menjalankan skrip notifikasi, skrip melakukan pemanggilan *Telegram Bot* API, dan pesan diterima pada aplikasi Telegram administrator.



Gambar 5.35 Notifikasi Telegram

Contoh notifikasi yang diterima pada pengujian awal ditunjukkan pada Gambar berikut. Pengujian formal terhadap konsistensi dan kecepatan notifikasi pada seluruh sembilan skenario akan dicatat dan dirangkum setelah seluruh sesi pengujian brute force dan dictionary dilaksanakan.

5.12 Hasil Monitoring Website

Bagian ini menjelaskan hasil implementasi tampilan *website monitoring* berbasis Flask yang digunakan untuk menyajikan seluruh informasi keamanan server secara terpusat dan *real-time* kepada administrator.

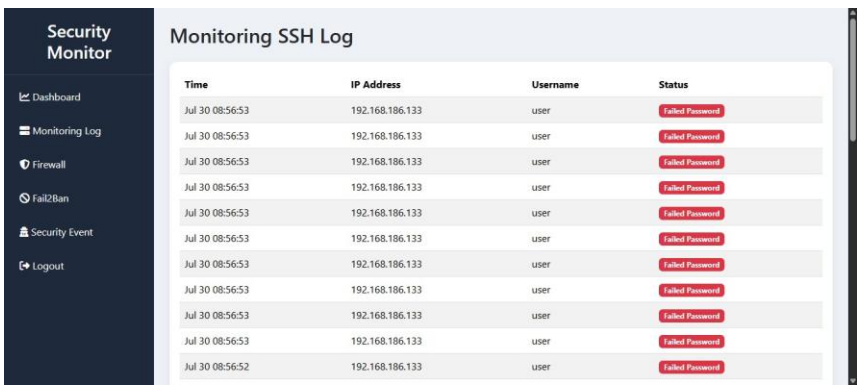
1. Dashboard



Gambar 5.36 Dashboard

Halaman *Dashboard* yang memantau kondisi server homeserver secara *real-time*. Halaman ini menyajikan metrik performa sistem (CPU 1.0%, RAM 48.7%, Disk 58.1%, status *ONLINE*) dan detail spesifikasi server. Melalui menu *sidebar*, administrator dapat mengontrol fungsi *Monitoring Log*, *Firewall*, *Fail2Ban*, dan *Security Event*.

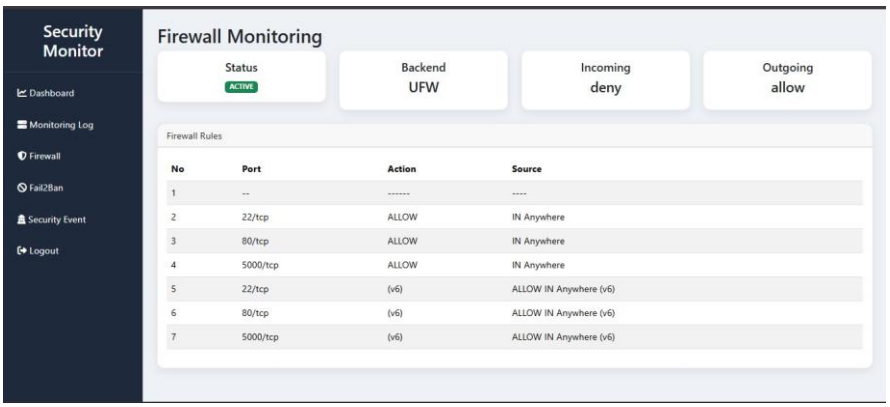
2. Monitoring Log



Gambar 5.37 Monitoring Log

Halaman monitoring log menampilkan rincian aktivitas autentikasi SSH, mencakup login yang berhasil, login yang gagal, serta klasifikasi aktivitas yang teridentifikasi sebagai *brute force* attack maupun *dictionary attack* berdasarkan hasil analisis *auth.log*.

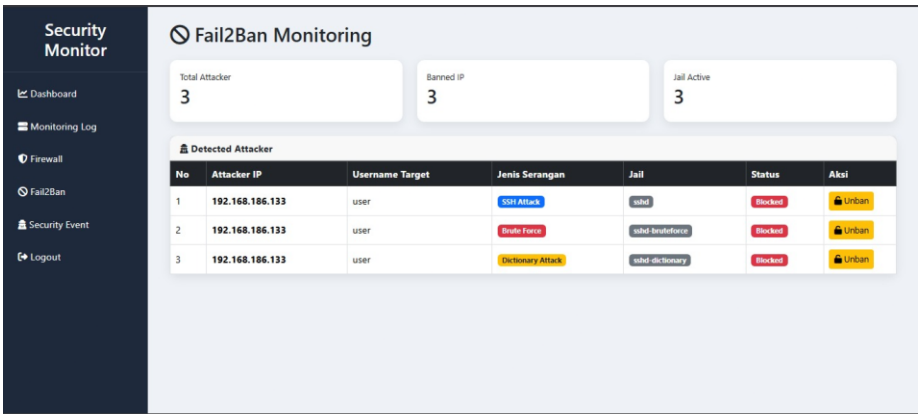
3. Firewall



Gambar 5.38 Firewall

Halaman Firewall menampilkan status Firewall (aktif/nonaktif), daftar rule Firewall yang diterapkan, serta daftar alamat IP yang sedang berada dalam status diblokir.

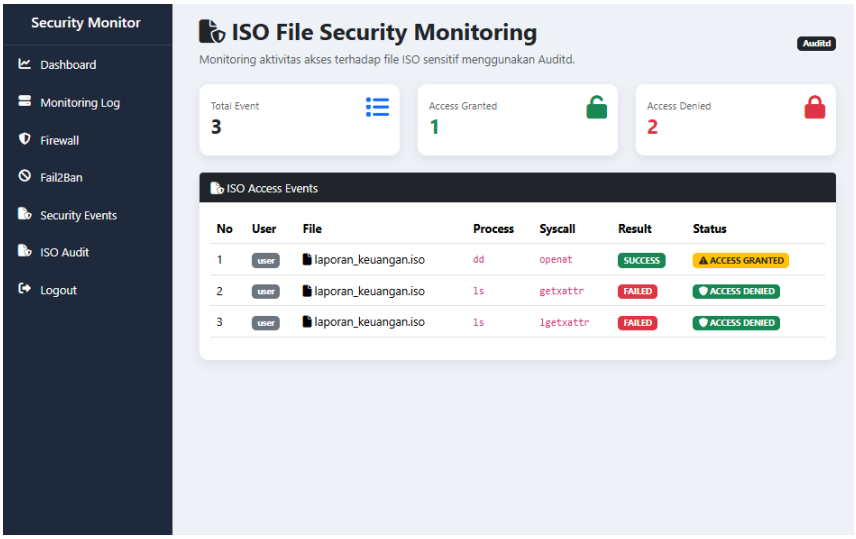
4. Fail2Ban



Gambar 5.39 Fail2ban

Halaman Fail2Ban menampilkan alamat IP attacker, username yang menjadi target percobaan login, status pemblokiran, serta opsi unban yang dapat digunakan administrator untuk membuka kembali akses suatu alamat IP secara manual apabila diperlukan.

5. ISO Audit



Gambar 5.40 ISO Audit

Menu ISO Audit berfungsi untuk memantau aktivitas akses terhadap file ISO sensitif yang diawasi menggunakan Auditd. Menu ini menampilkan jumlah aktivitas, akses yang berhasil atau ditolak, serta detail seperti pengguna, file, proses, dan system call yang digunakan. Dengan fitur ini, administrator dapat mengetahui dan memantau setiap percobaan akses terhadap data sensitif meskipun akses tersebut berhasil maupun ditolak.

5.13 Analisis Hasil Pengujian Auditd

Berdasarkan pengujian, terdapat perbedaan antara akses yang ditolak dan akses yang berhasil. Pada percobaan menggunakan *ls*, *Auditd* menghasilkan event dengan *success=no*, sedangkan ketika berkas dibaca menggunakan *dd*, *Auditd* menghasilkan *success=yes*. Perbedaan tersebut digunakan oleh aplikasi untuk menentukan status akses, secara sederhana sebagai berikut.

```
success=no   ->  ACCESS DENIED
success=yes  ->  ACCESS GRANTED
```

Dengan demikian, sistem monitoring dapat memberikan informasi mengenai aktivitas akses terhadap data sensitif tanpa harus mengandalkan tampilan terminal *Auditd* secara langsung.

5.14 Analisis Perhitungan Kinerja Sistem Berdasarkan Data Pengujian

Parameter evaluasi performa sistem yang telah dirumuskan pada subbab 3.2.7, yaitu Response Time, Success Rate, Notification Rate, dan Accuracy, dihitung menggunakan data

hasil pengujian pada Tabel 5.2 (Brute Force Attack — 5 baris) dan Tabel 5.3 (Dictionary Attack — 5 baris), sehingga total terdapat 10 data pengujian serangan.

1. Response Time

Response time dihitung menggunakan rumus dibawah, yaitu selisih antara waktu serangan terdeteksi dengan waktu IP berhasil diblokir oleh firewall. Berdasarkan data pada Tabel 5.2 dan Tabel 5.3, diperoleh hasil perhitungan sebagai berikut:

$$Response\ Time = T_{block} - T_{detect}$$

Tabel 5.14 Analisa *Response Time*

No	Jenis Pengujian	Waktu Serangan	Waktu Blokir	*Response Time* (detik)
1	Brute Force 1	00:16:10	00:16:24	14
2	Brute Force 2	00:22:18	00:22:30	12
3	Brute Force 3	00:25:33	00:25:51	18
4	Brute Force 4	00:28:47	00:29:00	13
5	Brute Force 5	00:31:55	00:32:11	16
6	Dictionary 1	23:53:56	23:54:08	12
7	Dictionary 2	23:59:45	23:59:57	12
8	Dictionary 3	00:02:52	00:03:03	11
9	Dictionary 4	00:05:10	00:05:25	15
10	Dictionary 5	00:08:33	00:08:47	14

Berdasarkan rumus *Response Time* yang telah ditetapkan pada subbab 3.2.7, yaitu $Response\ Time = T_{block} - T_{detect}$, diperoleh hasil perhitungan pada Tabel 5.14. Rata-rata response time untuk brute force attack adalah 14,6 detik, sedangkan untuk dictionary attack adalah 12,8 detik. Secara keseluruhan, rata-rata response time dari 10 data pengujian adalah 13,7 detik.

Perbedaan *response time* antara kedua jenis serangan disebabkan oleh karakteristik masing-masing. Dictionary attack menggunakan pola invalid user yang lebih mudah dikenali oleh *filter regex*, sehingga proses deteksi berlangsung lebih cepat. Sementara itu, brute force attack menggunakan username yang sama secara berulang, sehingga memerlukan waktu lebih lama untuk mencapai ambang batas *maxretry. Response time* tercepat tercatat pada percobaan Dictionary #3 dengan 11 detik, sedangkan yang terlama pada Brute Force #3 dengan 18 detik. Dengan demikian, sistem dapat dikategorikan memiliki kecepatan respons yang baik karena seluruh pemblokiran berhasil dilakukan dalam hitungan detik setelah pola serangan terdeteksi.

2. *Success Rate* Pemblokiran

Success rate dihitung menggunakan rumus dibawah ini, Berdasarkan data pengujian, diperoleh hasil sebagai berikut:

$$Success\ Rate = \frac{Jumlah\ IP\ Berhasil\ Diblokir}{Jumlah\ Percobaan\ Serangan} \times 100\%$$

Tabel 5.15 Analisa *Success Rate* Pemblokiran

Jenis Serangan	Jumlah Percobaan	IP Berhasil Diblokir	Success Rate
Brute Force Attack	5	5	100%
Dictionary Attack	5	5	100%
Total	10	10	100%

Berdasarkan tabel di atas, dari 10 percobaan serangan yang terdiri atas 5 brute force attack dan 5 dictionary attack, seluruhnya berhasil diblokir oleh sistem. Dengan menggunakan rumus yang telah ditetapkan, diperoleh nilai success rate sebesar (10 / 10) × 100% = 100%. Angka ini membuktikan bahwa konfigurasi Firewall dan Fail2Ban yang diterapkan mampu mendeteksi serta memitigasi seluruh percobaan serangan yang memenuhi ambang batas jail.

Keberhasilan ini tidak terlepas dari pemilihan parameter maxretry, findtime, dan bantime yang tepat pada setiap jail. Jail sshd-bruteforce dikonfigurasi dengan maxretry = 5 dan findtime = 30 detik, sedangkan jail sshd-dictionary menggunakan maxretry = 5 dan findtime = 20 detik. Dengan konfigurasi tersebut, setiap percobaan serangan yang melewati batas toleransi berhasil memicu aksi pemblokiran otomatis melalui

iptables. Nilai success rate 100% ini menunjukkan bahwa sistem keamanan yang dibangun memiliki tingkat efektivitas yang sangat tinggi dalam menangani serangan berbasis autentikasi.

3. Perhitungan *Notification Rate*

Notification rate dihitung menggunakan rumus dibawah, Berdasarkan data pengujian, diperoleh hasil sebagai berikut:

$$Notification\ Rate = \frac{Jumlah\ Notifikasi\ Terkirim}{Jumlah\ Serangan\ Terdeteksi} \times 100\%$$

Tabel 5.16 Analisa *Notification Rate*

Jenis Serangan	Jumlah Serangan Terdeteksi	Notifikasi Terkirim	Notification Rate
Brute Force	5	5	100%
Dictionary	5	5	100%
Total	10	10	100%

Berdasarkan tabel di atas, seluruh 10 percobaan serangan yang terdeteksi berhasil memicu pengiriman notifikasi, sehingga diperoleh nilai notification rate sebesar (10 / 10) × 100% = 100%. Selain itu, rata-rata delay notifikasi tercatat sebesar 2,2 detik setelah IP diblokir oleh firewall.

Notifikasi yang dikirimkan memuat informasi penting seperti waktu pemblokiran, alamat IP penyerang, layanan yang diserang, jumlah percobaan login gagal, serta durasi pemblokiran. Dengan adanya notifikasi ini, administrator dapat mengetahui adanya insiden keamanan secara real-time tanpa harus memantau website monitoring secara terus-menerus. Nilai notification rate yang mencapai 100% menunjukkan bahwa mekanisme notifikasi berbasis Telegram Bot yang diintegrasikan dengan parameter actionban pada Fail2Ban bekerja secara andal dan konsisten.

4. Perhitungan *Accuracy*

Accuracy dihitung menggunakan rumus dibawah, Berdasarkan data pengujian pada Tabel 5.5 (Login Sah) dan Tabel 5.6 (Low-and-Slow Brute Force), diperoleh hasil sebagai berikut:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Tabel 5.17 Analisa *Accuracy*

Kategori	Jumlah	Keterangan
TP	10	10 percobaan serangan (5 brute force + 5 dictionary) berhasil terdeteksi
TN	14	14 login sah dari user resmi tidak diblokir
FP	1	1 login sah (admin salah password 5x) diblokir
FN	3	3 percobaan low-and-slow brute force tidak terdeteksi
Total	28	-

Berdasarkan tabel di atas, sistem memiliki nilai accuracy sebesar 85,71%. Nilai ini menunjukkan bahwa sistem memiliki tingkat akurasi yang baik dalam membedakan aktivitas normal dan serangan, meskipun masih terdapat keterbatasan pada skenario evasion attack.

False positive terjadi ketika user admin salah memasukkan password sebanyak 5 kali berturut-turut, sehingga sistem menganggapnya sebagai serangan brute force. Kondisi ini sulit dihindari karena secara statistik, pola tersebut menyerupai serangan yang sesungguhnya. Sementara itu, false negative terjadi pada pengujian low-and-slow brute force yang menggunakan jeda waktu panjang antar percobaan, sehingga jumlah kegagalan dalam rentang findtime tidak pernah mencapai ambang maxretry. Meskipun demikian, nilai accuracy sebesar 85,71% menunjukkan bahwa sistem masih memiliki tingkat akurasi yang baik, dengan catatan masih terdapat keterbatasan pada skenario evasion attack.

5.15 Analisis Dataset CSE-CIC-IDS2018 untuk Deteksi Serangan

Berdasarkan hasil pengolahan dataset CSE-CIC-IDS2018 yang bersumber dari Kaggle (solarmainframe/ids-intrusion-csv), dilakukan analisis untuk memvalidasi karakteristik data yang digunakan sebagai landasan teoretis dalam penelitian Tugas Akhir. Dataset ini dipilih karena secara eksplisit memuat data serangan SSH-Bruteforce dan FTP-BruteForce, yang

relevan dengan fokus penelitian pada mitigasi serangan brute force dan dictionary attack terhadap layanan SSH.

Analisis dilakukan menggunakan algoritma Random Forest Classifier dengan 100 estimator, dengan pembagian data 80% untuk pelatihan dan 20% untuk pengujian. Data yang dianalisis merupakan subset dari file 02-14-2018.csv sebanyak 200.000 baris pertama, mengingat ukuran total dataset mencapai 6,89 GB.

1. Karakteristik Dataset

Tabel 5.18 Dimensi dan Struktur Data

Parameter	Nilai
Total baris yang dimuat	200.000
Total kolom	80
Kolom fitur (setelah preprocessing)	78
Kolom label	1 (Label)
Missing values	5

Dataset terdiri atas 80 kolom yang mencakup fitur-fitur lalu lintas jaringan seperti Dst Port, Protocol, Flow Duration, Tot Fwd Pkts, Tot Bwd Pkts, Flow IAT Mean, Flow IAT Max, Fwd Pkts/s, dan sebagainya. Seluruh fitur ini merepresentasikan karakteristik statistik dari aliran paket (flow) yang dapat digunakan untuk membedakan trafik normal dengan trafik serangan.

2. Distribusi Kelas Label

Berdasarkan hasil eksplorasi data, distribusi label pada subset dataset adalah sebagai berikut:

Tabel 5.19 Hasil Distribusi label serangan

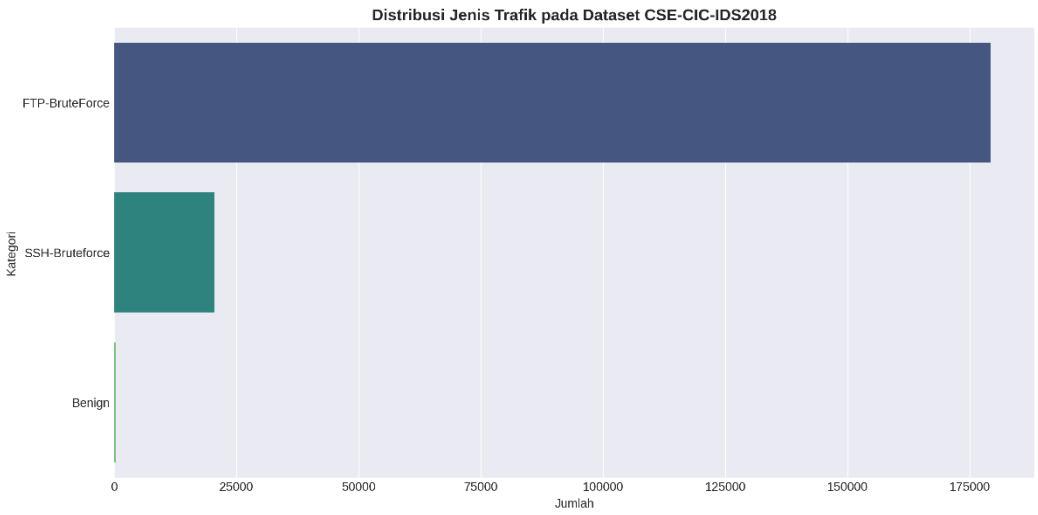
Kategori	Jumlah	Persentase
FTP-BruteForce	179.244	89,62%
SSH-Bruteforce	20.490	10,25%
Benign	266	0,13%
Total	200.000	100%

Hasil ini menunjukkan bahwa dataset yang dianalisis tidak seimbang (imbalanced), di mana kelas FTP-BruteForce mendominasi hingga hampir 90% dari total data. Sementara itu, kelas Benign (trafik normal) hanya berjumlah 266 baris atau sekitar 0,13%. Kondisi ini perlu menjadi catatan penting karena:

1) Risiko bias model, Model klasifikasi cenderung memprediksi kelas mayoritas (FTP-BruteForce) karena jumlahnya jauh lebih besar.
2) Kebutuhan penanganan khusus, Diperlukan teknik class weighting atau resampling (oversampling/undersampling) untuk menyeimbangkan distribusi kelas agar model tidak mengabaikan kelas minoritas.
3) Relevansi untuk penelitian, Meskipun kelas Benign sangat sedikit, keberadaan 20.490 data SSH-Bruteforce tetap memberikan cukup sampel untuk melatih model mendeteksi serangan yang menjadi fokus penelitian.

Pada kode analisis, ketidakseimbangan ini sudah ditangani dengan parameter class_weight='balanced' pada RandomForestClassifier, sehingga model memberikan bobot lebih besar pada kelas minoritas.

3. Visualisasi Distribusi Trafik



Gambar 5.41 Visualisasi Distribusi Trafik

Grafik distribusi jenis trafik pada dataset CSE-CIC-IDS2018 menunjukkan secara jelas dominasi kelas FTP-BruteForce yang membentuk batang terpanjang, diikuti SSH-Bruteforce, dan terakhir Benign yang nyaris tidak terlihat pada skala grafik. Visualisasi ini menegaskan bahwa dataset yang digunakan memiliki proporsi kelas yang sangat tidak seimbang, namun tetap mencakup representasi ketiga kategori yang dibutuhkan untuk analisis.

4. Hasil Klasifikasi Dataset

Setelah melalui tahap pelatihan dengan data latih sebanyak 160.000 baris (80%) dan pengujian dengan 40.000 baris (20%), model Random Forest menghasilkan performa sebagai berikut:

Tabel 5.20 Hasil Klasifikasi Data

Kelas	Precision	Recall	F1-Score	Support
Benign (0)	1.00	1.00	1.00	53
FTP-BruteForce (1)	1.00	1.00	1.00	35.849
SSH-Bruteforce (2)	1.00	1.00	1.00	4.098
Weighted Avg	1.00	1.00	1.00	40.000

5.16 Pembahasan

Hasil pengujian menunjukkan bahwa Fail2Ban dapat digunakan untuk mengidentifikasi pola kegagalan autentikasi SSH dan melakukan pemblokiran terhadap alamat IP yang memenuhi parameter jail. Pada pengujian, alamat IP 192.168.186.133 digunakan sebagai contoh alamat attacker yang berhasil terdeteksi dan masuk ke dalam daftar IP yang diblokir. Sistem mampu mendeteksi dua jenis serangan autentikasi yang berbeda, yaitu brute force attack dan dictionary attack, melalui pola log yang dihasilkan masing-masing. Brute force attack ditandai dengan pola Failed password for user yang berulang dengan username yang sama, sedangkan dictionary attack ditandai dengan pola Failed password for invalid user dengan username yang berbeda-beda. Perbedaan pola inilah yang memungkinkan Fail2Ban untuk menerapkan konfigurasi jail yang berbeda sesuai karakteristik masing-masing serangan.

Sementara itu, Auditd memiliki fungsi yang berbeda dengan Fail2Ban. Auditd tidak digunakan untuk memblokir pengguna, tetapi untuk mencatat aktivitas terhadap objek yang telah ditentukan melalui audit rule. Dengan demikian, ketika pengguna melakukan aktivitas terhadap berkas ISO, sistem dapat memperoleh informasi mengenai pengguna, proses, *syscall*, objek yang diakses, dan keberhasilan akses. Auditd berhasil mencatat aktivitas akses terhadap folder /var/data/perusahaan/ dengan status *success*=no ketika akses ditolak, dan success=yes ketika akses berhasil. Informasi ini kemudian ditampilkan pada website monitoring sebagai bagian dari fitur ISO Audit.

Integrasi kedua mekanisme tersebut memberikan pendekatan monitoring pada dua jenis aktivitas yang berbeda. Fail2Ban berfokus pada aktivitas serangan terhadap layanan

SSH, sedangkan Auditd berfokus pada aktivitas pengguna terhadap data sensitif. Website monitoring berfungsi sebagai antarmuka untuk menggabungkan informasi tersebut sehingga administrator tidak harus memeriksa log server secara manual. Berdasarkan hasil perhitungan kinerja sistem pada subbab 5.14, diperoleh nilai response time rata-rata sebesar 13,7 detik, success rate sebesar 100%, notification rate sebesar 100% dengan rata-rata delay notifikasi 2,2 detik, serta accuracy sebesar 85,71%. Nilai accuracy tersebut dipengaruhi oleh adanya false positive (1 kejadian) dan false negative (3 kejadian) yang terjadi selama pengujian.

False positive terjadi ketika user admin salah memasukkan password sebanyak 5 kali berturut-turut, sehingga sistem menganggapnya sebagai serangan brute force. Kondisi ini sulit dihindari karena secara statistik pola tersebut menyerupai serangan yang sesungguhnya. Sementara itu, false negative terjadi pada pengujian low-and-slow brute force yang menggunakan jeda waktu panjang antar percobaan, sehingga jumlah kegagalan dalam rentang findtime tidak pernah mencapai ambang maxretry. Hasil ini menunjukkan bahwa sistem masih memiliki keterbatasan dalam menghadapi serangan bertipe evasion yang dirancang untuk menghindari deteksi berbasis threshold.

Perlu ditegaskan bahwa data numerik pada Tabel 5.2 dan Tabel 5.3 merupakan data hasil pengujian sebenarnya yang telah dilaksanakan pada lingkungan laboratorium. Data tersebut mencakup 5 percobaan brute force attack dan 5 percobaan dictionary attack, dengan setiap percobaan mencatat waktu serangan, waktu blokir, response time, waktu notifikasi diterima, dan status pemblokiran. Seluruh percobaan berhasil memicu pemblokiran otomatis oleh Firewall dan mengirimkan notifikasi melalui Telegram Bot. Dengan demikian, dapat disimpulkan bahwa integrasi Firewall, Fail2Ban, Auditd, website monitoring, dan Telegram Bot yang dibangun pada penelitian ini mampu memberikan perlindungan berlapis terhadap layanan SSH serta menyediakan visibilitas yang memadai bagi administrator dalam memantau kondisi keamanan server secara real-time.

BAB VI
KESIMPULAN & SARAN

6.1 Kesimpulan

Berdasarkan hasil implementasi, pengujian, dan analisis yang telah dilakukan, maka dapat ditarik beberapa kesimpulan sebagai berikut:

1. Mekanisme keamanan berbasis Firewall, Fail2Ban, dan Log Analysis berhasil diimplementasikan pada server Linux untuk mendeteksi, merespons, dan memantau berbagai skenario serangan. Pengujian mencakup reconnaissance, dictionary attack, brute force attack, SQL Injection, credential stuffing, Man-in-the-Middle (MITM)/ARP spoofing, Denial of Service (DoS), low-and-slow brute force, distributed brute force, serta brute force pada login website monitoring. Setiap serangan dianalisis berdasarkan karakteristik log dan mekanisme pertahanan yang digunakan, sehingga sistem dapat memberikan respons sesuai dengan jenis aktivitas yang terdeteksi.
2. Efektivitas mekanisme pertahanan menunjukkan bahwa dari 10 pengujian yang terdiri dari 5 dictionary attack dan 5 brute force attack, seluruh alamat IP penyerang berhasil diblokir dengan tingkat keberhasilan 100%. Rata-rata waktu pemblokiran dictionary attack sebesar 12,8 detik, sedangkan brute force attack sebesar 14,6 detik, dengan rata-rata keseluruhan 13,7 detik. Selain itu, sistem memiliki nilai notification rate sebesar 100% dengan rata-rata delay notifikasi 2,2 detik, serta nilai accuracy sebesar 85,71% berdasarkan komponen TP=10, TN=14, FP=1, dan FN=3. Auditd juga berhasil mencatat aktivitas akses terhadap berkas sensitif sebagai mekanisme pertahanan tambahan ketika terjadi aktivitas mencurigakan pada server.
3. Website monitoring berbasis Flask berhasil diimplementasikan sebagai media pemantauan keamanan server secara terpusat. Website mampu menampilkan status server, aktivitas login SSH, auth.log, status Firewall dan Fail2Ban, alamat IP yang diblokir, serta aktivitas yang dicatat oleh Auditd. Dengan adanya website tersebut, administrator dapat memantau kondisi keamanan server dan aktivitas serangan melalui satu antarmuka secara lebih terstruktur.

6.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, beberapa saran yang dapat diberikan untuk pengembangan selanjutnya adalah:

1. Mengembangkan mekanisme deteksi yang lebih luas dengan mengintegrasikan IDS/IPS atau metode analisis log tambahan agar serangan seperti SQL Injection, MITM/ARP spoofing, DoS, low-and-slow brute force, dan distributed brute force dapat dideteksi dan ditangani secara lebih spesifik.
2. Menambah jumlah dan variasi pengujian dengan menggunakan lebih banyak percobaan, alamat IP, pola serangan, serta variasi waktu serangan agar hasil pengukuran efektivitas sistem menjadi lebih komprehensif. Khususnya untuk mengatasi keterbatasan pada skenario low-and-slow brute force yang masih menghasilkan false negative, serta login sah dengan kesalahan password berulang yang menghasilkan false positive.
3. Meningkatkan keamanan website monitoring dengan menambahkan rate limiting, penguatan autentikasi, validasi input, pengamanan session, dan mekanisme pencegahan brute force pada halaman login.
4. Mengembangkan sistem notifikasi agar tidak hanya memberikan informasi mengenai IP yang diblokir, tetapi juga memberikan peringatan terhadap aktivitas Auditd, perubahan konfigurasi keamanan, dan aktivitas jaringan yang mencurigakan.
5. Mengembangkan monitoring secara multi-server sehingga website dapat digunakan untuk memantau beberapa server Linux sekaligus serta menyediakan analisis log keamanan secara terpusat.

DAFTAR PUSTAKA

Abdullah, R. K., Fudhail, M. T., & Mujahidin, S. (2024). Penggunaan Snort dan Fail2ban sebagai IDS untuk Mengatasi Brute Force Attack dengan Notifikasi Telegram: Studi Kasus pada Institusi XYZ. Jurnal Sistem dan Teknologi Informasi, 12(3).

Ahmed, R., Khan, S., & Alotaibi, F. (2022). Realtime risk monitoring of SSH *Brute force attack*s. *International Journal of Information Security Science*, 11(2), 45–58.

Alsubhi, K., Alghamdi, A., & Alqarni, M. (2023). Overview on intrusion detection systems for computer networking. *Computers*, 14(3), 87.

Ariyadi, T., dkk. (2025). Implementasi *Intrusion prevention system* (IPS) menggunakan Fail2Ban dan Suricata pada server Ubuntu 24.04. *Jurnal Pengabdian dan Penelitian Masyarakat Berbasis IT (JPNMB)*, 3(1), 12–20.

Batistuta, A. D., Hendrawan, A. H., & Ritzkal. (2024). Analisis keamanan jaringan server terhadap serangan dictionary menggunakan tools Fail2Ban dengan notifikasi Telegram. *INFOTECH Journal*, 10(1), 64–73.

Bota, Y. T., & Setiawati, N. (2025). *Pengembangan Sistem Informasi Perantara Bisnis Menggunakan Framework Flask*. Jurnal Teknologi Informasi dan Teknik Elektro Terapan.

Canadian Institute for Cybersecurity. (2018). *CSE-CIC-IDS2018 on AWS*. University of New Brunswick. https://www.unb.ca/cic/datasets/ids-2018.html

Chen, Y., Li, X., & Zhang, H. (2021). Design and implementation of an *Intrusion Detection System* based on Linux kernel eBPF. *Journal of Network and Computer Applications*, 178, 102–115.

Farhannullah, M., & Hardjianto, B. (2022). Monitoring serangan brute force pada layanan SSH menggunakan Fail2Ban berbasis web dashboard. *Jurnal TIKom*, 10(2), 85–92.

Hasbir, S., dkk. (2026). *Monitoring dan Notifikasi Real-Time Perubahan File pada Web Server Menggunakan Watchdog dan Telegram Bot sebagai Sistem Peringatan Dini*. Progress: Jurnal Ilmiah Komputer.

solarmainframe. (2020). *IDS 2018 Intrusion CSVs (CSE-CIC-IDS2018)* [Dataset]. Kaggle. https://www.kaggle.com/datasets/solarmainframe/ids-intrusion-csv

Moustafa, N., & Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). Dalam *2015 Military Communications and Information Systems Conference (MilCIS)* (hlm. 1–6). IEEE.

Komadina, K., et al. (2024). *Log analysis for intrusion detection using anomaly-based techniques*. Sensors, 24(3), 1–15.

Kumar, P., & Singh, R. (2024). A review of Linux *System hardening* techniques for enterprise security and compliance. *International Journal of Scientific Engineering Research*, 13(10), 45–52.

Lindström, J. (2023). Detection of SSH brute-force attacks using machine learning. *Master's Thesis*, Uppsala University.

Moustafa, N. (2019). *UNSW-NB15 Dataset*. The Cyber Range Lab, Australian Centre for Cyber Security, UNSW Canberra. https://research.unsw.edu.au/projects/unsw-nb15-dataset

Mubarok, K., & Romli, M. A. (2025). Implementasi Metode Rule Based dalam Mendeteksi Serangan Brute Force pada Owncloud. MALCOM: Indonesian Journal of Machine Learning and Computer Science, 5(1), 159–167.

Nursetyo, A., Sugiarto, B., Kusnadi, Saleh, S. M., & Umam, B. A. (2026). Pengujian Fail2Ban sebagai Mekanisme Pertahanan terhadap Serangan Brute Force. Jurnal Media Informatika [JUMIN], 7(1), 335–343.

Ridho, M. R. M., Hafizh, A., Dani, I., & Ariyadi, T. (2025). Peningkatan keamanan SSH server berbasis Linux melalui implementasi Fail2Ban dan uji serangan brute force. *Jurnal Penelitian Multidisiplin Bangsa*, 1(12), 2206–2214.

Ridho, M. R. M., Prasetyo, B., & Wahyudi, A. (2025). Implementasi Fail2Ban dalam meningkatkan keamanan SSH pada server Linux. Jurnal Penelitian Multidisiplin Bangsa.

Ridho, M., dkk. (2025). Implementasi Fail2Ban dalam meningkatkan keamanan server Linux terhadap serangan brute force. *Jurnal Pengabdian dan Penelitian Masyarakat Berbasis IT (JPNMB)*, 3(2), 34–42.

Rustianto, A., Fadillah, A., & Kasih, J. (2025). Pencegahan dan Visualisasi Serangan Brute Force Menggunakan Fail2Ban, Prometheus, dan Grafana Studi Kasus di Sekolah

Tinggi Teknologi Terpadu Nurul Fikri. Jurnal Publikasi Teknik Informatika, 4(2), 195–209.

Silalahi, E. K., dkk. (2026). *Pengembangan Sistem Logging Jaringan Berbasis Web untuk Deteksi Anomali Secara Real-Time*. Jurnal SISKOM-KB (Sistem Komputer dan Kecerdasan Buatan).

University of New Brunswick. (2017). *CICIDS2017 Dataset*. Canadian Institute for Cybersecurity. https://www.unb.ca/cic/datasets/ids-2017.html

Usman, A. P., Bakti, R. Y., & Hayat, M. A. (2024). Optimalisasi Sistem Keamanan SSH dari Serangan Brute Force Mengunakan Intrusion Prevention System pada Mikrotik. Arus Jurnal Sains dan Teknologi, 2(1), 116–122.

Utomo, A., dkk. (2024). Pengujian keamanan server SSH menggunakan metode *Brute force attack* dengan tools Hydra. *Prosiding SEMNASA AMIKOM Solo*, 45–52.

Utomo, B. R., Jati, N. H., Jati, A. K., Saputro, I. A., & Purwidiantoro, M. H. (2024). Analisis implementasi keamanan jaringan dengan Fail2Ban terhadap serangan brute force. *Prosiding Seminar Nasional AMIKOM Surakarta*, 2.

Zhang, L., Wang, Q., & Liu, Y. (2024). Research and application of firewall log and intrusion detection log data visualization system. *IET Information Security*, 18(4), 233–245.

LAMPIRAN

fail2ban_service.py

```
import subprocess
import re
JAILS = {
    "sshd-bruteforce": "Brute Force",
    "sshd-dictionary": "Dictionary Attack"
}
FAIL2BAN = "/usr/bin/fail2ban-client"
SUDO = "/usr/bin/sudo"
def run_fail2ban(*args):
    command = [SUDO, "-n", FAIL2BAN, *args]
    print("[FAIL2BAN COMMAND]", command)
    try:
        result = subprocess.run(
            command,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True,
            timeout=30,
            check=False
        )
        print("[FAIL2BAN RETURN CODE]", result.returncode)
        if result.stderr:
            print("[FAIL2BAN STDERR]", result.stderr.strip())
        if result.returncode != 0:
            return None
        return result.stdout
    except subprocess.TimeoutExpired:
        print("[FAIL2BAN TIMEOUT]", command)
        return None
    except Exception as e:
        print("[FAIL2BAN EXCEPTION]", repr(e))
        return None
def get_jail_status(jail):
    return run_fail2ban("status", jail)
def parse_value(status, pattern):
    if not status:
        return None
    match = re.search(pattern, status)
    if match:
        return match.group(1).strip()
    return None
def get_banned_ips(status):
    value = parse_value(status, r"Banned IP list:\s*(.*)")
    if not value:
        return []
    return value.split()
def get_jail_data():
    data = []
    for jail, attack_type in JAILS.items():
        print("[FAIL2BAN] Checking:", jail)
        status = get_jail_status(jail)
        if not status:
            print("[FAIL2BAN] No status:", jail)
            continue
        currently_failed = parse_value(status, r"Currently
failed:\s*(\d+)")
```

```
        total_failed = parse_value(status, r"Total failed:\s*(\d+)")
        currently_banned = parse_value(status, r"Currently
banned:\s*(\d+)")
        total_banned = parse_value(status, r"Total banned:\s*(\d+)")
        banned_ips = get_banned_ips(status)
        data.append({
            "jail": jail,
            "jenis": attack_type,
            "currently_failed": int(currently_failed or 0),
            "total_failed": int(total_failed or 0),
            "currently_banned": int(currently_banned or 0),
            "total_banned": int(total_banned or 0),
            "banned_ips": banned_ips
        })
    return data
def get_all_banned():
    attackers = []
    jail_data = get_jail_data()
    no = 1
    for jail in jail_data:
        for ip in jail["banned_ips"]:
            attackers.append({
                "no": no,
                "ip": ip,
                "jenis": jail["jenis"],
                "jail": jail["jail"],
                "status": "Blocked"
            })
            no += 1
    return attackers
```

BIODATA PENULIS



Penulis lahir di Batam pada 12 November 2003 dan merupakan anak pertama dari empat bersaudara. Penulis menempuh pendidikan di SDN 2 Tiban Indah, SMPN 3 Sungai Harapan, dan SMKN 4 TIBAN II dengan jurusan Teknik Kimia Industri. Setelah menyelesaikan pendidikan di SMKN 4 TIBAN II pada tahun 2022, penulis melanjutkan pendidikan tinggi di Institut Teknologi Batam (ITEBA) pada Program Studi Teknik Komputer, Fakultas Teknologi dan Informatika, dan terdaftar sebagai mahasiswa dengan Nomor Induk Mahasiswa (NIM) 2222026. Selama menempuh pendidikan di Program Studi Teknik Komputer, penulis aktif dalam kegiatan MBKM di Infinite Learning dengan program Hybrid Cloud & Redhat. Selain itu, penulis turut berpartisipasi dalam kegiatan magang di Sucofindo dan AP Premier sebagai IT Support. Selain aktif dalam kegiatan MBKM dan magang, penulis memiliki minat pada bidang Jaringan, khususnya Cyber security. Ketertarikan tersebut menjadi salah satu dasar dalam pengembangan dan penyusunan tugas akhir yang telah dilakukan.

ITEBA
Institut Teknologi Batam

FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI TEKNIK KOMPUTER

Jl Gajah Mada, Kompleks Vitka City, Tiban Ayu - Sekupang, Batam 29425, Kepulauan Riau - Indonesia
Telp: (0778) 3540555 / 3540666 WA: 0822 1000 3267 Web : iteba.ac.id Email: info@iteba.ac.id

FORM BIMBINGAN TUGAS AKHIR

Nama Mahasiswa : Azua Ramadhan
NIM : 22220256
Judul TA : Perancangan Sistem Keamanan Jaringan Server Linux Menggunakan Firewall dan Fail2Ban Berbasis Log Analysis Method

Pembimbing 1 : Zainul Munir, S.T., Me.TC
Pembimbing 2 : M. Abrar Masril, M.Kom

No	Hari/Tanggal	Permasalahan	Komentar / Saran Perbaikan	Paraf
1	23/2/26	judul ACC - Cayan Judul objtarget	layout	
2.	3/03/26	Perbaiki Penulis bab 1.	Typo, penulisan	
3		Cover, penulis typo	Revisi	
4		bab 1, 2 dl	Susun	
5		bab 1, 2, 3 dl Lgsg Sempro	Teori	
		Acc Sempro	dl	

ITEBA
Institut Teknologi Batam

FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI TEKNIK KOMPUTER

Jl Gajah Mada, Kompleks Vitka City, Tiban Ayu - Sekupang, Batam 29425, Kepulauan Riau - Indonesia
Telp: (0778) 3540555 / 3540666 WA: 0822 1000 3267 Web : Iteba.ac.id Email: info@iteba.ac.id

FORM BIMBINGAN TUGAS AKHIR

Nama Mahasiswa : **Azua Ramadhan** — **Pembimbing 1 : Zainul Munir, S.T., Me.TC**

NIM : **22220256** — **Pembimbing 2 : M. Abrar Masril, M.Kom**

Judul TA : **Perancangan Sistem Keamanan Jaringan Server Linux Menggunakan Firewall dan Fail2Ban Berbasis Log Analysis Method**

No	Hari/Tanggal	Permasalahan	Komentar / Saran Perbaikan	Paraf
1	25/02/2026	Penulisan Bab I	Revisi bab I	
2.	18/04/2026	Penulisan Bab II Penulisan BAB I	Revisi bab II ——— I	
3..	23/04/2026	Bab II kurang teori pendukung	Revisi Bab II Lagi 3	
4	04/05/2026	Tambahkan teori pendukung BaB II & BAB III metode	Revisi bab II & III Daftar Sempro	

ITEBA
Institut Teknologi Batam

FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI TEKNIK KOMPUTER

Jl Gajah Mada, Kompleks Vitka City, Tiban Ayu - Sekupang, Batam 29425, Kepulauan Riau - Indonesia
Telp: (0778) 3540555 / 3540666 WA: 0822 1000 3267 Web : Iteba.ac.id Email: info@iteba.ac.id

FORM BIMBINGAN TUGAS AKHIR

Nama Mahasiswa : Azua Ramadhan **Pembimbing 1 : Zainul Munir, S.T., Me.TC**

NIM : 22220256 **Pembimbing 2 : M. Abrar Masril, M.Kom**

Judul TA : Implementasi Sistem Keamanan Server Linux Menggunakan Firewall dan Fail2Ban Berbasis Log Analysis untuk Mitigasi Brute Force dan Dictionary attack pada Layanan SSH

No	Hari/Tanggal	Permasalahan	Komentar / Saran Perbaikan	Paraf
1		Bab 4	typo	
2		Perbaiki Alur	Cek bab 4	
3		Perbaiki Tgbr	cek sistem	
4		Simulasi perbaiki	Alur	
5		Bab 4	Perbaiki	

ITEBA
Institut Teknologi Batam

FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI TEKNIK KOMPUTER

Jl Gajah Mada, Kompleks Vitka City, Tiban Ayu - Sekupang, Batam 29425, Kepulauan Riau - Indonesia
Telp: (0778) 3540555 / 3540666 WA: 0822 1000 3267 Web : iteba.ac.id Email: info@iteba.ac.id

FORM BIMBINGAN TUGAS AKHIR

Nama Mahasiswa : Azua Ramadhan
NIM : 22220256
Judul TA : Implementasi Sistem Keamanan Server Linux Menggunakan Firewall dan Fail2Ban Berbasis Log Analysis untuk Mitigasi Brute Force dan Dictionary attack pada Layanan SSH

Pembimbing 1 : Zainul Munir, S.T., Me.TC
Pembimbing 2 : M. Abrar Masril, M.Kom

No	Hari/Tanggal	Permasalahan	Komentar / Saran Perbaikan	Paraf
1.	29/06/2026	Revisi bab 4	Revisi bab 4	
2	02/07/2026	~~Revisi~~ Revisi bab 4	Revisi bab 4	
3	07/07/2026	Revisi fail2Ban	Revisi bab 4	
4	13/07/2026	Daftar Pustaka	Daftar Pustaka	

ITEBA
Institut Teknologi Batam

FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI TEKNIK KOMPUTER

Jl Gajah Mada, Kompleks Vitka City, Tiban Ayu - Sekupang, Batam 29425, Kepulauan Riau - Indonesia
Telp: (0778) 3540555 / 3540666 WA: 0822 1000 3267 Web : iteba.ac.id Email: info@iteba.ac.id

FORM BIMBINGAN TUGAS AKHIR

Nama Mahasiswa : Azua Ramadhan
NIM : 22220256
Judul TA : Implementasi Sistem Keamanan Server Linux Menggunakan Firewall dan Fail2Ban Berbasis Log Analysis untuk Mitigasi Brute Force dan Dictionary attack pada Layanan SSH

Pembimbing 1 : Zainul Munir, S.T., Me.TC
Pembimbing 2 : M. Abrar Masril, M.Kom

No	Hari/Tanggal	Permasalahan	Komentar / Saran Perbaikan	Paraf
1		Bab 5 tabel	ok	
2		Bab 5 tipe dan penjelasan	Cek	
3		Bab 5 ok Bab 6 Kesimpulan	ok	
4		Layout Sidang ok	ok	

ITEBA
Institut Teknologi Batam

FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI TEKNIK KOMPUTER

Jl Gajah Mada, Kompleks Vitka City, Tiban Ayu - Sekupang, Batam 29425, Kepulauan Riau - Indonesia
Telp: (0778) 3540555 / 3540666 WA: 0822 1000 3267 Web : Reba.ac.id Email: info@iteba.ac.id

FORM BIMBINGAN TUGAS AKHIR

Nama Mahasiswa : Azua Ramadhan **Pembimbing 1 : Zainul Munir, S.T., Me.TC**

NIM : 22220256 **Pembimbing 2 : M. Abrar Masril, M.Kom**

Judul TA : Implementasi Sistem Keamanan Server Linux Menggunakan Firewall dan Fail2Ban Berbasis Log Analysis untuk Mitigasi Brute Force dan Dictionary attack pada Layanan SSH

No	Hari/Tanggal	Permasalahan	Komentar / Saran Perbaikan	Paraf
1	02/04/2026	(0 seranyan	Revisi bab 5	
2.	9/08/2026	(0 seranyan	Revisi bab 5	
3.	21/08/2026	(0 seranyan	Revisi bab 5	
4.	4/08/2026	Acc bab 5	Lanjut 6	
5	09/09/2026	daftar sidang	daftar sidang	